

v3.0

AI-Powered Distributed Endpoint Monitoring and Remote Scanning Platform

Complete System Documentation — every module, every connection, end to end.

This document explains the whole platform: the admin server, the client agents, the REST and WebSocket protocols that link them, the database, the analytics, the security model, and every way the pieces connect. Read the Connection Lifecycle (section 8) and How Everything Connects (section 15) for the full end-to-end picture.

Table of Contents

1. Executive Summary	10
1.1 The One-Sentence Summary	10
1.2 What This Document Covers	10
1.3 How to Read This Document	10
2. System Overview	10
2.1 What It Does	10
2.2 Who It Is For	11
2.3 Design Goals and Principles	11
2.4 Platform Scope	11
3. System Architecture	11
3.1 High-Level Architecture	11
3.2 Admin Server Components	12
3.3 Client Agent Components	12
3.4 Communication Channels	13
3.5 End-to-End Data Flow	13
3.6 Deployment Topologies	13
3.7 Networking and Ports	13
4. Technology Stack	14
5. Repository Layout	14
5.1 Two Runtimes, One Codebase	15
6. Quick Start	15
6.1 Prerequisites	15
6.2 Install	15
6.3 Start the Admin Server	15
6.4 Start a Client Agent	15
6.5 Approve the Client	16
6.6 What Happens Next	16
7. Core Concepts	16
7.1 Client, Agent, Device	16
7.2 Registration Key vs Hardware Fingerprint	16
7.3 Approval Workflow	16
7.4 Heartbeat and Health	16
7.5 Device States	16
7.6 Companies and Tenants	17
7.7 The Admin Self-Client	17
8. Client Connection Lifecycle	17
8.1 Connection Creation (Registration)	17

Registration resolution order	17
8.2 Approval	17
8.3 Ongoing Connection (Heartbeat + WebSocket)	18
8.4 Connection Deletion	18
8.5 Connection Settings and Reset	18
9. Scanner API Module (Core)	19
9.1 Core Data Models	19
Company, ClientGroup, Client, ScanResult	19
Settings and Audit	19
Organization and Asset Models	19
9.2 Agent-Facing Endpoints	20
9.3 Client Management Endpoints	20
9.4 Settings and Admin Endpoints	21
9.5 Authentication and Authorization	22
Five authentication mechanisms	22
RBAC permission classes	22
9.6 Middleware	22
9.7 Company / Tenant Isolation	23
9.8 Key Algorithms	23
Change detection (diff_utils.compute_scan_diff)	23
Account lockout (auth_utils.check_account_lock)	23
Asset lifecycle state machine	23
9.9 Management Commands (scanner_api)	23
10. Monitoring Module	23
10.1 Monitoring Data Models	23
10.2 Agent-Facing Endpoints	24
10.3 Device Management Endpoints	24
10.4 WebSocket Consumers	25
Agent channel: ws/agent/<agent_id>/	25
Dashboard channel: ws/dashboard/	25
10.5 Event Bus and Subscribers	25
10.6 Health Scoring	26
10.7 Change Detection	26
10.8 Alert Engine	26
10.9 Anomaly Detection	26
10.10 Feature Store	26
10.11 Predictive Analytics	26
10.12 Scheduled Scanning (APScheduler)	27
10.13 Security (HMAC + Rate Limiting)	27
10.14 Reports	27

11. Intelligence Module	27
11.1 Data Models	27
11.2 Alert Management	28
11.3 Notifications	28
11.4 Audit Logging	28
11.5 Report Engine	28
11.6 Key Endpoints	28
12. Maintenance Module	29
12.1 Data Models	30
12.2 Maintenance Lifecycle	30
12.3 License Management	30
12.4 Alerts	30
12.5 Key Endpoints	30
13. Organization and Asset Management	32
13.1 Locations	32
13.2 Departments	32
13.3 Employees	33
13.4 Assignments	33
13.5 Asset Lifecycle	33
13.6 Cross-Module Insights	34
14. Client Agent in Detail	34
14.1 Startup Flow	34
14.2 HTTP Communicator	35
14.3 WebSocket Client	35
14.4 Hardware Fingerprint	35
14.5 Scanner (Collected Data)	35
14.6 Metrics	36
14.7 Event Monitors	36
14.8 Event Dispatcher and Offline Queue	36
14.9 Offline Resilience	36
14.10 Background and Autostart Mode (Windows)	36
15. How Everything Connects (End-to-End Scenarios)	37
15.1 Onboarding a New Device	37
15.2 Real-Time Scan Command	37
15.3 Scheduled Scan Dispatch	37
15.4 Event Detection Pipeline	38
15.5 Alert Lifecycle	38
15.6 Offline Client Recovery	38

15.7 Change Detection and Notification	39
15.8 Serverless (Vercel) Flow	39
16. Security Architecture	39
16.1 Authentication Layers	39
16.2 Authorization (RBAC)	40
16.3 Agent Authentication Details	40
16.4 Tenant Isolation	40
16.5 HTTP Hardening	40
16.6 Secrets Management	40
16.7 Auditing	40
17. REST API Reference (Complete)	40
17.1 scanner_api: Registration, Communication, Scans	41
17.2 scanner_api: Settings, Admin, Activity	41
17.3 scanner_api: Authentication	42
17.4 scanner_api: Organization	43
17.5 scanner_api: Assets	44
17.6 monitoring: Agent and Device	45
17.7 monitoring: Alerts, Schedules, Reports	46
17.8 intelligence	46
17.9 maintenance	48
17.10 System / Diagnostics	49
18. WebSocket Protocol	50
18.1 Dashboard Channel (browser)	50
18.2 Agent Channel (agent)	50
18.3 Command Reference	50
18.4 Message Flow Guarantees	50
19. Event Types and the Event Bus	51
19.1 How a Published Event Is Handled	51
20. Database Schema Overview	51
20.1 Table Families	51
20.2 Key Relationships	52
20.3 Database Location	52
21. Reports and Exports	52
21.1 Monitoring Reports	52
21.2 Intelligence Report Engine	53
21.3 Maintenance and License Exports	53
22. Background Tasks and Scheduling	53
23. Deployment Guide	53

23.1 Self-Hosted (Local / VPS)	54
23.2 nginx Reverse Proxy with HTTPS	54
23.3 Vercel Serverless	54
23.4 Cloud Discovery	55
24. Building Executables	55
25. Configuration Reference	55
25.1 Environment Variables	55
25.2 Important Setting Keys	55
25.3 Client Configuration Files	56
25.4 Admin CLI Options	56
26. Operations and Administration	57
26.1 Management Commands	57
26.2 Reset Procedures	57
26.3 Running as a Service	57
27. Troubleshooting	57
28. Known Issues and Limitations	58
29. Appendix	58
29.1 Device Status Indicators	59
29.2 Scan Data Collected	59
29.3 Health Score Formula	59
29.4 Monitoring Thresholds	60
29.5 Ports	60
29.6 Glossary	60
30. Deep-Dive: REST API Worked Examples	60
30.1 Session login	61
30.2 JWT token endpoints	61
30.3 API key management	61
30.4 Client registration (core scanner_api)	61
30.5 Monitoring agent registration (HMAC)	62
30.6 Agent heartbeat (public variant used by the agent)	62
30.7 Scan submission and history	63
30.8 Scheduled scan management	63
30.9 Maintenance record creation	63
30.10 Intelligence alerts	64
30.11 Report generation	64
31. Deep-Dive: WebSocket Protocol and Message Catalog	64
31.1 Connection and auth handshake	64
31.2 Server-to-agent messages	65

31.3 Agent-to-server messages	66
31.4 Dashboard broadcast messages	66
31.5 Offline handling and recovery	67
32. Deep-Dive: Database Schema Reference	67
32.1 scanner_api: core and organization	67
32.1.1 Company (companies)	67
32.1.2 ClientGroup (client_groups)	68
32.1.3 Client (clients) - central agent record	68
32.1.4 ScanResult (scan_results)	68
32.1.5 AddonDevice (addon_devices)	69
32.1.6 ActivityLog (activity_logs)	69
32.1.7 Setting (settings)	69
32.1.8 AdministratorProfile (administrator_profiles)	69
32.1.9 AuditLog (audit_logs)	70
32.1.10 LoginHistory / LoginAttempt / DeviceFingerprint	70
32.1.11 Location (locations)	70
32.1.12 Department (departments)	71
32.1.13 Employee (employees)	71
32.1.14 EmployeeAssetAssignment (employee_asset_assignments)	72
32.1.15 OrgAuditLog (org_audit_logs)	72
32.1.16 Asset categories and vendors	72
32.1.17 Asset (assets) - largest table	72
32.1.18 AssetAssignment / AssetTransfer / AssetHistory / AssetDocument	73
32.1.19 ApiKey (api_keys)	74
32.2 monitoring: device intelligence	74
32.2.1 DeviceMonitoringInfo (monitoring_device_info)	74
32.2.2 HardwareInventory / SoftwareInventory	75
32.2.3 DeviceHeartbeat (monitoring_device_heartbeat)	75
32.2.4 DeviceMetrics (monitoring_device_metrics)	76
32.2.5 DeviceHistory (monitoring_device_history) - immutable	76
32.2.6 DeviceAlert (monitoring_device_alerts)	76
32.2.7 AgentVersion / AgentSecret	77
32.2.8 ScheduledScan / PendingScan / ScanScheduleLog	77
32.3 intelligence: alerts, insights, audit	78
32.3.1 Alert (intelligence_alerts)	78
32.3.2 AlertHistory / AlertRule	78
32.3.3 Notification / NotificationPreference	79
32.3.4 Report / ScheduledReport	79
32.3.5 AuditLogEntry / ComplianceLog / DashboardAnalytics / RetentionPolicy	80
32.4 maintenance: asset care, licenses, compliance	81
32.4.1 MaintenanceRecord (maintenance_records)	81
32.4.2 MaintenanceHistory / MaintenanceDocument	82
32.4.3 WarrantyRecord / DowntimeRecord	82

32.4.4 SoftwareLicense / LicenseAssignment / LicenseHistory	83
32.4.5 ComplianceRecord / MaintenanceAlert	84
32.5 Relationship map	84
33. Deep-Dive: Client Agent Source Walkthrough	85
33.1 main.py - entry point (1247 lines)	85
33.2 communicator.py - HTTP + WebSocket (400 lines)	85
33.3 config, discovery, runtime	86
33.4 identity: fingerprint and keys	86
33.5 scanner.py and metrics.py	87
33.6 events/ - monitors and dispatcher	87
33.7 Startup sequence	87
34. Deep-Dive: Admin Server Source Walkthrough	88
34.1 Project configuration (django_admin/)	88
34.2 scanner_api app (core)	88
34.3 monitoring app (device intelligence)	89
34.4 intelligence app (insights)	90
34.5 maintenance app (asset care)	91
34.6 Management commands and operations	91
35. Deep-Dive: Configuration and Environment Reference	92
35.1 Environment variables (admin/django_admin/settings.py)	92
35.2 Django settings groups	92
35.3 Database and storage modes	93
35.4 Client configuration	93
36. Deep-Dive: Event Bus, Change Detection and Intelligence Pipeline	93
36.1 Event catalog (EventType)	93
36.2 Subscribers (what each event does)	94
36.3 Health scoring (admin/monitoring/health.py)	94
36.4 Anomaly detection (anomaly_detection.py)	95
36.5 Predictive engine (predictive.py)	95
36.6 Change detection (change_detection.py)	95
36.7 Maintenance and license alert rules (maintenance/alerts.py)	95
36.8 Intelligence alert lifecycle (intelligence/alerts.py)	95
37. Deep-Dive: Reports, Analytics and Dashboards	95
37.1 Intelligence report types	96
37.2 Monitoring reports (PDF and CSV)	96
37.3 Maintenance exports and analytics	96
37.4 Dashboards and analytics endpoints	97
38. Deep-Dive: Troubleshooting Matrix	98

39. FAQ99

How does an agent get approved?

99

What is the difference between the two registration flows?

99

Why does the agent use HTTP heartbeats instead of WebSocket?

99

How are offline clients handled during scheduled scans?

99

Where is the scan diff computed?

99

Can the admin server run without a database?

99

What ports does the agent use?

99

How do I reset a locked admin account?

99

Why does the PDF say some WebSocket features are dormant?

99

40. Glossary99

41. Appendix: Quick Reference100

41.1 Ports and protocols

100

41.2 Key constants

101

41.3 HTTP status conventions

101

41.4 Where-is-it index

101

1. Executive Summary

System Scanner Pro is an AI-powered distributed endpoint monitoring and remote scanning platform. It combines a centralized **Django admin server** with lightweight **Python client agents** that run on every managed machine. Agents collect hardware and software inventory, stream live metrics, monitor system events (USB, files, processes, software), and execute scheduled or on-demand scans. The admin dashboard provides fleet-wide visibility with real-time WebSocket updates, change detection, alerts, predictive analytics, and report generation.

1.1 The One-Sentence Summary

One admin server centrally owns the fleet; many client agents report to it continuously; a two-channel protocol (REST for registration/heartbeats/scans, WebSocket for real-time commands and events) keeps both sides in sync; and an analytics layer turns the stream of inventory, metrics, and events into health scores, alerts, anomalies, and predictions.

1.2 What This Document Covers

- **Architecture** — how the admin server, client agent, discovery services, and deployment targets fit together (section 3).
- **Connection lifecycle** — registration, approval, heartbeat, WebSocket, deletion, and reset (section 8). This is the heart of the platform.
- **Every module** — scanner_api, monitoring, intelligence, maintenance, and the client agent (sections 9–14).
- **End-to-end flows** — concrete scenarios showing which component calls which endpoint and which event fires next (section 15).
- **Security** — JWT, API keys, sessions, HMAC agent authentication, tenant isolation (section 16).
- **Full API and WebSocket reference** — every endpoint and every message (sections 17–18).
- **Operations** — deployment, building executables, configuration, administration, troubleshooting (sections 23–27).

1.3 How to Read This Document

If you want to understand the **concept and how everything links**, read sections 2–8 first, then section 15 (scenarios). If you are integrating or extending the API, read sections 9–12 with sections 17–18 as the reference. If you operate or deploy the system, read sections 5, 6, 23–27. A glossary and appendices are at the end.

2. System Overview

2.1 What It Does

- **Inventories machines** — CPU, RAM, storage, motherboard, GPU, OS, network, peripherals, software, antivirus, Windows updates, and user accounts.
- **Keeps machines visible** — heartbeats every 30 seconds, online/offline status, health scores, and live dashboards.
- **Detects change** — diffs between consecutive scans and reports added/removed/modified hardware and software.
- **Monitors behaviour** — USB insertions, critical file changes, process starts/stops, and software install/uninstall, streamed as events.
- **Acts remotely** — admin can push scan_now and config_update commands to any agent in real time over WebSocket.
- **Predicts problems** — disk-full time, failure-risk scores, and 30/60/90-day capacity forecasts from the metric stream.
- **Reports and exports** — fleet, device, and alert reports in PDF and CSV, plus a full report engine with scheduled reports.

2.2 Who It Is For

- **IT administrators** — fleet inventory, asset tracking, and remote scan control.
- **Security teams** — USB/process/software change monitoring, unauthorized software and antivirus-removal alerts.
- **Asset / compliance managers** — asset lifecycle, licenses, warranties, maintenance, compliance and audit trails.
- **Operations teams** — predictive failure risk and capacity planning.

2.3 Design Goals and Principles

Distributed first	One admin server; any number of agents on Windows, Linux, or macOS.
Real-time first	WebSocket for commands and events; HTTP only as a fallback.
Self-healing client	Exponential backoff, offline queues, heartbeat watchdog, and re-discovery keep the agent alive without operator help.
Data isolation	Every admin server owns its own database; within a server, companies partition fleet data and every action is tenant-scoped.
Approval before trust	Devices are never silently accepted; an admin approves them, and re-approval is required after deletion.
Auditable	Activity logs, device history, and immutable audit logs record who did what.
AI-ready	A feature store and prediction pipeline are built in, ready to feed ML models.

2.4 Platform Scope

Admin server	Django 5.x + DRF + Channels; runs on any host that can run Python 3.10+.
Client agent	Python 3.10+ using only the standard library plus websockets and watchdog.
Databases	SQLite for local/self-hosted; PostgreSQL (Supabase) for cloud/serverless.
Browsers	Any modern browser; frontend is Django templates + vanilla JS + Bootstrap 5.3 + Chart.js.
Cloud targets	Vercel (serverless Django via api/index.py) with automatic HTTP-polling fallback.

3. System Architecture

The platform is split into two runtimes that talk over the network: the **admin server** (a Django application) and the **client agent** (a standalone Python process). A small set of auxiliary services supports them: UDP discovery, a cloud registry, and a serverless entry point.

3.1 High-Level Architecture

ADMIN SERVER (Django)
+-----+
scanner_api (REST core, auth, assets, org)
monitoring (agents, devices, WS, scheduler, AI)
intelligence (alerts, notifications, reports, audit)
maintenance (maintenance, warranties, licenses)
Event Bus (pub/sub) + APScheduler + Channels
SQLite OR PostgreSQL/Supabase
+-----+
TCP HTTP + WebSocket UDP (45000)

```

+-----+-----+ +-----+-----+ +-----+-----+
| CLIENT AGENT | | CLIENT AGENT | | UDP DISCOVERY |
| scanner | | ... (1..N) | | broadcast |
| monitors | | event queues | | listener |
| websocket | | backoff | | |
+-----+-----+ +-----+-----+ +-----+-----+
|
+-----+-----+ +-----+-----+
| CLOUD REGISTRY | | VERCEL (serverless Django, when used)|
| (Supabase table | | api/index.py, HTTP only, polls |
| server_registry) | | WebSocket is auto-disabled |
+-----+-----+ +-----+-----+

```

3.2 Admin Server Components

Component	Description
scanner_api	Core Django app. Owns the Client / ScanResult / Setting models, the REST API at /api/, authentication (session, cookie, JWT, API keys), middleware, tenant isolation, and the organization + asset-management modules.
monitoring	Monitoring app. Agent registration/heartbeat/inventory endpoints, device management, alerts, WebSocket consumers, the event bus, APScheduler-based scheduled scanning, anomaly detection, feature store, predictive analytics, health scoring, change detection, and PDF/CSV reports.
intelligence	AI/insight app. Cross-module alert management, notification centre, immutable audit logs, compliance logs, a report engine with scheduled reports, and a dashboard analytics snapshot.
maintenance	Maintenance app. Maintenance records, warranties, downtime, software licenses, license assignments, compliance records, and maintenance analytics.
django_admin	Project configuration: settings.py, url routing, ASGI (Channels WebSocket) and WSGI entry points.
Background threads	UDP discovery listener/broadcaster (port 45000), admin self-client heartbeat loop, admin self-scan, and cloud-registry refresh every 300 s.

3.3 Client Agent Components

Component	Description
main.py	Entry point. Resolves the admin URL, registers, waits for approval, runs the initial scan, starts the heartbeat loop, WebSocket client, event monitors, and the scheduled scan loop. Also handles background/autostart mode on Windows.
communicator.py	HTTP client (Communicator) with exponential backoff and an in-memory offline queue, plus the WebSocket client with auto-reconnect and Vercel detection.
config.py	Loads/saves client_config.json, prompts for the admin URL, and performs UDP discovery of the admin server.
discovery.py	Queries the cloud registry (Supabase server_registry table) to find the admin server's public address.
fingerprint.py	Computes a stable hardware fingerprint from motherboard serial, CPU id, disk serial, and MAC addresses, hashed with SHA-256.
key_manager.py	Creates/persists the registration key and device fingerprint in client_key.json.
scanner.py	Collects the full hardware/software inventory on Windows/Linux/macOS.
metrics.py	Collects live CPU, RAM, disk, uptime, and network reachability for heartbeats.

Component	Description
events/	USB, file, process, and software monitors plus the batching EventDispatcher with a disk-persisted offline queue.

3.4 Communication Channels

REST (HTTP)	Registration, approval status, heartbeat, scan submission, scan config, monitoring agent register/heartbeat/inventory, pending scans, reports, and all admin panel APIs. Uses JSON over HTTP(S).
WebSocket (agent)	ws://HOST/ws/agent/<agent_id>/ — authenticated with HMAC; the admin sends scan_now/config_update/ping commands; the agent streams heartbeats, scan results, and events.
WebSocket (dashboard)	ws://HOST/ws/dashboard/ — pushes fleet events, alerts, and health changes to open admin dashboards in real time.
UDP discovery	Port 45000. The admin broadcasts ADMIN_HERE and answers DISCOVER_ADMIN; agents listen and switch to the discovered admin.
Cloud registry	Supabase server_registry table. The admin publishes its public URL/IP; agents query it when they cannot reach the admin directly.

3.5 End-to-End Data Flow

A single lifecycle flows through every subsystem. The arrows show where each hop is handled:

1. Agent starts -> key_manager (client_key.json) -> config (admin URL)
2. Agent calls POST /api/register (scanner_api.RegisterClientView)
3. Agent polls POST /api/ping (scanner_api.PingClientView) [every 30 s]
4. Agent registers monitoring identity (monitoring.AgentRegisterView) -> AgentSecret
5. Agent sends POST /api/monitoring/agent/heartbeat-public (metrics)
6. Heartbeat -> DeviceHeartbeat row -> health score -> DeviceAlert checks
-> event bus events -> subscribers -> WebSocket broadcast to dashboards
7. Agent submits scans (POST /api/scan or /api/monitoring/agent/inventory)
-> ScanResult / HardwareInventory / SoftwareInventory
-> change detection -> event bus -> alerts + notifications
8. Admin pushes a command: scheduler or API -> notify_agent() -> WebSocket group
agent_<id> -> AgentConsumer.send_command -> agent runs scan_now
9. Offline agent: schedules create PendingScan -> agent pulls on reconnect
via GET /api/monitoring/agent/pending-scans

3.6 Deployment Topologies

Self-hosted (single host)	Admin server on a LAN host bound to 0.0.0.0:80; agents on the same LAN use UDP auto-discovery. SQLite database.
VPS / dedicated	Admin behind nginx with HTTPS; agents use the public domain/IP; optional cloud-registry registration; SQLite or PostgreSQL.
Vercel serverless	api/index.py adapts Django to serverless. PostgreSQL/Supabase required. No WebSocket, no UDP, no APScheduler; agents fall back to HTTP polling.
Multi-tenant	One server, many companies; every list/detail view is company-scoped and the URL prefix /<user>-<company>/ reinforces isolation.

3.7 Networking and Ports

Method	Endpoint	Purpose
80 / CUSTOM (TCP)	Web + API + WebSocket	Admin dashboard, REST API, and WebSocket upgrade.
443 (TCP)	HTTPS	Production TLS (nginx or Vercel).
45000 (UDP)	Auto-discovery	Admin broadcasts ADMIN_HERE and listens for DISCOVER_ADMIN.

4. Technology Stack

Backend	Django 5.x, Django REST Framework 3.15, Django Channels 4.x (WebSocket), APScheduler 3.x (scheduling).
Database	SQLite (default/self-hosted), PostgreSQL / Supabase (production/serverless) via dj-database-url + psycopg2.
Authentication	Django sessions, signed cookie fallback, JWT (PyJWT), API keys (SHA-256 hashed), HMAC-SHA256 for agent traffic.
WebSocket	Channels InMemoryChannelLayer (capacity 1000); Redis recommended for horizontal scale but not required.
Frontend	Django templates, vanilla JavaScript, Bootstrap 5.3, Chart.js.
Reports	ReportLab (PDF), Python csv and openpyxl (XLSX), qrcode + Pillow (QR codes).
Client agent	Python 3.10+ standard library, websockets, watchdog.
Build	PyInstaller 6.x (onefile executables for admin and client).
Serverless	Mangum (WSGI to Lambda) via api/index.py on Vercel.
Cloud registry	Supabase REST (PostgREST) server_registry table.

5. Repository Layout

Everything lives in one repository. The layout below is the map used throughout this document.

```

admin-client/
|-- admin/ # Admin server (Django)
| |-- main.py # Entry point: migrate, superuser, threads, runserver
| |-- manage.py # Django management commands
| |-- runtime.py # Frozen/data-dir helpers (packaged builds)
| |-- scanner.py # Local machine scan (admin self-scan)
| |-- scanner_api/ # Core app: models, views, auth, middleware, assets/org
| |-- monitoring/ # Agents, devices, WS, scheduler, AI, reports
| |-- intelligence/ # Cross-module alerts, notifications, audit, reports
| |-- maintenance/ # Maintenance, warranties, downtime, licenses
| |-- django_admin/ # settings.py, urls.py, asgi.py, wsgi.py
| |-- templates/ static/ # Admin panel HTML/JS/CSS
| `-- data/ # SQLite DB + installer artifacts (dev)
|-- client/ # Client agent (standalone Python)
| |-- main.py # Agent entry point
| |-- communicator.py # HTTP + WebSocket transport
| |-- config.py # Config + admin URL prompt + UDP discovery
| |-- discovery.py # Cloud registry discovery
| |-- fingerprint.py # Hardware fingerprint
| |-- key_manager.py # Registration key + fingerprint storage
| |-- scanner.py # Hardware/software inventory collection

```

```
| |-- metrics.py # Live CPU/RAM/disk metrics
| |-- events/ # USB/file/process/software monitors + dispatcher
| `-- scans/ # Local scan result backups
|-- api/index.py # Vercel serverless WSGI bootstrap
|-- build_client.py # PyInstaller build script (client + admin)
|-- deploy_vercel.py # Manual Vercel -> cloud-registry registration
|-- setup_cloud_discovery.sql # Supabase server_registry schema
|-- vercel.json runtime.txt # Vercel config
|-- requirements.txt .env.template
|-- docs/ # This document + generator
`-- .github/workflows/ # Hourly cloud-registry safety-net workflow
```

5.1 Two Runtimes, One Codebase

The admin folder and client folder are independent Python runtimes. The admin uses the full Django dependency stack from requirements.txt. The client deliberately depends on almost nothing: it uses the Python standard library for HTTP (urllib) and system probing, and optionally uses websockets and watchdog. This keeps the installed agent small, reliable, and easy to freeze into a single .exe with PyInstaller.

6. Quick Start

6.1 Prerequisites

- Python 3.10 or newer on the admin host and on every client machine.
- Network access between agents and the admin (TCP 80/custom for HTTP+WS, UDP 45000 for discovery).

6.2 Install

```
cd admin-client
pip install -r requirements.txt
```

6.3 Start the Admin Server

```
python admin/main.py
# First run prompts for the bind IP (0.0.0.0 for all interfaces), then:
# - runs migrations (creates admin/data/scanner.db)
# - creates the default admin user admin / admin123
# - creates the admin self-client
# - starts UDP discovery on port 45000 (listen + broadcast)
# - registers the public IP in the cloud registry
# - serves the dashboard on port 80
# Dashboard: http://localhost
# Login: admin / admin123
```

6.4 Start a Client Agent

```
python client/main.py # interactive: enter admin URL
python client/main.py http://192.168.1.100:80 # or pass it directly
# On first run the agent:
# 1. Generates registration key + hardware fingerprint (client_key.json)
# 2. Resolves the admin URL (argument / env / cloud / UDP / prompt)
# 3. Registers (POST /api/register) and waits for approval
# 4. Runs the initial hardware scan and submits it
# 5. Registers its monitoring identity (AgentSecret)
# 6. Starts the 30-second heartbeat loop
# 7. Starts the WebSocket client (if supported)
# 8. Starts USB/file/process/software event monitors
```

6.5 Approve the Client

- Open the dashboard (<http://localhost>) and find the new client with a yellow Pending dot.
- Click Approve (single or bulk). Approval flips `approved=True` and status to online.
- If auto-approve is enabled in Settings, approval is automatic.

6.6 What Happens Next

After approval the agent reports heartbeats and metrics every 30 seconds, submits scheduled scans (default every hour), streams detected events, and can receive `scan_now` / `config_update` commands. The dashboard and monitoring pages update live through WebSocket. Section 15 walks through these flows in detail.

7. Core Concepts

7.1 Client, Agent, Device

Client	The server-side record of a machine: <code>registration_key</code> , <code>fingerprint</code> , <code>approval</code> , <code>status</code> , <code>last_seen</code> , <code>scan config</code> , <code>owner</code> , and <code>company</code> .
Agent	The client-side process running on a machine. It owns the registration key and fingerprint and talks to the admin server.
Device (monitoring)	The monitoring view of a client (<code>DeviceMonitoringInfo</code>), carrying monitoring status, health, IP, OS, agent version, and heartbeat counters.
Monitoring agent	The agent's identity in the monitoring module: an <code>agent_id</code> (UUID) paired with an HMAC secret (<code>AgentSecret</code>).

7.2 Registration Key vs Hardware Fingerprint

The **registration key** is an 8-character random identifier generated on first run and persisted in `client_key.json`. It is the address used by every HTTP call. The **hardware fingerprint** is a SHA-256 hash (truncated to 16 hex chars) of motherboard serial, CPU id, disk serial, and MAC addresses. It survives IP changes, hostname changes, and even re-installs, which lets the server re-link a re-registering machine to its existing record.

7.3 Approval Workflow

New devices are pending until an admin approves them (unless auto-approve is on). Deletion is a soft delete; a deleted device that pings again is resurrected as **pending** and must be approved again. This is a strict trust model: no device is ever silently re-admitted.

7.4 Heartbeat and Health

Every 30 seconds the agent pings the server. The server refreshes `last_seen`, marks the client online, stores a `DeviceHeartbeat` with CPU/RAM/disk metrics, recomputes the health score, and evaluates threshold alerts. If no heartbeat arrives within the stale threshold (default 120 s) the client is marked offline.

7.5 Device States

Method	Endpoint	Purpose
PENDING	Registered, not approved	Yellow dot; appears in dashboards, needs approval.
ONLINE	Heartbeating now	Green dot; last heartbeat within the stale threshold.
OFFLINE	No heartbeat	Red dot; last heartbeat beyond the stale threshold.

Method	Endpoint	Purpose
BLOCKED	Admin revoked	Purple dot; agent secrets deactivated, re-approval impossible until unblocked.
MAINTENANCE / INACTIVE	Operator set	Used by the monitoring module for administrative statuses.

7.6 Companies and Tenants

A Company is the tenant entity. Every admin user is bound to a company through an AdministratorProfile. Clients, settings, groups, locations, departments, employees, and assets all carry a company foreign key, and every list/detail API is company-scoped. The UI enforces this with a URL prefix: /<user>-<company>/.

7.7 The Admin Self-Client

The admin server also registers itself as a client (key ADMIN-<hostname>) so it appears on its own dashboard, keeps a heartbeat loop alive, and can run local scans of its host machine. Its key is stored in the Setting admin_client_key and its identity is skipped from fingerprint re-linking to avoid hijack.

8. Client Connection Lifecycle

The connection lifecycle is the core of the platform. A machine moves through **creation** (registration) → **approval** → **monitoring** (heartbeat + WebSocket) and finally **deletion**. Both sides coordinate through a well-defined REST + WebSocket contract. This section is the reference for how a machine becomes part of the fleet.

8.1 Connection Creation (Registration)

On first run the client generates a registration key and hardware fingerprint (persisted to client_key.json), resolves the admin URL, and calls POST /api/register. The server logic lives in admin/scanner_api/views.py (RegisterClientView).

```
POST /api/register
{
  "registration_key": "Q69REJ58", # unique per installation
  "hostname": "DESKTOP-7X",
  "platform": "Windows",
  "client_version": "1.6.1",
  "device_fingerprint": "3BB3C334AF6021BF" # motherboard+CPU+disk+MAC hash
}
200 -> {"status": "ok", "approved": true|false, "auto_approved": ...}
201 -> created
403 -> key marked deleted (reinstall required)
```

Registration resolution order

- **Key exists and is deleted** → the row is re-surfaced as pending; it is never auto-approved. The device must be approved again.
- **Key exists and is active** → hostname/platform/version/last_seen/IP are refreshed; approval is kept only when auto-approve is on, and if the fingerprint changed the approval is reset to pending (prevents stolen-key reuse).
- **Key is new but the fingerprint already exists** → the existing device row is re-keyed to the new registration key (survives reinstalls). The admin self-client is excluded from this re-link.
- **Otherwise** → a brand-new Client row is created as pending (or online if auto-approve is enabled).

Newly registered clients inherit the company and owner of the admin server's self-client, so fleet data stays partitioned per company from the moment of registration.

8.2 Approval

Unless auto-approve is enabled, a registered client stays pending until an admin approves it. Approval flips approved=True, sets status online, and assigns ownership.

Method	Endpoint	Purpose
POST	/api/approve	Approve a single client by registration_key.
POST	/api/approve-multiple	Bulk approve a list of registration_keys.
POST	/api/monitoring/devices/<uuid>/approve	Approve via the monitoring module.
GET	/api/clients/<key>/status	Client-side poll: approved or pending.

Auto-approve is controlled by the auto_approve Setting (Settings page or PUT /api/settings). On Vercel it is forced to false so every device requires manual approval.

8.3 Ongoing Connection (Heartbeat + WebSocket)

Once approved, the client maintains the connection through two parallel channels:

- **HTTP heartbeat** — every 30 seconds POST /api/ping refreshes last_seen, last_ip, hostname, and version and marks the client online. It carries CPU/RAM/disk metrics (via /api/monitoring/agent/heartbeat-public), returns trigger_scan when the admin asked for one, and flushes the agent's offline queue.
- **WebSocket channel** — ws://HOST/ws/agent/<agent_id>/ authenticated with HMAC signature + timestamp. Used for real-time commands (scan_now, config_update) and event streaming. Falls back to HTTP polling when the upgrade is rejected (for example on Vercel).

Offline behaviour: if the admin is unreachable the client applies exponential backoff with jitter, persists events to a disk queue, and replays them on reconnect. The heartbeat watchdog restarts a crashed heartbeat thread automatically. A device is flagged offline when no heartbeat arrives within the stale threshold (default 120 s).

8.4 Connection Deletion

Deletion is a **soft delete**: the server sets deleted=True so the device vanishes from dashboards while its historical scans and audit trail remain. Deletion also publishes a DEVICE_DELETED event on the event bus and records an ActivityLog entry.

Method	Endpoint	Purpose
DELETE	/api/clients/<key>	Soft-delete a single client.
POST	/api/clients/delete-multiple	Bulk soft-delete a list of registration_keys.
POST	/api/monitoring/devices/<uuid>/block	Block a device (revoke connection).

After deletion, a re-registration attempt with the same key is rejected with 403 ("Client has been removed. Reinstall required."). The fingerprint re-registration path is ignored once deleted=True, so the operator must un-block or re-register deliberately. Deletion is not a hard purge; for a permanent wipe, delete the row directly or do a full factory reset (section 26).

8.5 Connection Settings and Reset

Method	Endpoint	Purpose
GET	/api/settings/connection	Return stored admin server URL + connection token.
POST	/api/settings/connection	Explicitly generate a new URL + token.
PUT	/api/settings/connection	Regenerate only the connection token.

- **Reset admin bind IP** — python admin/main.py --reset.

- **Reset a client admin URL** — delete client_config.json and re-run, or edit admin_url.
- **Re-register a client** — delete client_key.json; a new key is generated and the fingerprint re-links the device to the existing record.
- **Full reset** — delete the database plus client key/config files, then restart both sides.

9. Scanner API Module (Core)

scanner_api is the heart of the admin server. It owns the fleet data models, the agent-facing registration/heartbeat/scan endpoints, all authentication, the middleware stack, and the organization + asset-management modules.

9.1 Core Data Models

Company, ClientGroup, Client, ScanResult

Field	Type	Purpose
Company	Model	The tenant entity. Unique name + slug; isolates fleet data.
ClientGroup	Model	User-defined grouping for clients (company-scoped).
Client	Model	A registered device: registration_key (unique), hostname, platform, status (pending/online/offline), approved, last_seen, last_ip, device_fingerprint, scan_interval, scan_enabled, scan_requested, owner (User), company, group, tags, purchase/vendor/warranty fields, notes, deleted (soft-delete). is_stale property uses max(scan_interval*2, stale_threshold_seconds).
ScanResult	Model	One scan payload per client (JSON scan_data); source of change detection. Indexed [client, -created_at].
AddonDevice	Model	Secondary device attached to a client (name, serial, cost, category).

Settings and Audit

Field	Type	Purpose
Setting	Model	Key/value global or per-company settings; class helpers Setting.get/set.
ActivityLog	Model	Auditable admin actions: register, approve, scan, scan_request, delete, update, login, setting_change.
AuditLog	Model	Immutable security trail: login success/failure, logout, password changed, and other security events.
LoginHistory	Model	Per-user login sessions with browser/OS/location and duration.
LoginAttempt	Model	Failed-login counting for account lockout.
AdministratorProfile	Model	1:1 with User; company binding, timezone, currency, date format, dashboard default, notification flags, MFA fields.
DeviceFingerprint	Model	Trusted browser/device fingerprint registry for a user.

Organization and Asset Models

Field	Type	Purpose
Location	Model	Office/building/floor/room/address; company-scoped, status Active/Archived/Closed.
Department	Model	Company department with head, budget, and location.
Employee	Model	Employee with code, email, department, designation, manager, location, status.

Field	Type	Purpose
EmployeeAssetAssignment	Model	Tracks which Client (device) is assigned to which employee.
OrgAuditLog	Model	Immutable audit of employee/department/location changes.
AssetCategory	Model	Category tree for assets.
AssetVendor	Model	Vendor master data.
Asset	Model	The asset: auto asset_id AST#####, category, manufacturer/model, QR/barcode, financial fields, warranty block, 13-state lifecycle, optional client link, assigned_to employee.
AssetAssignment	Model	Asset → employee assignment record.
AssetTransfer	Model	Transfer of an asset between employees/departments/locations.
AssetHistory	Model	Immutable, append-only history of asset actions with JSON snapshots.
AssetDocument	Model	Base64-stored documents attached to an asset.
ApiKey	Model	API key with SHA-256 key_hash, rate limit, allowed IPs, expiry, last_used.

9.2 Agent-Facing Endpoints

Method	Endpoint	Purpose
POST	/api/register	Register / refresh a client (section 8.1).
POST	/api/approve	Approve a single client.
POST	/api/approve-multiple	Bulk approve clients.
POST	/api/ping	Heartbeat; returns trigger_scan when requested; resurrects soft-deleted clients as pending.
POST	/api/scan	Submit a scan result; updates client OS/CPU/RAM.
POST	/api/scan/local	Scan the admin server machine (background thread).
POST	/api/scan/all	Set scan_requested=True on all approved clients.
GET	/api/health	Lightweight health check (no DB).
GET	/api/admin-client	Keeps the admin self-client online (dashboard poll).
POST	/api/supabase/register	Registers the Vercel URL in the cloud registry (service-key auth).

9.3 Client Management Endpoints

Method	Endpoint	Purpose
GET	/api/clients	List non-deleted clients (tenant-scoped).
GET	/api/clients/<key>/status	Approval status for agent polling.
GET	/api/clients/<key>	Full detail + scan_changes diff between last two scans.
DELETE	/api/clients/<key>	Soft-delete a client.

Method	Endpoint	Purpose
POST	/api/clients/delete-multiple	Bulk soft-delete.
PUT	/api/clients/<key>/manual	Update manual fields.
GET / POST	/api/clients/<key>/addons	List / add addon devices.
DELETE	/api/clients/<key>/addons/<id>	Remove an addon device.
GET / PUT	/api/clients/<key>/scan-config	Read / update scan interval and enabled.
POST	/api/clients/<key>/scan-now	Trigger an immediate scan.
GET	/api/clients/<key>/scan-results	Latest scan result.
GET	/api/scan/history	Searchable/filterable scan history.
GET	/api/admin/scan-changes	Per-client scan diffs.
GET	/api/activity-log	Recent activity log.
GET / POST	/api/groups	List / create client groups.
DELETE	/api/groups/<id>	Delete a client group.

9.4 Settings and Admin Endpoints

Method	Endpoint	Purpose
GET / PUT	/api/settings	Global settings: auto_approve, stale_threshold_seconds, scan_all_interval, admin_client_key.
GET / PUT	/api/settings/organization	Organization profile.
GET / PUT	/api/settings/security	Security settings.
GET / PUT	/api/settings/notifications	Notification settings.
GET / POST / PUT	/api/settings/connection	Connection URL + token management.
GET / PUT	/api/settings/dashboard	Dashboard widget settings.
GET / POST	/api/admin/users	List / create admin users.

Method	Endpoint	Purpose
DELETE	/api/admin/users/<id>	Delete an admin user.
GET	/api/admin/stats	System statistics.

9.5 Authentication and Authorization

Five authentication mechanisms

Django sessions	Primary for the admin panel. Login by username or email, account logout, remember-me (30-day) vs default 7-day sessions.
Signed cookie fallback	A scanner_auth cookie (TimestampSigner, salt scanner-auth-cookie, 30-day max age) restores the session on Vercel cold starts.
JWT (PyJWT)	HS256, access 60 min / refresh 7 days, issuer system-scanner-pro. Endpoints token/obtain, token/refresh, token/verify. Role derived from superuser flag.
API keys	SHA-256 hashed key with rate limit, allowed IPs, expiry; header X-API-Key.
HMAC (agent)	Agent↔server messages signed with HMAC-SHA256 + timestamp; managed by the monitoring module (section 10.16).

RBAC permission classes

Field	Type	Purpose
ROLE_HIERARCHY	dict	super_admin=3, admin=2, viewer=1.
IsSuperAdmin	class	Superuser only.
IsAdmin	class	Admin role or above.
IsViewer	class	Any authenticated user.
IsAdminOrReadOnly	class	Writes require admin.
HasRole(role)	class	Requires a specific role.
CanManageDevices / Alerts / Schedules	class	Role-gated device, alert, and schedule management.
CanViewReports / CanGenerateReports	class	Role-gated report access.

9.6 Middleware

CookieAuthMiddleware	Injects the user from a valid scanner_auth signed cookie when anonymous.
CompanyPrefixMiddleware	Routes /<user>-<company>/ URL prefixes: resolves the prefix from the session/profile, strips it before dispatch, and redirects authenticated users to the prefixed URL.
SessionTimeoutMiddleware	Idle timeout from Setting.session_timeout_minutes (default 120 min); redirects to /login/?timeout=1.
SecurityHeadersMiddleware	Adds X-Content-Type-Options, X-Frame-Options DENY, X-XSS-Protection, and Referrer-Policy.
Whitenoise / CORS	Serves static files and allows cross-origin API calls.

9.7 Company / Tenant Isolation

- `get_user_company(request)` gets or creates the profile and auto-creates a Company named after the username if unassigned.
- `get_admin_owned_clients(request)`: superusers see their company's clients plus unowned; regular admins see their own clients plus unowned pending (so anyone can approve new devices).
- Every list view (clients, scans, activity, groups, locations, departments, employees, assets, assignments, org-audit, analytics, global search) is company-scoped.
- Settings, activity, audit, and login history all carry a company foreign key.

9.8 Key Algorithms

Change detection (`diff_utils.compute_scan_diff`)

Compares the last two `ScanResult.scan_data` dicts, ignoring metadata fields. Recursive dict diff, list diff keyed by name/model/serial/kb/device/mac/sid, and specialised peripheral and storage comparisons produce human-readable +, -, and arrow change lines. Exposed on the client detail page and `/api/admin/scan-changes`.

Account lockout (`auth_utils.check_account_lock`)

Counts failed LoginAttempts in the last `lock_duration_minutes` (default 30) versus `max_login_attempts` (default 5) and reports the remaining lock time.

Asset lifecycle state machine

Assets move through 13 states (Draft, Pending Approval, Approved, Purchased, Available, Assigned, Maintenance, Lost, Disposed, Archived, and others) with `VALID_TRANSITIONS` enforced on every status change; every transition writes an immutable `AssetHistory` entry.

9.9 Management Commands (`scanner_api`)

```
python admin/manage.py clear_data # wipe non-Django tables, recreate admin
python admin/manage.py scan_local # run a local scan of the admin host
python admin/manage.py stale_checker # mark stale clients offline (loop)
python admin/manage.py createsuperuser # create a superuser
```

10. Monitoring Module

The monitoring app is where agents connect and where fleet intelligence lives. It provides the agent registration/heartbeat/inventory APIs, device management, WebSocket consumers, the event bus, scheduled scanning, health scoring, change detection, anomaly detection, the feature store, predictive analytics, and reports. It is mounted at `/api/monitoring/` and its WebSocket routes at `/ws/`.

10.1 Monitoring Data Models

Field	Type	Purpose
DeviceMonitoringInfo	Model	One row per client; monitoring_status, health_level/score, IP/MAC, OS info, agent_version, heartbeat counters, tags, notes.
HardwareInventory	Model	Immutable hardware component snapshot (cpu/ram/storage/gpu/motherboard/network) with an MD5 fingerprint and scan_id batch.
SoftwareInventory	Model	Software snapshot rows; is_present flips to False when a package is removed in a later scan.
DeviceHeartbeat	Model	One row per heartbeat: cpu/ram/disk pct, disk free/total, network, uptime, response time.

Field	Type	Purpose
DeviceMetrics	Model	Hourly/daily rollups (avg/max/min cpu/ram/disk, health, uptime) for charts.
DeviceHistory	Model	Immutable per-device audit trail (categories: registration, status_change, hardware_change, software_change, health_change, security_event, alert_generated, admin_action, agent_update, remote_command).
DeviceAlert	Model	Alert records with type, severity, status (active/acknowledged/resolved/dismissed), and details JSON.
AgentVersion	Model	Published agent releases (version, release notes, download URL, mandatory flag, file hash).
AgentSecret	Model	HMAC credentials for an agent (agent_id unique, secret_key, device_fingerprint, is_active, last_used).
ScheduledScan	Model	Recurring/one-time scan schedules (interval/daily/weekly/monthly/once) with targeting.
PendingScan	Model	Offline scan queue; agents pull pending scans on reconnect.
ScanScheduleLog	Model	Per-execution log of scheduled scans (triggered/completed/failed/skipped).

10.2 Agent-Facing Endpoints

Method	Endpoint	Purpose
POST	/api/monitoring/agent/register	Register a monitoring agent; returns a secret_key. Matches an existing Client by fingerprint, then hostname, then client_key; creates one if needed.
POST	/api/monitoring/agent/heartbeat	HMAC-authenticated heartbeat (X-Agent-ID, X-Signature, X-Timestamp); creates DeviceHeartbeat, computes health, checks alerts.
POST	/api/monitoring/agent/heartbeat-public	Public heartbeat using only registration_key (no HMAC); used by the standard client.
POST	/api/monitoring/agent/inventory	HMAC-authenticated full hardware + software inventory snapshot; runs change detection.
GET	/api/monitoring/agent/version-check	Check for a newer agent version.
GET/POST	/api/monitoring/agent/pending-scans	Fetch queued scans; acknowledge executed/failed.

10.3 Device Management Endpoints

Method	Endpoint	Purpose
GET	/api/monitoring/dashboard	Fleet aggregates: status counts, health distribution, alerts, 24h CPU/RAM/disk trends, 7-day health trend.
GET	/api/monitoring/trends	Configurable fleet trend series.
GET	/api/monitoring/devices	List devices with search/status/health/platform filters.
POST	/api/monitoring/devices/bulk	Bulk approve / block / maintenance / inactive.

Method	Endpoint	Purpose
GET /PUT	/api/monitoring/devices/<uuid>	Device detail / status update.
POST	/api/monitoring/devices/<uuid>/approve	Approve device (also approves the Client).
POST	/api/monitoring/devices/<uuid>/block	Block device and deactivate all AgentSecrets.
GET	/api/monitoring/devices/<uuid>/metrics	Raw heartbeat data points.
GET	/api/monitoring/devices/<uuid>/history	Immutable device history.
GET	/api/monitoring/devices/<uuid>/alerts	Device alerts.
GET	/api/monitoring/devices/<uuid>/hardware	Hardware inventory history.
GET	/api/monitoring/devices/<uuid>/software	Present software list.
GET	/api/monitoring/devices/<uuid>/heartbeats	Heartbeat history.
GET	/api/monitoring/alerts	Fleet-wide alert list.
POST	/api/monitoring/alerts/<uuid>/action	Acknowledge / resolve / dismiss an alert.
GET /POST	/api/monitoring/agent-versions	List / publish agent versions.
GET /PUT	/api/monitoring/settings/unauthorized-software	Read / update the unauthorized software blocklist.

10.4 WebSocket Consumers

Agent channel: ws/agent/<agent_id>/

The agent connects and sends an auth message {type:'auth', agent_id, secret, signature, timestamp}. The consumer validates agent_id + secret_key against an active AgentSecret and requires the timestamp to be within 300 s (anti-replay). On success the agent is added to the channel group agent_<agent_id> and marked online. Inbound messages: heartbeat, scan_result, event, status_update, pong. Outbound messages: auth_success/auth_failed, command (scan_now/config_update), ping, heartbeat_ack.

Dashboard channel: ws/dashboard/

Any open admin dashboard can connect (read-only if anonymous). The server pushes fleet events (device heartbeat, health changes, HW/SW changes, alerts, agent status) to the dashboard group in real time. Browsers can subscribe to a single device group to receive per-device updates.

10.5 Event Bus and Subscribers

monitoring/event_bus.py defines an in-process pub/sub singleton and an EventType enum covering hardware, software, health, alerts, device lifecycle, agent version, heartbeat, and scan events. monitoring/subscribers.py registers default handlers that:

- Create/update DeviceAlerts and notify all admins on hardware/software/offline/deleted/version changes (severity-gated).

- Create in-app notifications for all hardware/software changes.
- Broadcast every event to the dashboard WebSocket group and to the device group.
- Write immutable DeviceHistory entries for each event category.

The event bus is the glue between the monitoring module and the dashboard: a change detected anywhere (agent inventory, scheduler, heartbeat) is published once and every subscriber reacts in order, so dashboards, alerts, history, and notifications all stay consistent.

10.6 Health Scoring

`calculate_health_score(heartbeat, software)` computes a 0–100 score and a level (healthy ≥ 80 , warning ≥ 50 , critical below). Weights: CPU 25%, RAM 25%, disk 20%, connectivity 15%, software health 15%. CPU/RAM lose points above 70% and floor out beyond 85%; disk falls above 80% and floors at 95%; connectivity is 100 when the agent can reach the internet; software health is 100 when an antivirus product is present, 60 otherwise. A level change publishes `HEALTH_LEVEL_CHANGED`.

10.7 Change Detection

`monitoring/change_detection.py` compares hardware inventory (by MD5 component fingerprints) and software snapshots. Hardware changes are added/removed/modified with per-component severity (storage is critical, motherboard/cpu warning, ram/gpu/network info). Software changes are added/removed/version_changed; names on the unauthorized list produce an unauthorized (warning) change, and a removed antivirus produces `antivirus_removed` (critical). Each change becomes an event-bus event and therefore an alert, notification, history entry, and dashboard broadcast.

10.8 Alert Engine

`monitoring/alerts.py` runs heartbeat-based checks: `high_cpu`, `high_ram`, and `low_disk` (thresholds from Settings: `alert_cpu_threshold` 90, `alert_ram_threshold` 90, `alert_disk_threshold` 95, `alert_disk_free_gb` 5; disk becomes critical at 98% or under 2 GB free). `check_offline_alerts` creates `device_offline` alerts at 300/900/1800 seconds and resolves them when the device returns. Alerts have a lifecycle: active → acknowledged → resolved, or dismissed, tracked in `DeviceAlert.status`.

10.9 Anomaly Detection

Threshold	Static rules; confidence 0.95. CPU/RAM 85% warning / 95% critical; disk 90% warning / 98% critical; disk free 5 GB warning / 2 GB critical.
Z-score	Requires 10+ history points; flags $ z > 2.5$; critical if $z > 4$, warning if $z > 3$; confidence rises with z .
IQR	Requires 20+ points; outliers beyond $Q1 - 1.5 \times IQR$ / $Q3 + 1.5 \times IQR$; info severity.
Trend	Requires 20+ points; flags rapid changes $> 50\%$ between the last 5 and last 20 observations.

10.10 Feature Store

`monitoring/feature_store.py` maintains a per-device in-memory feature cache for future ML models. `extract_features()` builds performance features (resource pressure, high/low flags), resource features (cpu-ram correlation, disk-io pressure, uptime, needs_reboot), connectivity flags, software features (software count, antivirus presence, remote-access tools), hardware features, and temporal features (hour/day/weekend/business-hours, deltas and rates). `get_feature_matrix()` and `export_for_training()` produce training-ready matrices.

10.11 Predictive Analytics

Disk-full time	Linear regression on disk usage trend; $\text{hours_to_full} = (100 - \text{current})/\text{slope}$. URGENT under 24 h, schedule cleanup under 168 h, plan expansion under 720 h.
----------------	--

Failure risk	Weighted 0–100 score from resource pressure (30), uptime (15), CPU volatility, and declining health (25). Critical >70, warning >40.
Capacity needs	Linear-regression forecast at 30/60/90 days; >95% means add capacity, >80% means plan expansion.

10.12 Scheduled Scanning (APScheduler)

monitoring/scheduler.py wraps APScheduler. Every enabled ScheduledScan maps to a job. When a job fires it resolves its target clients (all approved, or a specific list, optionally filtered by platform), splits them into online and offline, and:

- **Online** → sends a live scan_now command to the agent_<id> WebSocket group via notify_agent(), and logs ScanScheduleLog(status='triggered').
- **Offline** → creates a PendingScan row and logs ScanScheduleLog(status='skipped'). The agent pulls its pending scans on reconnect via GET /api/monitoring/agent/pending-scans and acknowledges them.

Schedule CRUD is exposed under /api/monitoring/schedules (list/create/update/toggle/status/pending/history). On Vercel the scheduler does not run (no persistent process); external cron can call the schedule endpoints instead.

10.13 Security (HMAC + Rate Limiting)

monitoring/security.py provides generate_api_secret() (64 hex chars), compute_signature() (HMAC-SHA256 over the raw request body), verify_signature() with constant-time comparison, and verify_timestamp() (X-Timestamp within 300 s, anti-replay). authenticate_agent() resolves X-Agent-ID to an active AgentSecret and verifies the signature. A per-key in-memory RateLimiter (60 requests/minute for heartbeats) protects the endpoints. Blocking a device deactivates all of its AgentSecrets, so its HMAC identity is revoked immediately.

10.14 Reports

monitoring/reports.py generates fleet, device, and alert reports. PDFs use ReportLab (A4, branded tables); CSV exports use the csv module. Endpoints under /api/monitoring/reports/ (section 21 covers report content in detail).

11. Intelligence Module

The intelligence app is the cross-module insight layer. It unifies alerts from every module (asset, monitoring, maintenance, license, security, compliance, system), manages in-app notifications, writes an immutable audit trail, evaluates compliance, and provides a report engine with scheduled reports. It is mounted at /api/intelligence/.

11.1 Data Models

Field	Type	Purpose
Alert	Model	Unified alert with module, severity (information/warning/critical/emergency), category, status, escalation_level (0–3), dedup_hash, assigned_user, resolved_time.
AlertHistory	Model	Immutable per-alert history of status/severity changes.
AlertRule	Model	User-defined rules (threshold_gt/lt>equals/contains) with suppress and auto-resolve windows.
Notification	Model	In-app notification to a user; status unread/read/archived.
NotificationPreference	Model	Per-user gates: email/in-app toggles, per-severity and per-module flags, frequency, quiet hours.
Report	Model	Generated report (25 report types) stored as base64 with format, filters, row count, status.
ScheduledReport	Model	Recurring report definition (daily/weekly/monthly/quarterly) with recipients and retention policy.

Field	Type	Purpose
AuditLogEntry	Model	Immutable cross-module audit entry (module, action, object, old/new values, severity, description).
ComplianceLog	Model	Compliance check results against frameworks (iso_27001/itil/soc2/internal/gdpr).
DashboardAnalytics	Model	Periodic KPI snapshot for the dashboard.
RetentionPolicy	Model	Data retention per scope (alerts, notifications, reports, audit logs, compliance logs).

11.2 Alert Management

intelligence/alerts.py creates alerts with a SHA-256 dedup hash so repeated conditions update the existing alert instead of duplicating it. Alerts can be acknowledged, resolved, dismissed, and assigned. `escalate_alerts()` raises severity levels by age (emergency >1 h → level 3, critical >4 h → level 2, warning >24 h → level 1). `run_alert_checks()` generates alerts from business rules: expired/expiring warranties, overdue/due maintenance, expired/expiring/over-seat licenses, and stale clients.

11.3 Notifications

intelligence/notifications.py gates every notification against the user's NotificationPreference (in-app toggle, severity, module, quiet hours). `create_alert_notifications()` notifies every active user for a new alert. Users mark notifications read/archived; the dashboard bell shows the unread count.

11.4 Audit Logging

intelligence/audit.py is the central audit writer. It extracts the real client IP (X-Forwarded-For), browser user-agent, and device platform, and writes immutable AuditLogEntry rows. Convenience helpers cover login, logout, failed login, asset create/update/assign/delete, report download/generate, and settings changes.

11.5 Report Engine

intelligence/reports.py implements nine report types: asset_inventory, asset_assignment, expiring_licenses, upcoming_maintenance, device_health, compliance_report, monthly_summary, software_inventory, and audit_report, in CSV, Excel, or PDF. Every generated report is persisted as a Report row; exports stream the stored base64 file.

11.6 Key Endpoints

Method	Endpoint	Purpose
GET	/api/intelligence/dashboard	Analytics snapshot + alert/notification/audit summaries.
GET / POST	/api/intelligence/alerts	List / create alerts.
GET	/api/intelligence/alerts/<uuid>	Alert detail.
POST	/api/intelligence/alerts/<uuid>/action	Acknowledge / resolve / dismiss / assign.
GET	/api/intelligence/alerts/<uuid>/history	Alert history.
POST	/api/intelligence/alerts/bulk	Bulk alert actions.
POST	/api/intelligence/alerts/run-checks	Run the business-rule alert checks.

Method	Endpoint	Purpose
GET	/api/intelligence/alerts/export	Export alerts as CSV.
GET /POST	/api/intelligence/alerts/rules	List / create alert rules.
GET /PUT/DELETE	/api/intelligence/alerts/rules/<uuid>	Alert rule CRUD.
GET	/api/intelligence/notifications	User notifications + unread count.
POST	/api/intelligence/notifications/mark-all-read	Mark all read.
GET /PUT	/api/intelligence/notifications/preferences	Notification preferences.
POST	/api/intelligence/notifications/<uuid>/action	Read / archive.
GET /POST	/api/intelligence/reports	List / generate reports.
GET	/api/intelligence/reports/<uuid>	Report detail (logs download).
GET	/api/intelligence/reports/<uuid>/export	Download stored report file.
GET /POST	/api/intelligence/scheduled-reports	List / create scheduled reports.
GET /PUT/DELETE	/api/intelligence/scheduled-reports/<uuid>	Scheduled report CRUD.
GET	/api/intelligence/audit-logs	Audit log list with filters.
GET	/api/intelligence/audit-logs/export	Export audit logs (max 5000 rows).
GET /POST	/api/intelligence/compliance	Compliance log list / create.
GET	/api/intelligence/compliance/dashboard	Compliance rate by framework/status.
GET /POST	/api/intelligence/retention-policies	Retention policies.
GET /PUT	/api/intelligence/settings	Intelligence settings (escalation, retention).

12. Maintenance Module

The maintenance app manages the physical IT asset lifecycle: maintenance records, warranties, downtime, software licenses, license assignments, and compliance. It is mounted at /api/maintenance/.

12.1 Data Models

Field	Type	Purpose
MaintenanceRecord	Model	A maintenance job: auto maintenance_id MNT#####, type, status (Draft→...→Completed/Overdue), approval status, asset, department, vendor, technician, schedule dates, cost, priority, recurrence. is_overdue property.
MaintenanceHistory	Model	Immutable history of maintenance actions with JSON snapshots.
MaintenanceDocument	Model	Base64 documents attached to a maintenance record.
WarrantyRecord	Model	Warranty: auto WAR#####, asset, provider, coverage, contract, support contacts, cost, computed_status (Active/Expiring Soon/Expired/...).
DowntimeRecord	Model	Asset downtime windows with auto-computed duration and reason.
SoftwareLicense	Model	License: auto LIC#####, type (Per User/Per Device/Subscription/OEM/...), status, encrypted + masked key, seats purchased/used, purchase/expiration dates, cost, department.
LicenseAssignment	Model	License → Asset/Employee/Department assignment; tracks active and removed assignments.
LicenseHistory	Model	Immutable license history.
ComplianceRecord	Model	Compliance findings (license_expiration, seat_overuse, unauthorized_software, missing_license, compliance_violation).
MaintenanceAlert	Model	Maintenance/license/warranty alerts with severity and status.

12.2 Maintenance Lifecycle

Records start as Draft + Approval Pending. Approving moves them to Approved (or Cancelled). Status transitions are validated against a state map; entering In Progress auto-sets start_date; completing auto-sets completion_date and moves the linked asset back to Available. Completed records are immutable (no update/delete).

12.3 License Management

Licenses store the license key encrypted (license_key_encrypted) and display only a masked form (license_key_masked). Assigning a license to an asset/employee/department increments seats_used with a database atomic F() update; removing an assignment decrements it and is refused when the license is archived or seats are exhausted.

12.4 Alerts

maintenance/alerts.py's check_and_generate_alerts() creates maintenance_overdue, maintenance_due, warranty_expiring/expired, license_expiration (7-day critical / 30-day warning / 60-day info), license_expired, and license_seat_exhaustion alerts. It also flips warranty and license statuses to Expired when past due.

12.5 Key Endpoints

Method	Endpoint	Purpose
GET /POST	/api/maintenance/maintenance	List / create maintenance records.

Method	Endpoint	Purpose
GET /PUT/DELETE	/api/maintenance/maintenance/<uuid>	Maintenance CRUD.
POST	/api/maintenance/maintenance/<uuid>/status	Transition maintenance status.
POST	/api/maintenance/maintenance/<uuid>/approve	Approve / reject maintenance.
POST	/api/maintenance/maintenance/<uuid>/upload	Attach a document.
GET	/api/maintenance/maintenance/export	Export maintenance CSV.
GET /POST	/api/maintenance/warranties	List / create warranties.
GET /DELETE	/api/maintenance/warranties/<uuid>	Warranty detail / delete.
GET /POST	/api/maintenance/downtime	List / create downtime.
POST	/api/maintenance/downtime/<uuid>/end	Close an open downtime window.
GET /POST	/api/maintenance/licenses	List / create licenses.
GET /PUT/DELETE	/api/maintenance/licenses/<uuid>	License CRUD.
POST	/api/maintenance/licenses/<uuid>/archive	Archive a license.
POST	/api/maintenance/licenses/assign	Assign a license.
POST	/api/maintenance/licenses/assignments/<uuid>/remove	Remove a license assignment.
GET	/api/maintenance/licenses/export	Export licenses CSV.
GET /POST	/api/maintenance/compliance	List / create compliance records.
POST	/api/maintenance/compliance/<uuid>/action	Acknowledge / resolve / dismiss.
GET	/api/maintenance/alerts	Maintenance alerts.
POST	/api/maintenance/alerts/<uuid>/action	Alert action.
POST	/api/maintenance/alerts/check	Run alert checks.

Met hod	Endpoint	Purpose
GET	/api/maintenance/dashboard	Maintenance KPIs and distributions.
GET	/api/maintenance/analytics/cost-trend	Monthly cost trend.
GET	/api/maintenance/analytics/vendor-performance	Vendor completion/quality metrics.
GET	/api/maintenance/analytics/failure-rate	Asset failure rates.
GET	/api/maintenance/analytics/downtime	Downtime analytics.
GET	/api/maintenance/analytics/license-dashboard	License KPIs.
GET	/api/maintenance/analytics/license-utilization	Per-license utilization.
GET	/api/maintenance/analytics/license-cost	License cost analysis.

13. Organization and Asset Management

These modules live inside scanner_api and give the platform its IT-asset-management face. They connect employees and departments to scanned devices and to tracked assets.

13.1 Locations

Met hod	Endpoint	Purpose
GET /POST	/api/locations	List / create locations.
GET /PUT	/api/locations/<uuid>	Location CRUD (Closed is read-only).
POST	/api/locations/<uuid>/delete	Delete (blocked while assets are active).
POST	/api/locations/<uuid>/archive	Archive a location.
GET	/api/locations/<uuid>/dashboard	Location dashboard.
GET /POST	/api/locations/<uuid>/export	Export locations CSV.

13.2 Departments

Departments auto-seed eight defaults per company. Employees and assets link to departments, so a department cannot be deleted while it still owns either.

Met hod	Endpoint	Purpose
GET /POST	/api/departments	List / create departments.

Met hod	Endpoint	Purpose
GET /PUT	/api/departments/<uuid>	Department CRUD.
POST	/api/departments/<uuid>/delete	Delete (blocked with employees/assets).
POST	/api/departments/<uuid>/disable	Disable a department.
GET	/api/departments/<uuid>/dashboard	Department dashboard.

13.3 Employees

Met hod	Endpoint	Purpose
GET /POST	/api/employees	List (search/dept/loc/status/has_assets) / create employees.
GET /PUT	/api/employees/<uuid>	Employee CRUD.
POST	/api/employees/<uuid>/delete	Delete (blocked with assignment history).
POST	/api/employees/<uuid>/deactivate	Deactivate and auto-return assets.
GET	/api/employees/<uuid>/assets	Employee's assets.

13.4 Assignments

Met hod	Endpoint	Purpose
GET /POST	/api/assignments	List / create asset assignments (blocks double-assigning an asset).
POST	/api/assignments/<uuid>/return	Return an assigned asset.
POST	/api/assignments/bulk	Bulk assignments.
GET	/api/org/audit-logs	Organization audit logs.
GET	/api/org/stats	Organization dashboard stats.

13.5 Asset Lifecycle

Assets start as Draft and flow through Pending Approval, Approved, Purchased, Available, and Assigned, with Maintenance, Lost, Disposed, and Archived states. Every transition is validated against the state machine and recorded in immutable AssetHistory. Assets can be assigned to employees, returned, transferred, retired, and disposed, and carry QR codes for physical scanning.

Met hod	Endpoint	Purpose
GET /POST	/api/assets	List / create assets (filters incl. warranty expiring/expired).

Method	Endpoint	Purpose
GET PUT DELETE	/api/assets/<uuid>	Asset CRUD.
POST	/api/assets/<uuid>/status	Validated status transition.
POST	/api/assets/<uuid>/assign	Assign to an employee.
POST	/api/assets/<uuid>/return	Return from assignment.
POST	/api/assets/<uuid>/transfer	Transfer asset.
POST	/api/assets/<uuid>/retire	Retire asset.
POST	/api/assets/<uuid>/dispose	Dispose asset.
GET	/api/assets/<uuid>/history	Immutable asset history.
GET	/api/assets/<uuid>/qr	QR code image.
GET	/api/assets/dashboard	Asset dashboard.
GET	/api/assets/analytics	Asset analytics.
GET POST	/api/assets/import	CSV/XLSX import.
GET	/api/assets/export	Asset export.
GET	/api/scan/<qr_code>	QR lookup of an asset.

13.6 Cross-Module Insights

Method	Endpoint	Purpose
GET	/api/executive-analytics	KPIs across assets, monitoring, maintenance, licenses, and security plus recent activity.
GET	/api/global-search	Search across assets, employees, departments, locations, clients, licenses, and alerts.

14. Client Agent in Detail

The client agent is a standalone Python process that runs on every managed machine. It is deliberately dependency-light (standard library plus websockets and watchdog) so it can be frozen into a single executable with PyInstaller.

14.1 Startup Flow

1. Crash-log bootstrap (client_crash.log records every import step)
2. main(): rescue mode? silent mode? -> autostart re-registration
3. Single-instance check (Windows named mutex)
4. Load or create registration key + fingerprint (client_key.json)

5. Resolve admin URL: `env > CLI arg > cloud discovery > UDP > prompt`
6. Reachability loop: `GET /api/health`; on failure rediscover
7. `POST /api/register`; wait for approval (`poll /api/clients/<key>/status`)
8. Run initial scan; `POST /api/scan`
9. Register monitoring identity -> `AgentSecret (agent_id + secret)`
10. (Frozen Windows) spawn detached background copy; close terminal
11. Start threads: heartbeat loop, watchdog, cloud discovery, UDP listener
12. Start WebSocket client (if supported)
13. Start event monitors + event dispatcher
14. Enter scheduled scan loop (interval from server scan-config)

14.2 HTTP Communicator

register	<code>POST /api/register</code> with key, hostname, platform, version, fingerprint.
ping	<code>POST /api/ping</code> ; carries heartbeat; reads <code>trigger_scan</code> ; flushes offline queue.
submit_scan	<code>POST /api/scan</code> with full <code>scan_data</code> (120 s timeout).
check_status	<code>GET /api/clients/<key>/status</code> for approval polling.
fetch_latest_scan	<code>GET /api/clients/<key>/scan-results</code> .
get_scan_config	<code>GET /api/clients/<key>/scan-config</code> .
monitor_register	<code>POST /api/monitoring/agent/register</code> -> <code>secret_key</code> .
monitor_heartbeat	<code>POST /api/monitoring/agent/heartbeat</code> with HMAC headers.
monitor_heartbeat_public	<code>POST /api/monitoring/agent/heartbeat-public</code> with <code>registration_key</code> only.
monitor_inventory	<code>POST /api/monitoring/agent/inventory</code> with HMAC headers.
monitor_version_check	<code>GET /api/monitoring/agent/version-check</code> .

Retries: connection errors, 5xx, and 429 are retried up to 3 times with exponential backoff plus jitter (base 1 s, max 30 s). 4xx are not retried. The in-memory offline queue (500-item cap) holds scan posts until the server is reachable again.

14.3 WebSocket Client

WebSocketClient connects to `ws(s)://HOST/ws/agent/<agent_id>/` and authenticates with `{type:'auth', agent_id, secret, signature, timestamp}`. It auto-reconnects with exponential backoff (2 s up to 60 s), sends an application-level ping every 20 s, and permanently disables itself if the server answers the WebSocket upgrade with a plain HTTP response (Vercel detection), falling back to HTTP polling. Inbound commands: `auth_success` (replays pending commands), `auth_failed`, `command` (`scan_now/config_update/ping`), `ping`, `heartbeat_ack`.

14.4 Hardware Fingerprint

`generate_fingerprint()` hashes motherboard serial, CPU id, first disk serial, and all MAC addresses (joined and sorted) plus machine architecture with SHA-256 and truncates to 16 hex characters. This stable identity survives IP/hostname changes and reinstalls, which is what makes fingerprint-based re-registration work.

14.5 Scanner (Collected Data)

Field	Type	Purpose
Processor	Snapshot	Manufacturer, model, serial, cores, threads, speed, cache/architecture.
RAM	Snapshot	Manufacturer, capacity, serial, speed, form factor per module.
Storage	Snapshot	Disks (model, serial, size, interface) + partitions (filesystem, mount, free).
Motherboard	Snapshot	Manufacturer, model, serial, BIOS version.
GPU	Snapshot	Name, vendor, driver, dedicated memory.

Field	Type	Purpose
OS	Snapshot	Name, version, build, architecture, install details.
Network	Snapshot	Interfaces (name, MAC, IPv4, status).
Peripherals	Snapshot	Keyboard, mouse, audio, webcam, printers, USB storage/other.
Software	Snapshot	Installed applications (name, version, publisher) - Windows.
Windows Updates	Snapshot	KB IDs and descriptions (Windows only).
Antivirus	Snapshot	Antivirus products from SecurityCenter2 (Windows only).
User Accounts	Snapshot	Local user accounts (Windows only).

14.6 Metrics

`collect_metrics()` returns `cpu_usage_pct`, `ram_usage_pct`, `disk_usage_pct`, `disk_free_gb`, `disk_total_gb`, `network_connected` (probed via TCP to 8.8.8.8:53 and 1.1.1.1:53), and `uptime_seconds`, using WMI on Windows, `/proc` on Linux, and system commands on macOS.

14.7 Event Monitors

Method	Endpoint	Purpose
USB	5 s poll	USB insertion/removal/status-change (WMI on Windows, <code>lsusb</code> on Linux, <code>system_profiler</code> on macOS).
PROCESS	10 s poll	Process start/termination; flags suspicious names (mimikatz, <code>psexec</code> , <code>netcat</code> , <code>meterpreter</code> , ...) and processes running from temp folders.
SOFTWARE	60 s poll	Software install/remove/version change; antivirus disabled and firewall disabled detection via SecurityCenter2 (Windows).
FILE	Real-time (watchdog)	Critical file created/modified/deleted/moved on system paths; 2 s debounce; extensions and filenames whitelist.

14.8 Event Dispatcher and Offline Queue

`EventDispatcher` batches events (every 5 s or when 50 accumulate) and sends them over WebSocket first, then HMAC-signed HTTP, then public HTTP. If no transport is available it persists the batch to `%APPDATA%\SystemScannerPro\offline_events\batch_*.json` and replays those files on the next start. The dispatcher reports sent/failed/disk-queued/replayed counters.

14.9 Offline Resilience

Exponential backoff	HTTP retries with jitter; heartbeat backoff 5 s doubling to 30 s.
Offline event queue	Disk-persisted event batches replayed on reconnect/restart.
Heartbeat watchdog	Restarts a crashed heartbeat thread (max 5 restarts, check every 10 s).
Re-discovery	On repeated failures the agent re-queries cloud discovery and UDP discovery and switches to a reachable admin URL.
Passive UDP listener	Continuously listens on port 45000 and adopts any ADMIN_HERE broadcast (unless the URL is manual).
Vercel fallback	WebSocket auto-disabled; everything continues over HTTP polling.

14.10 Background and Autostart Mode (Windows)

The packaged Windows agent can run hidden. On a manual launch it registers autostart (Run key plus a Startup-folder `.vbs` launcher), spawns a fully detached hidden copy (`--rescue --silent`) and closes the terminal. The

hidden copy redirects output to client_agent.log, detaches its console, and runs the agent. A named mutex prevents duplicate instances, and closing the console window spawns a rescue copy so the agent stays online.

15. How Everything Connects (End-to-End Scenarios)

This section is the map of the platform: concrete scenarios showing exactly which component calls which endpoint and which event fires next. Each scenario traces one business flow through all the modules described in sections 8–14.

15.1 Onboarding a New Device

```
AGENT ADMIN SERVER
| client_key.json created
| admin URL resolved (arg/env/cloud/UDP)
|-- POST /api/register -----> scanner_api.RegisterClientView
| resolve key/fingerprint -> new Client (pending)
| DeviceMonitoringInfo created (via monitor module)
| event bus: DEVICE_REGISTERED
| ActivityLog: register
|<-- approved: false
|-- POST /api/ping (loop, 30s) -----> PingClientView
|-- GET /api/clients/<key>/status --> approved? (loop until admin acts)
|
ADMIN clicks Approve on /api/approve or /api/monitoring/devices/<uuid>/approve
| Client.approved=True, status=online
| event bus: DEVICE_APPROVED
| ActivityLog: approve
| dashboard WebSocket broadcast
|-- POST /api/scan (initial scan) ---> ScanResult stored, change baseline set
|-- monitoring agent register -----> AgentSecret issued (agent_id + secret)
|-- WebSocket connect ws/agent/<id>/ -> HMAC auth -> agent_<id> group
|-- event monitors + dispatcher start
DEVICE IS NOW AN ONLINE, MONITORED FLEET MEMBER
```

15.2 Real-Time Scan Command

```
ADMIN (dashboard or API) ADMIN SERVER AGENT
POST /api/clients/<key>/scan-now sets scan_requested=True
sends WebSocket command:
notify_agent() -> group_send agent_<id>
AgentConsumer.send_command
|-- {type:'command',
| command_type:'scan_now'}
|<-- runs collect_all()
|-- POST /api/scan
|-- {type:'scan_result'}
ScanResult stored
event bus: scan_completed
dashboard broadcast (live update)
|<-- dashboard shows the fresh scan
If the agent is offline: scan_requested stays set; on the next /api/ping the server
returns
trigger_scan:true and the agent runs the scan immediately (HTTP path).
```

15.3 Scheduled Scan Dispatch

```
APScheduler job fires (monitoring/scheduler.py)
|-- resolve target clients (all approved, or selected, platform-filtered)
```

```
|
+-- ONLINE client -----> WebSocket command scan_now
| ScanScheduleLog: triggered
|
|-- OFFLINE client -----> PendingScan created (offline queue)
ScanScheduleLog: skipped
broadcast schedule_executed to dashboard
Later, the offline agent reconnects:
AGENT -- GET /api/monitoring/agent/pending-scans --> list
AGENT -- POST /api/scan (executes the scan)
AGENT -- POST /api/monitoring/agent/pending-scans {scan_id, status:'executed'}
SERVER -- PendingScan marked executed, ScanScheduleLog completed
```

15.4 Event Detection Pipeline

Something changes on a machine (USB inserted, file modified, process started, software installed, antivirus disabled):

```
AGENT
USB/File/Process/Software monitor detects the delta vs baseline
`-> on_event(event) -> EventDispatcher
batch (5s / 50 events)
send WebSocket {type:'event', event_type, severity, event_data}
(fallback: HMAC HTTP / public HTTP / disk queue)
ADMIN SERVER (AgentConsumer._handle_event)
`-> DeviceHistory row (category security_event or status_change)
`-> DeviceAlert when severity is warning/critical
`-> broadcast device_event + new_alert to dashboards
SERVER-SIDE INVENTORY PATH (agent inventory submission)
/api/monitoring/agent/inventory -> detect_hardware_changes / detect_software_changes
`-> event bus events (hw_component_*, sw_*)
|-> subscribers create DeviceAlerts + admin notifications
|-> broadcast_change -> dashboard WebSocket
`-> record DeviceHistory
RESULT: the change is simultaneously a history entry, an alert, a notification,
and a live dashboard update.
```

15.5 Alert Lifecycle

```
SOURCE MONITORING/INTELLIGENCE/MAINTENANCE OPERATOR
heartbeat high CPU alerts.py high_cpu -> DeviceAlert active
offline detection check_offline_alerts -> device_offline
business rules intelligence.run_alert_checks() -> unified Alert
maintenance rules maintenance.check_and_generate_alerts() -> MaintenanceAlert
all of these ALSO:
`-> create_alert_notifications() -> user Notification (preference-gated)
`-> dashboard WebSocket (live badge)
OPERATOR ACTION
acknowledge -> status acknowledged, AlertHistory recorded
resolve -> status resolved, resolved_at set
dismiss -> status dismissed
assign -> assigned_user set (intelligence alerts)
ESCALATION (intelligence)
open alerts age past thresholds -> escalate_alerts() raises severity level
```

15.6 Offline Client Recovery

1. Agent loses connectivity -> heartbeat fails, backoff doubles (5s..30s)
2. Events keep arriving -> EventDispatcher persists batches to disk
3. Server marks the client offline (stale threshold) -> DEVICE_OFFLINE event

```

-> device_offline alert + dashboard red dot
4. After repeated failures the agent re-discovers:
cloud discovery -> UDP discovery -> reachable URL
5. Agent reconnects, pings, flushes the offline queue (events replay)
6. Agent pulls pending scheduled scans and executes them
7. Server marks online -> DEVICE_ONLINE event -> alert resolved, green dot

```

15.7 Change Detection and Notification

```

SCAN N (t=0) SCAN N+1 (t=3600s)
software: [A, B, C] software: [A, B, D]
hardware: [Disk SN-1] hardware: [Disk SN-1, Disk SN-2]
/api/scan stores ScanResult N+1
`-> scanner_api diff (last two scans) -> scan_changes on client detail
/api/monitoring/agent/inventory (or heartbeat + software list)
`-> detect_software_changes: C removed (info), D added (info)
`-> detect_hardware_changes: Disk SN-2 added (critical storage change)
`-> event bus: sw_removed, sw_installed, hw_component_added
`-> DeviceAlert: sw_removed, hw_component_added + admin notifications
`-> DeviceHistory rows + dashboard broadcasts
Unauthorized software? -> sw_unauthorized warning alert
Antivirus removed? -> sw_antivirus_removed CRITICAL alert

```

15.8 Serverless (Vercel) Flow

```

CLIENT VERCEL (api/index.py) SUPABASE
starts cold start:
|-- cloud discovery ----> migrate, admin user, register
| server URL + token in registry
|<-- https://...vercel.app
|-- POST /api/register -----> Django (serverless)
|-- POST /api/ping -----> 30s polling (no WebSocket)
|-- heartbeat-public -----> DeviceHeartbeat
|-- version-check ----->
WebSocket auto-disabled (HTTP rejection detected)
APScheduler/UDP unavailable -> schedules via external cron hitting
the /api/monitoring/schedules endpoints instead

```

16. Security Architecture

16.1 Authentication Layers

Admin panel (sessions)	Login by username or email; account lockout after 5 failures in 30 min; remember-me 30-day vs default 7-day sessions; CSRF-exempt session authentication for DRF.
Signed cookie fallback	scanner_auth cookie (TimestampSigner, 30-day max age, httpOnly, SameSite=Lax, Secure on Vercel) restores identity on cold starts.
JWT	HS256 access (60 min) / refresh (7 days) token pairs, issuer system-scanner-pro; obtain/refresh/verify endpoints.
API keys	SHA-256 hashed keys with rate limiting, optional IP allow-lists, and expiry; raw key returned once at creation.
Agent HMAC	Every agent HTTP endpoint verifies HMAC-SHA256 over the raw body with a timestamp (300 s window) against an active AgentSecret.

Resilient backend

Recovers the admin superuser if the user row is missing after a database reset or cold start.

16.2 Authorization (RBAC)

Roles are super_admin, admin, and viewer. Permission classes gate device/alert/schedule management and report access. In practice most API views also enforce tenant scoping (company/owner) rather than role checks, so visibility is the primary authorization model.

16.3 Agent Authentication Details

1. Agent registers -> server returns secret_key (AgentSecret row)
2. Every authenticated request adds headers:
X-Agent-ID: <agent_id>
X-Signature: HMAC-SHA256(secret_key, raw_body)
X-Timestamp: <unix float>
3. Server: timestamp within 300s (replay protection)
4. Server: active AgentSecret found by agent_id
5. Server: constant-time HMAC comparison passes
6. Rate limit (60/min per agent) applied

Blocking a device deactivates all its AgentSecrets -> identity revoked instantly.

16.4 Tenant Isolation

- Every admin server uses its own database (SQLite locally; the cloud instance for Vercel).
- Clients, settings, groups, locations, departments, employees, and assets carry a company FK.
- List/detail views filter by company (superuser) or owner (regular admin).
- Unowned pending clients are visible to all admins so new devices can be approved.
- The UI URL prefix /<user>-<company>/ reinforces the boundary.

16.5 HTTP Hardening

- SecurityHeadersMiddleware: X-Content-Type-Options nosniff, X-Frame-Options DENY, X-XSS-Protection, Referrer-Policy strict-origin-when-cross-origin.
- SECURE_BROWSER_XSS_FILTER and SECURE_CONTENT_TYPE_NOSNIFF enabled.
- Secure cookies on Vercel; HTTPS redirects handled by the proxy.
- Agent secrets are 64-hex-char random values; connection tokens use secrets.token_hex(16).

16.6 Secrets Management

Secrets live in .env (DJANGO_SECRET_KEY, SUPABASE_SERVICE_KEY, etc.); .env is git-ignored. License keys are stored encrypted with a masked display. API keys are stored only as SHA-256 hashes. Report the following to your security team: the cloud-discovery client embeds a service-role JWT in client/discovery.py, and example credentials appear in repository files (.env.template, .env.vercel). Rotate these before production.

16.7 Auditing

Three audit layers: ActivityLog (admin actions), AuditLog/LoginHistory (authentication and security events), and DeviceHistory + AssetHistory (immutable device and asset trails). The intelligence app adds a unified immutable AuditLogEntry. All are company-scoped.

17. REST API Reference (Complete)

All endpoints are relative to the admin base URL. Prefixes: /api/ (scanner_api), /api/monitoring/ (monitoring), /api/intelligence/ (intelligence), /api/maintenance/ (maintenance). WebSocket endpoints are covered in section 18.

17.1 scanner_api: Registration, Communication, Scans

Method	Endpoint	Purpose
POST	/api/register	Register / refresh a client.
POST	/api/approve	Approve a single client.
POST	/api/approve-multiple	Bulk approve clients.
POST	/api/ping	Client heartbeat (online status, trigger_scan).
GET	/api/clients/<key>/status	Approval status.
POST	/api/scan	Submit a scan result.
POST	/api/scan/local	Scan the admin server machine.
POST	/api/scan/all	Trigger scans on all approved clients.
GET	/api/scan/history	Scan history with filters.
GET	/api/clients	List non-deleted clients.
GET	/api/clients/<key>	Client detail + scan diff.
DELETE	/api/clients/<key>	Soft-delete client.
POST	/api/clients/delete-multiple	Bulk soft-delete.
PUT	/api/clients/<key>/manual	Update manual fields.
GET / POST	/api/clients/<key>/addons	List / add addon devices.
DELETE	/api/clients/<key>/addons/<id>	Remove an addon device.
GET / PUT	/api/clients/<key>/scan-config	Scan interval / enabled.
POST	/api/clients/<key>/scan-now	Trigger immediate scan.
GET	/api/clients/<key>/scan-results	Latest scan result.
GET	/api/admin-client	Admin self-client keep-alive.
GET	/api/health	Lightweight health check.
POST	/api/supabase/register	Register the server in the cloud registry.

17.2 scanner_api: Settings, Admin, Activity

Met hod	Endpoint	Purpose
GET /PUT	/api/settings	Global settings.
GET /PUT	/api/settings/organization	Organization profile.
GET /PUT	/api/settings/security	Security settings.
GET /PUT	/api/settings/notifications	Notification settings.
GET /POST/ PUT	/api/settings/connection	Connection URL + token.
GET /PUT	/api/settings/dashboard	Dashboard widget settings.
GET /POST	/api/admin/users	List / create admin users.
DELETE	/api/admin/users/<id>	Delete admin user.
GET	/api/admin/stats	System statistics.
GET	/api/admin/scan-changes	Per-client scan diffs.
POST	/api/admin/change-password	Change password.
GET	/api/activity-log	Recent activity log.
GET /POST	/api/groups	List / create client groups.
DELETE	/api/groups/<id>	Delete a group.

17.3 scanner_api: Authentication

Met hod	Endpoint	Purpose
POST	/api/auth/login	Session login (username or email).
POST	/api/auth/logout	Session logout.
GET	/api/auth/me	Current user profile.
GET /PUT	/api/auth/profile	Get / update profile.
POST	/api/auth/change-password	Change password.

Met hod	Endpoint	Purpose
POST	/api/auth/upload-avatar	Upload an avatar.
GET	/api/auth/login-history	Login history.
GET	/api/auth/audit-logs	Audit log trail.
GET	/api/auth/active-sessions	Active sessions.
POST	/api/auth/token/obtain	Obtain JWT access + refresh tokens.
POST	/api/auth/token/refresh	Refresh the access token.
POST	/api/auth/token/verify	Verify a token.
GET /POST	/api/auth/api-keys	List / create API keys.
DELETE	/api/auth/api-keys/<id>	Revoke an API key.

17.4 scanner_api: Organization

Met hod	Endpoint	Purpose
GET /POST	/api/locations	List / create locations.
GET /PUT	/api/locations/<uuid>	Location CRUD.
POST	/api/locations/<uuid>/delete	Delete a location.
POST	/api/locations/<uuid>/archive	Archive a location.
GET	/api/locations/<uuid>/dashboard	Location dashboard.
GET /POST	/api/locations/<uuid>/export	Location CSV export / import.
GET /POST	/api/departments	List / create departments.
GET /PUT	/api/departments/<uuid>	Department CRUD.
POST	/api/departments/<uuid>/delete	Delete a department.
POST	/api/departments/<uuid>/disable	Disable a department.
GET	/api/departments/<uuid>/dashboard	Department dashboard.

Method	Endpoint	Purpose
GET /POST	/api/employees	List / create employees.
GET /PUT	/api/employees/<uuid>	Employee CRUD.
POST	/api/employees/<uuid>/delete	Delete an employee.
POST	/api/employees/<uuid>/deactivate	Deactivate + auto-return assets.
GET	/api/employees/<uuid>/assets	Employee assets.
GET /POST	/api/assignments	List / create assignments.
POST	/api/assignments/<uuid>/return	Return an assignment.
POST	/api/assignments/bulk	Bulk assignments.
GET	/api/org/audit-logs	Organization audit logs.
GET	/api/org/stats	Organization stats.

17.5 scanner_api: Assets

Method	Endpoint	Purpose
GET /POST	/api/assets	List / create assets.
GET /PUT/DELETE	/api/assets/<uuid>	Asset CRUD.
POST	/api/assets/<uuid>/status	Status transition.
POST	/api/assets/<uuid>/assign	Assign asset.
POST	/api/assets/<uuid>/return	Return asset.
POST	/api/assets/<uuid>/transfer	Transfer asset.
POST	/api/assets/<uuid>/retire	Retire asset.
POST	/api/assets/<uuid>/dispose	Dispose asset.
GET	/api/assets/<uuid>/history	Asset history.
GET	/api/assets/<uuid>/qr	Asset QR code.
GET	/api/assets/dashboard	Asset dashboard.

Method	Endpoint	Purpose
GET	/api/assets/analytics	Asset analytics.
GET /POST	/api/assets/import	CSV/XLSX import.
GET	/api/assets/export	Asset export.
POST	/api/assets/bulk	Bulk asset actions.
GET	/api/scan/<qr_code>	QR lookup.
GET	/api/executive-analytics	Cross-module KPIs.
GET	/api/global-search	Global search.

17.6 monitoring: Agent and Device

Method	Endpoint	Purpose
POST	/api/monitoring/agent/register	Agent registration (issues HMAC secret).
POST	/api/monitoring/agent/heartbeat	HMAC heartbeat.
POST	/api/monitoring/agent/heartbeat-public	Public heartbeat.
POST	/api/monitoring/agent/inventory	Hardware/software inventory snapshot.
GET	/api/monitoring/agent/version-check	Agent version check.
GET /POST	/api/monitoring/agent/pending-scans	Pending scans for an agent.
GET	/api/monitoring/dashboard	Fleet dashboard aggregates.
GET	/api/monitoring/trends	Fleet trends.
GET	/api/monitoring/devices	List devices.
POST	/api/monitoring/devices/bulk	Bulk device actions.
GET /PUT	/api/monitoring/devices/<uuid>	Device detail / status.
POST	/api/monitoring/devices/<uuid>/approve	Approve device.
POST	/api/monitoring/devices/<uuid>/block	Block device.
GET	/api/monitoring/devices/<uuid>/metrics	Device metrics.
GET	/api/monitoring/devices/<uuid>/history	Device history.
GET	/api/monitoring/devices/<uuid>/alerts	Device alerts.

Method	Endpoint	Purpose
GET	/api/monitoring/devices/<uuid>/hardware	Hardware inventory.
GET	/api/monitoring/devices/<uuid>/software	Software inventory.
GET	/api/monitoring/devices/<uuid>/heartbeats	Heartbeat history.

17.7 monitoring: Alerts, Schedules, Reports

Method	Endpoint	Purpose
GET	/api/monitoring/alerts	Fleet alerts.
POST	/api/monitoring/alerts/<uuid>/action	Ack / resolve / dismiss.
GET / POST	/api/monitoring/agent-versions	Agent version catalog.
GET / PUT	/api/monitoring/settings/unauthorized-software	Unauthorized software list.
GET / POST	/api/monitoring/schedules	List / create schedules.
GET / PUT / DELETE	/api/monitoring/schedules/<uuid>	Schedule CRUD.
POST	/api/monitoring/schedules/<uuid>/toggle	Enable / disable.
GET	/api/monitoring/schedules/<uuid>/history	Schedule run history.
GET	/api/monitoring/schedules/status	Scheduler status.
GET / DELETE	/api/monitoring/schedules/pending	Pending scans.
GET	/api/monitoring/reports/fleet/pdf	Fleet PDF report.
GET	/api/monitoring/reports/fleet/csv	Fleet CSV export.
GET	/api/monitoring/reports/device/<uuid>/pdf	Device PDF report.
GET	/api/monitoring/reports/device/<uuid>/csv	Device CSV export.
GET	/api/monitoring/reports/alerts/pdf	Alerts PDF report.
GET	/api/monitoring/reports/alerts/csv	Alerts CSV export.

17.8 intelligence

Met hod	Endpoint	Purpose
GET	/api/intelligence/dashboard	Analytics dashboard.
GET /PO ST	/api/intelligence/alerts	List / create alerts.
GET	/api/intelligence/alerts/<uuid>	Alert detail.
POS T	/api/intelligence/alerts/<uuid>/a ction	Ack / resolve / dismiss / assign.
GET	/api/intelligence/alerts/<uuid>/h istory	Alert history.
POS T	/api/intelligence/alerts/bulk	Bulk actions.
POS T	/api/intelligence/alerts/run-chec ks	Run alert checks.
GET	/api/intelligence/alerts/export	CSV export.
GET /PO ST	/api/intelligence/alerts/rules	Alert rules.
GET /PU T/D ELE TE	/api/intelligence/alerts/rules/<u uid>	Alert rule CRUD.
GET	/api/intelligence/notifications	Notifications.
POS T	/api/intelligence/notifications/m ark-all-read	Mark all read.
GET /PU T	/api/intelligence/notifications/p references	Preferences.
POS T	/api/intelligence/notifications/< uuid>/action	Read / archive.
GET /PO ST	/api/intelligence/reports	List / generate reports.
GET	/api/intelligence/reports/<uuid>	Report detail.
GET	/api/intelligence/reports/<uuid>/ export	Download report.
GET /PO ST	/api/intelligence/scheduled-repor ts	Scheduled reports.
GET /PU T/D ELE TE	/api/intelligence/scheduled-repor ts/<uuid>	Scheduled report CRUD.
GET	/api/intelligence/audit-logs	Audit logs.
GET	/api/intelligence/audit-logs/expo rt	Audit CSV export.

Met hod	Endpoint	Purpose
GET	/api/intelligence/audit-logs/<uui d>	Audit detail.
GET	/api/intelligence/audit-logs/user /<int>	User activity.
GET	/api/intelligence/audit-logs/modu les	Module list.
GET	/api/intelligence/audit-logs/acti ons	Action list.
GET /PO ST	/api/intelligence/compliance	Compliance logs.
GET	/api/intelligence/compliance/dash board	Compliance dashboard.
GET /PU T/D ELE TE	/api/intelligence/compliance/<uui d>	Compliance CRUD.
GET /PO ST	/api/intelligence/retention-polic ies	Retention policies.
GET /PU T	/api/intelligence/settings	Intelligence settings.

17.9 maintenance

Met hod	Endpoint	Purpose
GET /PO ST	/api/maintenance/maintenance	List / create maintenance.
GET /PU T/D ELE TE	/api/maintenance/maintenance/<uui d>	Maintenance CRUD.
POS T	/api/maintenance/maintenance/<uui d>/status	Status transition.
POS T	/api/maintenance/maintenance/<uui d>/approve	Approve / reject.
POS T	/api/maintenance/maintenance/<uui d>/upload	Upload document.
GET	/api/maintenance/maintenance/expo rt	CSV export.
GET /PO ST	/api/maintenance/warranties	List / create warranties.
GET /DE LET E	/api/maintenance/warranties/<uui d>	Warranty detail.

Method	Endpoint	Purpose
GET /POST	/api/maintenance/downtime	List / create downtime.
POST	/api/maintenance/downtime/<uuid>/end	End downtime.
GET /POST	/api/maintenance/licenses	List / create licenses.
GET /PUT/DELETE	/api/maintenance/licenses/<uuid>	License CRUD.
POST	/api/maintenance/licenses/<uuid>/archive	Archive license.
POST	/api/maintenance/licenses/assign	Assign license.
POST	/api/maintenance/licenses/assignments/<uuid>/remove	Remove assignment.
GET	/api/maintenance/licenses/export	License CSV export.
GET /POST	/api/maintenance/compliance	Compliance records.
POST	/api/maintenance/compliance/<uuid>/action	Compliance action.
GET	/api/maintenance/alerts	Maintenance alerts.
POST	/api/maintenance/alerts/<uuid>/action	Alert action.
POST	/api/maintenance/alerts/check	Run alert checks.
GET	/api/maintenance/dashboard	Dashboard.
GET	/api/maintenance/analytics/cost-trend	Cost trend.
GET	/api/maintenance/analytics/vendor-performance	Vendor performance.
GET	/api/maintenance/analytics/failure-rate	Failure rates.
GET	/api/maintenance/analytics/downtime	Downtime analytics.
GET	/api/maintenance/analytics/license-dashboard	License dashboard.
GET	/api/maintenance/analytics/license-utilization	License utilization.
GET	/api/maintenance/analytics/license-cost	License cost.

17.10 System / Diagnostics

Method	Endpoint	Purpose
GET	/__health/	Health check (public).
GET	/__diag/	Vercel DB/init diagnostics.
GET	/download-client/	Download the client installer with Content-SHA256.

18. WebSocket Protocol

WebSockets are served by Channels via ASGI (monitoring/routing.py, django_admin/asgi.py). Routes: ws/agent/<agent_id>/ and ws/dashboard/.

18.1 Dashboard Channel (browser)

```
CLIENT (browser) -> ws(s)://HOST/ws/dashboard/
SERVER sends -> {type:'connected', message:'Dashboard connected...'}
CLIENT -> -> {type:'ping'} SERVER -> {type:'pong'}
CLIENT -> -> {type:'subscribe_device', device_id}
CLIENT -> -> {type:'unsubscribe_device', device_id}
SERVER PUSHES (group 'dashboard'):
{type:'dashboard.update', data:{type:<event_type>, ...}}
{type:'dashboard.alert', data:{...alert...}}
{type:'dashboard.agent_status', data:{agent_id, status}}
{type:'device.update', data:{...}} (to device_<id> group)
```

18.2 Agent Channel (agent)

```
AGENT connects -> ws(s)://HOST/ws/agent/<agent_id>/
SERVER sends -> {type:'auth_required', message, agent_id}
AGENT sends -> {type:'auth', agent_id, secret, signature, timestamp}
(timestamp within 300s; secret matches AgentSecret)
SERVER sends -> {type:'auth_success', server_time, pending_commands:[...]}
OR -> {type:'auth_failed', message} then close 4003
AGENT -> SERVER:
{type:'heartbeat', cpu, ram, disk, ...} -> {type:'heartbeat_ack', health...}
{type:'scan_result', scan_type, scan_data} -> {type:'scan_ack'}
{type:'event', event_type, severity, event_data}
{type:'status_update', monitoring_status}
{type:'pong'}
SERVER -> AGENT (group 'agent_<agent_id>'):
{type:'command', command_type, command_id, payload} # scan_now/config_update
{type:'ping', server_time}
```

18.3 Command Reference

	Purpose
an immediate scan. Agent runs collect_all() and submits via POST n replies {type:'scan_result'}.	
interval_seconds / enabled. Agent merges the new scan config.	
-alive; agent replies pong.	

18.4 Message Flow Guarantees

- Agent heartbeat messages flow both ways: WS is the live channel; the 30-second HTTP ping is the durable fallback.

- Pending commands are read from DeviceMonitoringInfo.notes (semicolon-delimited) and delivered in auth_success.
- Offline agents keep working over HTTP; when they reconnect, the pending command list is delivered immediately.

19. Event Types and the Event Bus

The event bus (monitoring/event_bus.py) is an in-process pub/sub singleton. It is the connective tissue of the monitoring module: every notable occurrence is published once, and subscribers turn it into alerts, notifications, history, and dashboard broadcasts. Event types:

Method	Endpoint	Purpose
HW_COMPONENT_ADDED / REMOVED / MODIFIED	Hardware inventory change	Severity from component map (storage critical).
SW_INSTALLED / SW_REMOVED / SW_VERSION_CHANGED	Software change	Info by default.
SW_UNAUTHORIZED	Blocklisted software found	Warning.
SW_ANTIVIRUS_REMOVED	Antivirus product removed	Critical.
HEALTH_LEVEL_CHANGED / HEALTH_SCORE_UPDATED	Health recomputation	Dashboard + history.
ALERT_CREATED / ACKNOWLEDGED / RESOLVED / DISMISSED	Alert lifecycle	Dashboard broadcast.
DEVICE_REGISTERED / APPROVED / BLOCKED / DELETED / OFFLINE / ONLINE / STATUS_CHANGED	Device lifecycle	Alerts + history + dashboard.
AGENT_VERSION_CHANGED	Agent updated	Info alert + notification.
HEARTBEAT_RECEIVED	Every heartbeat	Metrics pipeline.
SCAN_COMPLETED / SCAN_SCHEDULED	Scan lifecycle	Dashboard + history.

19.1 How a Published Event Is Handled

```
publish(event)
`-> subscribers registered for that event_type
`-> wildcard subscriber (broadcasts to dashboard + device groups)
`-> alert-creation subscribers (severity-gated)
`-> notification subscriber (admin in-app notification)
`-> DeviceHistory writer (category from event prefix)
`-> recent-events ring buffer (queryable via get_recent_events)
```

20. Database Schema Overview

Each admin server owns its own database. Table prefixes: none (scanner_api tables like clients, scan_results), monitoring_, intelligence_, maintenance_.

20.1 Table Families

Component	Description
Identity & fleet	companies, client_groups, clients, scan_results, addon_devices, device_fingerprints.
Settings & audit	settings, activity_logs, audit_logs, login_history, login_attempts, administrator_profiles.

Component	Description
Organization	locations, departments, employees, employee_asset_assignments, org_audit_logs.
Assets	asset_categories, asset_vendors, assets, asset_assignments_v2, asset_transfers, asset_history, asset_documents, api_keys.
Monitoring	monitoring_device_info, monitoring_hardware_inventory, monitoring_software_inventory, monitoring_device_heartbeat, monitoring_device_metrics, monitoring_device_history, monitoring_device_alerts, monitoring_agent_versions, monitoring_agent_secrets.
Scheduling	monitoring_scheduled_scans, monitoring_pending_scans, monitoring_scan_schedule_logs.
Intelligence	intelligence_alerts, intelligence_alert_history, intelligence_alert_rules, intelligence_notifications, intelligence_notification_preferences, intelligence_reports, intelligence_scheduled_reports, intelligence_audit_logs, intelligence_compliance_logs, intelligence_dashboard_analytics, intelligence_retention_policies.
Maintenance	maintenance_records, maintenance_history, maintenance_documents, warranty_records, downtime_records, software_licenses, license_assignments, license_history, compliance_records, maintenance_alerts.

20.2 Key Relationships

- Client → ScanResult (1:N); Client → AddonDevice (1:N); Client → DeviceMonitoringInfo (1:1); Client → DeviceHeartbeat / HardwareInventory / SoftwareInventory / DeviceAlert / DeviceHistory / AgentSecret / PendingScan (1:N).
- Company → Client, Setting, Location, Department, Employee, Asset (1:N) — the tenant backbone.
- Employee → Client (through EmployeeAssetAssignment); Asset → Employee (assigned_to); License → Asset/Employee/Department (through LicenseAssignment).
- DeviceHistory and AssetHistory are immutable: save() raises on update.

20.3 Database Location

Method	Endpoint	Purpose
DEVELOPMENT	admin/data/scanner.db	SQLite next to the admin server.
WINDOWS (PACKAGED)	%APPDATA%\SystemScannerPro\scanner.db	Per-user app data.
LINUX (PACKAGED)	~/.local/share/SystemScannerPro/scanner.db	XDG data home.
MACOS (PACKAGED)	~/Library/Application Support/SystemScannerPro/scanner.db	App support.
VERCEL	PostgreSQL via DATABASE_URL	Supabase pooler (port 6543).

21. Reports and Exports

21.1 Monitoring Reports

Field	Type	Purpose
Fleet PDF	generate_fleet_pdf()	Fleet summary table (status counts, health) + per-device details.
Fleet CSV	generate_fleet_csv()	10-column fleet inventory export.

Field	Type	Purpose
Device PDF	generate_device_pdf(key)	Device info + active alerts.
Device CSV	generate_device_csv(key)	90-day alert history for one device.
Alerts PDF	generate_alerts_pdf(days=30)	Alert history table.
Alerts CSV	generate_alerts_csv(days=30)	Alert history export.

21.2 Intelligence Report Engine

The intelligence engine generates nine report types in CSV, Excel, or PDF, persists each as a Report row, and lets users schedule recurring reports. Report types: asset_inventory, asset_assignment, expiring_licenses, upcoming_maintenance, device_health, compliance_report, monthly_summary, software_inventory, audit_report.

21.3 Maintenance and License Exports

Maintenance records, licenses, locations, departments, employees, assets, alerts, and audit logs all have CSV export endpoints; assets additionally support CSV/XLSX import.

22. Background Tasks and Scheduling

Component	Description
Admin UDP discovery	main.py start_discovery_listener/broadcaster: answers DISCOVER_ADMIN and broadcasts ADMIN_HERE on UDP 45000.
Admin self-heartbeat	main.py admin_client_heartbeat_loop: keeps the admin self-client online every 30 s.
Admin self-scan	main.py admin_self_scan: one local scan on startup.
Cloud-registry refresh	main.py cloud_discovery_refresh_loop: re-registers the public IP every 300 s.
APScheduler	monitoring/scheduler.py: interval/daily/weekly/monthly/once scheduled scans, splitting targets into online (WebSocket) and offline (PendingScan).
Heartbeat watchdog	client/main.py HeartbeatWatchdog: restarts a dead heartbeat thread.
Client cloud discovery loop	client/main.py cloud_discovery_loop: re-queries the registry every 300 s.
Client UDP listener	client/main.py listen_admin_broadcast: adopts ADMIN_HERE broadcasts.
Event dispatcher	client/events/dispatcher.py: flushes batched events every 5 s.
Stale checker	manage.py stale_checker: marks stale clients offline (loop, 30 s).
Alert checkers	manage.py alert_checker / health_checker / offline_detector; POST alerts/check and alerts/run-checks.

23. Deployment Guide

23.1 Self-Hosted (Local / VPS)

```
# Install
pip install -r requirements.txt
# Start the admin server (binds 0.0.0.0, port 80)
python admin/main.py
# With a public domain and WebSocket support (ASGI via daphne):
python admin/main.py --host 0.0.0.0 --port 80 --domain scanner.example.com --asgi
# Agents connect with:
python client/main.py https://scanner.example.com
```

23.2 nginx Reverse Proxy with HTTPS

```
server { listen 80; server_name scanner.example.com; return 301 https://$host$request_uri;
}
server {
listen 443 ssl;
server_name scanner.example.com;
ssl_certificate /etc/letsencrypt/live/scanner.example.com/fullchain.pem;
ssl_certificate_key /etc/letsencrypt/live/scanner.example.com/privkey.pem;
location / {
proxy_pass http://127.0.0.1:80;
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
}
location /ws/ {
proxy_pass http://127.0.0.1:80;
proxy_http_version 1.1;
proxy_set_header Upgrade $http_upgrade;
proxy_set_header Connection "upgrade";
proxy_set_header Host $host;
proxy_read_timeout 86400;
}
}
sudo certbot --nginx -d scanner.example.com
```

23.3 Vercel Serverless

vercel.json routes everything to api/index.py (@vercel/python, maxDuration 30). The bootstrap sets DJANGO_SETTINGS_MODULE, migrates, creates the admin user, stores the admin URL in the cloud registry, and forces auto-approve off. Requirements: DATABASE_URL (Supabase PostgreSQL, pooler rewritten to port 6543), DJANGO_SECRET_KEY, DJANGO_DEBUG=False, DJANGO_ALLOWED_HOSTS.

```
# Vercel environment variables
DJANGO_SECRET_KEY = <random>
DJANGO_DEBUG = False
DJANGO_ALLOWED_HOSTS = *
DATABASE_URL = postgresql://postgres:****@db.<proj>.supabase.co:5432/postgres
SUPABASE_URL = https://<proj>.supabase.co
SUPABASE_SERVICE_KEY = <service role key>
# Vercel limitations (client auto-adapts)
WebSocket -> not supported; agents use HTTP polling
UDP 45000 -> not supported; clients use cloud discovery / URL
APScheduler -> not supported; use Vercel cron to call schedule endpoints
SQLite -> not supported; PostgreSQL required
```

23.4 Cloud Discovery

The admin server registers itself in the Supabase `server_registry` table (`setup_cloud_discovery.sql` creates it with public read and service-role write policies). Clients query it during startup and every 300 s to follow the admin if it moves. A GitHub Actions workflow (`supabase-register.yml`) re-registers hourly as a safety net.

24. Building Executables

`build_client.py` builds the client (and admin) into single-file executables with PyInstaller. The script auto-installs PyInstaller, collects client data files into the bundle, and writes a `.spec` file to avoid command-line length limits.

```
pip install pyinstaller
python build_client.py # builds client_scanner.exe
# Output handling:
# - dist/client_scanner.exe
# - copied to admin/data/client_scanner.exe (downloadable from /download-client/)
# - admin/data/client_scanner.zip (client_scanner.dat + run.bat + README)
# Optional code signing via env vars:
# CODE_SIGN_PFX, CODE_SIGN_PASSWORD, CODE_SIGN_TIMESTAMP
# Run the built client:
client_scanner.exe http://192.168.1.100:80
client_scanner.exe https://your-project.vercel.app
```

25. Configuration Reference

25.1 Environment Variables

Field	Type	Purpose
DJANGO_SECRET_KEY	string	Django secret key (override the insecure default).
DJANGO_DEBUG	bool	Debug mode.
DJANGO_ALLOWED_HOSTS	csv	Allowed hosts (* or comma list).
DATABASE_URL	url	PostgreSQL connection (required on Vercel).
DB_NAME / DB_USER / DB_PASSWORD / DB_HOST / DB_PORT	string	PostgreSQL components.
DB_CONN_MAX_AGE	int	0 on Vercel, 10 on persistent servers.
SUPABASE_URL	string	Supabase project URL (cloud registry).
SUPABASE_SERVICE_KEY	string	Service role key for registry writes.
VERCEL	flag	Set to 1 on Vercel (IS_VERCEL mode).
ADMIN_SERVER_URL	string	Client-side override of the admin URL.

25.2 Important Setting Keys

Field	Type	Purpose
auto_approve	bool	Auto-approve new registrations.
stale_threshold_seconds	int	Default 120; offline cutoff.

Field	Type	Purpose
scan_all_interval	int	Interval for scan-all.
admin_client_key	string	Admin self-client key.
admin_server_url / admin_connection_token	string	Connection settings.
session_timeout_minutes	int	Default 120.
alert_cpu_threshold / alert_ram_threshold	int	Default 90.
alert_disk_threshold / alert_disk_free_gb	int	Default 95 / 5.
unauthorized_software_list	csv	Software that triggers sw_unauthorized.
monitoring_warning_seconds / monitoring_offline_seconds / monitoring_critical_seconds	int	Offline alert tiers 300 / 900 / 1800.
alert_escalation_enabled, notification_retention_days, report_retention_days, audit_retention_days	mixed	Intelligence settings.

25.3 Client Configuration Files

Field	Type	Purpose
client_key.json	registration_key + fingerprint	Identity; do not delete unless re-registering.
client_config.json	admin_url, scan_interval, auto_start, manual_url	Connection and scan settings.
client_agent.log / client_crash.log	log files	Runtime and startup diagnostics.

25.4 Admin CLI Options

Method	Endpoint	Purpose
--PORT PORT	Server port	Default 80.
--HOST HOST	Bind address	Prompted on first run.
--DEBUG	Debug mode	Enables Django debug.
--USERNAME / --PASSWORD	Default admin	admin / admin123.
--RESET	Re-ask bind IP	Clears saved host.

Method	Endpoint	Purpose
--DOMAIN DOMAIN	Public domain	Registers https://<domain> for cloud discovery.
--ASGI	ASGI server	Serve with daphne (WebSocket support).

26. Operations and Administration

26.1 Management Commands

```
python admin/manage.py migrate # apply DB migrations
python admin/manage.py scan_local # scan the admin machine
python admin/manage.py scan_all # trigger scans on all clients
python admin/manage.py stale_checker # mark stale clients offline
python admin/manage.py alert_checker # evaluate offline alerts
python admin/manage.py health_checker # recompute health scores
python admin/manage.py offline_detector # mark offline devices
python admin/manage.py clear_data # wipe data, recreate admin
python admin/manage.py createsuperuser # create a superuser
python admin/manage.py changepassword # change a user's password
python admin/manage.py collectstatic # collect static files
```

26.2 Reset Procedures

Method	Endpoint	Purpose
RESET ADMIN IP	python admin/main.py --reset	Re-prompts for the bind address.
RESET ADMIN PASSWORD	Delete admin/data/scanner.db then re-run	Recreates DB + default admin.
RESET CLIENT	Delete client/client_key.json	New key + fingerprint re-link on re-run.
RESET CLIENT URL	Delete client_config.json	Re-prompts for the admin URL.
FULL FACTORY RESET	Delete DB + client key/config files	Brand-new state on both sides.

26.3 Running as a Service

```
# Windows scheduled task
schtasks /create /tn "SystemScanner" /tr "C:\path\client_scanner.exe http://SERVER" /sc
onstart
# Linux systemd
# /etc/systemd/system/scanner-client.service
[Unit]
Description=System Scanner Client
After=network.target
[Service]
ExecStart=/usr/bin/python3 /path/to/client/main.py http://SERVER
Restart=always
RestartSec=10
[Install]
WantedBy=multi-user.target
# macOS launchd (~/.Library/LaunchAgents/com.scanner.client.plist)
# RunAtLoad true, KeepAlive true
```

27. Troubleshooting

Endpoint	Purpose
Verify the admin is running and reachable; allow TCP 80 and UDP 45000 through the firewall; bind 0.0.0.0; use the full http:// URL.	
Admin offline, network issue, or URL missing the http(s) prefix.	
Check port 80 reachability; ensure /ws/ has proxy upgrade headers; on Parcel WebSocket is unsupported (HTTP fallback is normal).	
Approve from the dashboard, or enable auto-approve in Settings.	
The Live badge must be green on the monitoring page; the server must run ASGI (daphne/uvicorn).	
Run <code>python admin/manage.py migrate</code> .	
Look for '[OK] 4 event monitors active' in the client log; if offline, events are queued to disk and replay on reconnect.	
Consecutive errors trigger cloud + UDP re-discovery after 5 failures.	
Cold starts may take 5-10 s; first request may time out.	
Check DATABASE_URL, Supabase project state, and IP allow-list.	
Soft delete only removes from dashboards; re-registration with the same key returns 403 until the row is purged.	
Locking deactivates AgentSecrets; the device is re-listed as pending, not online.	

28. Known Issues and Limitations

Endpoint	Purpose
intelligence management command passes <code>escalate_only=...</code> but the function takes no arguments and returns a list; the API view calls it correctly.	
ScheduledReport has no background executor; nothing computes <code>next_run</code> or runs reports on schedule.	
No WebSocket, UDP, or APScheduler; sessions rely on signed cookies; SQLite is ephemeral.	
Client code reports VERSION 1.6.1 while the exe version resource is 3.0.0.0.	
References <code>build/build.py</code> , <code>pyusb</code> , <code>psutil</code> ; the real builder is <code>build_client.py</code> and the client uses <code>websockets</code> + <code>watchdog</code> + <code>stdlib</code> .	
<code>client_config.json</code> may carry a URL path (e.g. <code>/kailasam-shiv</code>); the communicator normalizes to <code>scheme://netloc</code> .	
Service-role JWT and example keys exist in repo files; rotate before production.	
<code>acknowledge_alert/resolve_alert</code> hardcode <code>acknowledged_by='admin'</code> .	
maintenance uses <code>seats_used >= purchased_seats</code> while intelligence uses <code>></code> ; inconsistent.	

29. Appendix

29.1 Device Status Indicators

Method	Endpoint	Purpose
ONLINE	Green dot	Heartbeats received (every 30 s).
OFFLINE	Red dot	No heartbeat for over the stale threshold (120 s).
PENDING	Yellow dot	Registered but not yet approved.
BLOCKED	Purple dot	Blocked by admin; agent secrets deactivated.
MAINTENANCE / INACTIVE	Operator-set	Administrative monitoring statuses.

29.2 Scan Data Collected

Field	Type	Purpose
Processor	Snapshot	Manufacturer, model, serial, cores, threads, speed, cache/architecture.
RAM	Snapshot	Per-module manufacturer, capacity, serial, speed, form factor.
Storage	Snapshot	Disks (model, serial, size, interface) + partitions (filesystem, mount).
Motherboard	Snapshot	Manufacturer, model, serial, BIOS version.
GPU	Snapshot	Name, vendor, driver, dedicated memory.
OS	Snapshot	Name, version, build, architecture, install date.
Network	Snapshot	Interfaces (name, MAC, IPv4, status).
Peripherals	Snapshot	Keyboard, mouse, audio, webcam, printers, USB devices.
Software	Snapshot	Installed apps (name, version, publisher).
Windows Updates	Snapshot	KB IDs + descriptions (Windows only).
Antivirus	Snapshot	AV products + firewall status (Windows only).
User Accounts	Snapshot	Local user accounts (Windows only).

29.3 Health Score Formula

CPU (25%)	100 when ≤70%; linear fall to 50 at 85%; floor 10 beyond 85%.
RAM (25%)	Same piecewise scoring as CPU.
Disk (20%)	100 when ≤80%; linear fall to 30 at 95%; floor 0 beyond 95%.
Connectivity (15%)	100 when connected; 0 when disconnected.
Software health (15%)	100 with antivirus present; 60 otherwise; 80 when no software data.
Level	≥80 healthy; ≥50 warning; below critical.

29.4 Monitoring Thresholds

Method	Endpoint	Purpose
MONITORING_WARNING_SECONDS	300	First offline warning.
MONITORING_OFFLINE_SECONDS	900	Marked offline.
MONITORING_CRITICAL_SECONDS	1800	Critical offline alert.
STALE_THRESHOLD (SETTING)	120 s	Client considered stale/offline.
HEARTBEAT INTERVAL	30 s	Agent ping cadence.
ANOMALY Z-THRESHOLD / IQR / LOOKBACK	2.5 / 1.5 / 168 h	Statistical detectors.

29.5 Ports

Method	Endpoint	Purpose
80 / CUSTOM (TCP)	HTTP + WebSocket	Admin web + API + WS upgrade.
443 (TCP)	HTTPS	Production TLS.
45000 (UDP)	Discovery	ADMIN_HERE broadcasts / DISCOVER_ADMIN queries.

29.6 Glossary

Agent	Client-side process that scans, monitors, and reports to the admin server.
Client	Server-side record of a machine (key, fingerprint, approval, status).
Registration key	8-char identity used by all HTTP calls; lives in client_key.json.
Hardware fingerprint	SHA-256 of motherboard/CPU/disk/MAC identity; survives reinstalls.
DeviceMonitoringInfo	Per-device monitoring state row in the monitoring module.
AgentSecret	HMAC credential pair (agent_id + secret) for agent authentication.
Event bus	In-process pub/sub that turns every notable occurrence into alerts, history, notifications, and broadcasts.
PendingScan	Offline scan queue; delivered to an agent when it reconnects.
Health score	0-100 composite of CPU, RAM, disk, connectivity, and software health.
Soft delete	Marking a row deleted=True instead of removing it, preserving history.
Company	Tenant entity; isolates fleet data and settings.
Cloud registry	Supabase server_registry table used for discovery.

This document was generated from the [project source and configuration](#). The authoritative code lives in `admin/` and `client/`. For the connection lifecycle see `admin/scanner_api/views.py` and `client/communicator.py`; for monitoring, `WebSocket`, `scheduler`, and `AI` see `admin/monitoring/`; for insights see `admin/intelligence/` and `admin/maintenance/`.

30. Deep-Dive: REST API Worked Examples

This section shows real request/response shapes for the most important endpoints. All bodies are JSON (Content-Type: application/json) unless stated otherwise. Agent HTTP endpoints are authenticated with HMAC-SHA256 over the raw body using headers X-Agent-ID, X-Signature, and X-Timestamp (max age 300 s), per `admin/monitoring/security.py`. Browser and API-key endpoints use session cookies, JWT, or API keys. Error

responses are DRF JSON with a "detail" key.

30.1 Session login

```
POST /api/auth/login (no auth required)
{
  "username": "admin",
  "password": "StrongPass!123",
  "remember_me": true
}
200 OK -> {
  "user": { "id": 1, "username": "admin", "email": "admin@example.com", "is_superuser": true },
  "message": "Login successful"
}
401 -> { "detail": "Invalid credentials", "attempts_remaining": 4 }
403 -> { "detail": "Account locked" }
```

A successful login writes an AuditLog (login_success), a LoginHistory row, records the IP and user agent, and issues the session cookie (scanner_auth, TimestampSigner, 30 days if remember_me). Failed logins write LoginAttempt rows and, past the lockout threshold, lock the account (see admin/scanner_api/auth_utils.py and validators.py).

30.2 JWT token endpoints

```
POST /api/auth/token/obtain
{
  "username": "admin",
  "password": "StrongPass!123"
}
200 OK -> { "access": "eyJhbGciOiJIUzI1NiIs... ", "refresh": "eyJhbGciOiJIUzI1NiIs... " }
POST /api/auth/token/refresh
{
  "refresh": "eyJhbGciOiJIUzI1NiIs... "
}
200 OK -> { "access": "eyJhbGciOiJIUzI1NiIs... " }
POST /api/auth/token/verify
{
  "token": "eyJhbGciOiJIUzI1NiIs... "
}
200 OK (valid) | 401 (expired or invalid)
```

JWT settings (admin/django_admin/settings.py): HS256, access token 60 minutes, refresh token 7 days, issuer "system-scanner-pro", signed with the Django SECRET_KEY.

30.3 API key management

```
GET /api/auth/api-keys -> { "api_keys": [ { "id": 1, "name": "CI", "is_active": true,
"created_at": "2026-08-14T09:00:00Z" } ] }
POST /api/auth/api-keys
{
  "name": "automation-1"
}
201 -> { "id": 2, "name": "automation-1", "key": "<plaintext shown once>",
"is_active": true }
DELETE /api/auth/api-keys/2 -> 204 No Content
```

Keys are generated with token_hex(32) and stored hashed (SHA-256) in the api_keys table (admin/scanner_api/api_key_auth.py). API-key requests are rate limited to 60 per minute.

30.4 Client registration (core scanner_api)

```
POST /api/register (no auth required)
```

```
{
  "hostname": "DESKTOP-AB12CD",
  "ip_address": "192.168.1.50",
  "mac_address": "AA:BB:CC:DD:EE:FF",
  "os": "Windows 10 Pro",
  "platform": "Windows",
  "client_version": "1.6.1",
  "username": "kailas",
  "device_type": "desktop"
}
201 Created -> { "client_key": "Ab12cD34", "registered": true, "approved": false }
409 Conflict -> { "detail": "..."} (duplicate key / fingerprint collision)
```

The agent calls this from client/communicator.py register(); the server keeps the same registration_key, refreshes the fingerprint, and resets status to pending so the admin can re-approve after a DB reset. Approval state is returned so the agent can wait for the admin (see client/main.py heartbeat_loop).

30.5 Monitoring agent registration (HMAC)

```
POST /api/monitoring/agent/register
```

```
X-Agent-ID: <agent_uuid>
```

```
X-Signature: <hmac-sha256-hex>
```

```
X-Timestamp: 1723600000.123
```

```
{
  "hostname": "DESKTOP-AB12CD",
  "os": "Windows 10 Pro",
  "platform": "Windows",
  "version": "1.6.1",
  "architecture": "AMD64",
  "cpu": "Intel Core i7-12700",
  "memory": "32768 MB",
  "hd": "1 TB SSD",
  "ip_address": "192.168.1.50",
  "mac_address": "AA:BB:CC:DD:EE:FF",
  "username": "kailas"
}
200 OK -> { "agent_id": "<uuid>", "secret_key": "<64-char-hex>",
  "approved": false }
```

The signature is HMAC-SHA256(key = secret_key, message = raw request body) as a hex digest (admin/monitoring/security.py compute_signature). The agent stores the returned secret_key and uses it for every subsequent signed call. RateLimiter caps requests at 60 per minute per key.

30.6 Agent heartbeat (public variant used by the agent)

```
POST /api/monitoring/agent/heartbeat-public
```

```
{
  "registration_key": "Ab12cD34",
  "cpu_usage_pct": 34.5,
  "ram_usage_pct": 61.2,
  "disk_usage_pct": 47.8,
  "disk_free_gb": 512.3,
  "disk_total_gb": 1000.0,
  "network_connected": true,
  "uptime_seconds": 482000,
  "load_average": 0.8,
  "agent_version": "1.6.1",
  "hostname": "DESKTOP-AB12CD",
  "current_user": "kailas"
}
```

```
}
200 OK -> { "status": "ok" }
```

The current agent sends heartbeats over this public HTTP endpoint every ~5 seconds (client/main.py heartbeat_loop -> monitor_heartbeat_public), which updates DeviceMonitoringInfo.last_heartbeat, computes the health score, runs check_and_create_alerts, and publishes health_level_changed / agent_version_changed events when values change.

30.7 Scan submission and history

```
POST /api/scan
{
  "client_key": "Ab12cD34",
  "scan_type": "scheduled",
  "scan_data": { "hostname": "DESKTOP-AB12CD", "platform": "Windows", "cpu": "...", "ram":
  "..."}
}
201 Created -> { "status": "ok" }
GET /api/scan/history?limit=50&offset=0
200 -> { "history": [ { "client_key": "Ab12cD34", "scan_type": "scheduled",
"created_at": "2026-08-14T09:00:00Z", "changes": null } ],
"total": 123, "limit": 50, "offset": 0 }
```

Scan payloads are diffed against the previous scan with compute_scan_diff (admin/scanner_api/diff_utils.py) so the dashboard can highlight added/removed/changed hardware and software on the client detail page.

30.8 Scheduled scan management

```
GET /api/monitoring/schedules?limit=50
200 -> { "schedules": [ { "id": "<uuid>", "name": "Daily full scan",
"schedule_type": "daily",
"time_of_day": "02:00:00", "scan_type": "full", "target_all": true,
"enabled": true, "last_run": null, "next_run": "2026-08-15T02:00:00Z",
"run_count": 0 } ], "total": 1 }
POST /api/monitoring/schedules
{
  "name": "Nightly full scan",
  "schedule_type": "daily",
  "time_of_day": "02:00:00",
  "scan_type": "full",
  "target_all": true,
  "enabled": true
}
201 -> { "id": "<uuid>", "name": "Nightly full scan", ... }
```

Schedules are mirrored into the APScheduler jobs at runtime (admin/monitoring/scheduler.py). When a job fires, online agents receive a WS command scan_now; offline agents get a PendingScan row that they pick up over HTTP from /api/monitoring/agent/pending-scans (header X-Registration-Key or query param key) and acknowledge with POST + status executed.

30.9 Maintenance record creation

```
POST /api/maintenance/maintenance
{
  "asset": "<uuid>",
  "maintenance_type": "Preventive",
  "priority": "High",
  "scheduled_date": "2026-09-01",
  "estimated_cost": "250.00",
  "vendor_name": "Acme IT Services",
  "description": "Quarterly cleaning and thermal repaste"
```

```

}
201 Created -> { "id": "<uuid>", "maintenance_id": "MNT000001", "status":
"Draft",
"approval_status": "Pending", "asset_name": "Laptop-01", ... }

```

Maintenance ids are auto-generated as MNT%06d in save(). Creating or updating a record writes MaintenanceHistory and runs maintenance.alerts.check_and_generate_alerts to detect due/overdue records.

30.10 Intelligence alerts

```

GET /api/intelligence/alerts?severity=warning&status=open&page_size=50&page=1
200 -> { "alerts": [ { "id": "<uuid>", "title": "Disk usage above 90%", "module":
"monitoring",
"severity": "warning", "category": "monitoring", "status": "open",
"escalation_level": 0, "age_hours": 3 } ], "total": 7, "page": 1 }
POST /api/intelligence/alerts/<uuid>/action
{
"action": "ack"
}
200 -> { "status": "ok", "alert": { "...": "..." } }
Actions: ack | resolve | escalate | dismiss

```

Alert creation deduplicates by SHA-256 dedup_hash with a suppression window (default 60 minutes), and escalation raises escalation_level over time (admin/intelligence/alerts.py). Resolving or dismissing an alert stamps resolved_time automatically.

30.11 Report generation

```

POST /api/intelligence/reports/generate
{
"report_type": "fleet_summary",
"format": "pdf",
"filters": { "status": "online", "platform": "Windows" }
}
200 -> { "id": "<uuid>", "name": "...", "status": "completed", "file_size":
182340,
"generated_at": "2026-08-14T09:00:00Z" }

```

Fleet/device/alert PDFs are also downloadable directly from the monitoring module (/api/monitoring/reports/fleet/pdf, /reports/device/pdf, /reports/alerts/pdf) as application/pdf attachments built by admin/monitoring/reports.py.

31. Deep-Dive: WebSocket Protocol and Message Catalog

Two Django-Channels sockets are exposed (admin/monitoring/routing.py). All frames are JSON text messages carrying a "type" key. The agent socket is authenticated with a secret-key lookup plus a timestamp freshness check; the dashboard socket relies on the session cookie and admits unauthenticated browsers in read-only mode.

Agent socket	ws/agent/<agent_id>/ - AgentConsumer. Used by the installed client agent.
Dashboard socket	ws/dashboard/ - DashboardConsumer. Used by the browser dashboard.

31.1 Connection and auth handshake

```

1. Client opens ws(s)://HOST/ws/agent/<agent_id>/
2. Server accepts and immediately sends:
{ "type": "auth_required", "message": "...", "agent_id": "<agent_id>" }
3. Client replies (client/communicator.py WebSocketClient._authenticate):
{ "type": "auth", "agent_id": "<agent_id>",
"secret": "<agent_secret_key>", "timestamp": "1723600000.123" }
4. Server validates agent_id match, non-empty secret, timestamp within 300 s,
then AgentSecret lookup (agent_id + secret_key + is_active=True).

```



```

Success -> { "type": "auth_success", "message": "...",
"server_time": "...", "pending_commands": [] }
Failure -> { "type": "auth_failed", "message": "Invalid credentials" } + close 4003
Missing agent_id in path -> close 4001

```

On auth success the server joins channel group `agent_<id>`, marks the device online (`Client.status`, `DeviceMonitoringInfo.monitoring_status`), broadcasts `agent_status=online` to the dashboard, and delivers any pending commands. If the WebSocket upgrade is rejected with a plain HTTP status (serverless/Vercel), the agent sets `ws_unsupported` and permanently falls back to HTTP polling for that process.

31.2 Server-to-agent messages

Method	Endpoint	Purpose
AUTH_REQUIRED	ws accept	Server demands authentication (first message after connect).
AUTH_SUCCESS	valid auth	Auth accepted; carries <code>pending_commands</code> that the client runs.
AUTH_FAILED	bad auth	Auth rejected; server closes with code 4003.
COMMAND	scan job / admin	Remote command: { <code>command_type</code> , <code>command_id</code> , <code>payload</code> }.
PING	keepalive	{ <code>server_time</code> }; client answers pong. Dormant - no active sender.
HEARTBEAT_ACK	after heartbeat	{ <code>health_score</code> , <code>health_level</code> , <code>pending_commands</code> , <code>server_time</code> }.
SCAN_ACK	after scan_result	{ <code>status</code> : ok error, <code>message</code> }.
ERROR	bad frame	Invalid JSON, pre-auth message, or unknown type.

```

scan_now command sent by the scheduler for online agents:
{ "type": "command", "command_type": "scan_now",
"command_id": "scan_Ab12cD34_1723600000",
"payload": { "scan_type": "full", "schedule_id": "<schedule-uuid>" } }
Client reaction (client/main.py handle_ws_command):
1. Runs collect_all() locally.
2. Submits the scan via HTTP POST /api/scan.
3. Replies { "type": "scan_result", "scan_type": "on_demand",
"scan_data": { "hostname": "...", "platform": "..." } }
config_update: merges payload.interval_seconds / payload.enabled into the local
scan config (designed; no active server sender today).
ping command: client replies { "type": "pong" }.

```

31.3 Agent-to-server messages

Method	Endpoint	Purpose
AUTH	handshake	{ agent_id, secret, timestamp }. signature is read but not verified.
EVENT	event dispatcher	{ event_type, severity, event_data }. Batched every 5 s / 50 max.
SCAN_RESULT	after scan_now	{ scan_type, scan_data } stored as a ScanResult row.
PONG	reply to ping	Keeps the server-side connection alive.
HEARTBEAT	agent heartbeat	Full metrics heartbeat over WS (implemented, not used by agent).
STATUS_UPDATE	status change	{ status } updates DeviceMonitoringInfo.monitoring_status.

Event message as sent by client/events/dispatcher.py:

```
{ "type": "event", "event_type": "usb_device_connected",
"severity": "warning",
"event_data": { "title": "USB device connected",
"message": "SanDisk 3.2Gen1 attached",
"device_id": "USB\\VID_0781&PID_5591\\...",
"device_name": "SanDisk 3.2Gen1", "type": "storage",
"serial": "AA0123456789", "vendor": "SanDisk" } }
```

Server _store_event: writes DeviceHistory (category security_event for critical, status_change otherwise), and for warning/critical creates a DeviceAlert and broadcasts new_alert to the dashboard.

31.4 Dashboard broadcast messages

Method	Endpoint	Purpose
AGENT_STATUS	auth/disconnect	{ agent_id, status: online offline, timestamp }.
DEVICE_HEARTBEAT	WS heartbeat	{ agent_id, health_score, health_level, cpu, ram, disk, timestamp }.
SCAN_COMPLETED	WS scan_result	{ agent_id, scan_type, timestamp }.

Method	Endpoint	Purpose
DEVICE_EVENT	WS event	{ agent_id, event_type, severity, timestamp }.
DEVICE_STATUS_UPDATE	WS status_update	{ agent_id, status, timestamp }.
NEW_ALERT	warning/critical event	{ agent_id, event_type, severity, details, timestamp }.
	event bus	Every EventBus publication is forwarded to the dashboard group.

The event bus forwards every published event type to the dashboard with keys event_type, client_id, client_key, hostname, severity, title, description, data, timestamp, source (Event.to_dict in admin/monitoring/event_bus.py). Real published types include device_registered, agent_version_changed, health_level_changed, hw_component_added/removed/modified, sw_installed/sw_removed/sw_version_changed/sw_unauthorized/sw_antivirus_removed, device_status_changed, device_approved, device_blocked, and schedule_executed.

31.5 Offline handling and recovery

- WS disconnect: AgentConsumer.disconnect marks monitoring_status=offline (only if it was online) and broadcasts agent_status=offline.
- Independent detector: the offline_detector management command flips stale devices to offline after stale_threshold_seconds (default 120 s) and creates a device_offline DeviceAlert.
- Queued events: while disconnected, agent events buffer in memory and are persisted to %APPDATA%/SystemScannerPro/offline_events/batch_*.json, replayed on the next dispatcher start.
- Pending scans: the scheduler creates PendingScan rows for offline agents; agents fetch and acknowledge them over HTTP.
- Reconnect: the agent reconnects with exponential backoff 2 s to 60 s, re-authenticating each time.

Accuracy notes: the agent currently sends heartbeats over HTTP, not WS, so heartbeat_ack/device_heartbeat are implemented but rarely fire. WS auth is a plain secret-key lookup plus timestamp check - the signature field is accepted but not verified. The pending_commands delivery path reads DeviceMonitoringInfo.notes, which nothing writes today, so it is dormant. The dashboard frontend listens for alert_created but the server only emits new_alert.

32. Deep-Dive: Database Schema Reference

Every model is listed below with its full field set as defined in the models.py files. Unless noted, id is UUIDField(primary_key=True, default=uuid.uuid4, editable=False). All apps use BigAutoField as default_auto_field. The scanner_api app is the hub: the monitoring, intelligence, and maintenance apps all reference scanner_api tables (Client, Asset, Employee, Department, Location, Company) rather than duplicating them.

32.1 scanner_api: core and organization

32.1.1 Company (companies)

Field	Type	Purpose
name	CharField(255)	Unique company name.

Field	Type	Purpose
slug	SlugField(128)	Unique, blank; auto-slugified in save().
created_at	DateTimeField	Auto-set on create.

32.1.2 ClientGroup (client_groups)

Field	Type	Purpose
name	CharField(128)	Group name.
description	TextField	Blank by default.
company	FK Company	CASCADE, null/blank, related_name=client_groups.
created_at	DateTimeField	Auto-set.

32.1.3 Client (clients) - central agent record

Field	Type	Purpose
registration_key	CharField(64)	Unique, indexed; agent identity.
hostname / platform	CharField	Machine name and OS family.
status	CharField(32)	pending / online / offline / etc.; default pending.
last_seen	DateTimeField	Null/blank.
approved	BooleanField	Default False.
owner	FK auth.User	SET_NULL, related_name=owned_clients.
company	FK Company	CASCADE, related_name=clients.
group	FK ClientGroup	SET_NULL, related_name=clients.
tags	CharField(512)	Comma tags, blank.
purchase_cost	DecimalField(12,2)	Null/blank.
purchase_date	DateField	Null/blank.
vendor_name / vendor_contact	CharField	Blank.
warranty_expiry	DateField	Null/blank.
notes	TextField	Blank.
scan_interval	IntegerField	Default 3600 s.
scan_enabled	BooleanField	Default True.
scan_requested	BooleanField	Default False.
last_ip	CharField(64)	Blank.
device_fingerprint	CharField(64)	Indexed, blank.
deleted	BooleanField	Soft delete flag.
client_version / os_version	CharField	Agent and OS version strings.
cpu_model / ram_info	CharField	Inventory summary strings.
created_at	DateTimeField	Auto-set.

32.1.4 ScanResult (scan_results)

Field	Type	Purpose
client	FK Client	CASCADE, null/blank, related_name=scans.
scan_type	CharField(32)	scheduled / full / quick / ...; default scheduled.
scan_data	JSONField	Full inventory JSON, default {}.
created_at	DateTimeField	Indexed with client (desc).

32.1.5 AddonDevice (addon_devices)

Field	Type	Purpose
client	FK Client	CASCADE, related_name=addons.
name / description / serial_number	Char/Text	Device metadata.
purchase_cost	DecimalField(12,2)	Null/blank.
category	CharField(128)	Blank.
added_at	DateTimeField	Auto-set.

32.1.6 ActivityLog (activity_logs)

Field	Type	Purpose
action	CharField(32)	register/approve/scan/scan_request/delete/update/login/setting_change.
client	FK Client	SET_NULL, related_name=activity_logs.
company	FK Company	CASCADE, related_name=activity_logs.
details	TextField	Blank.
created_at	DateTimeField	Indexed desc.

32.1.7 Setting (settings)

Field	Type	Purpose
key	CharField(255)	Primary key (e.g. stale_threshold_seconds).
value	TextField	Blank.
company	FK Company	CASCADE, null/blank, related_name=settings.

32.1.8 AdministratorProfile (administrator_profiles)

Field	Type	Purpose
user	OneToOne auth.User	CASCADE, related_name=admin_profile.
company	FK Company	CASCADE, related_name=admins.
phone_number / profile_picture_url	Char/URL	Optional profile data.
timezone / currency / date_format	CharField	Defaults UTC / USD / YYYY-MM-DD.
dashboard_default	CharField(50)	Default dashboard.
notification_email / notification_in_app	BooleanField	Default True.

Field	Type	Purpose
notification_daily_summary	BooleanField	Default False.
password_changed_at	DateTimeField	Null/blank.
mfa_enabled / mfa_secret	Bool / Char	MFA support fields.
created_at / updated_at	DateTimeField	Auto timestamps.
constraint	unique_together	(company, user).

32.1.9 AuditLog (audit_logs)

Field	Type	Purpose
user	FK auth.User	SET_NULL, no related_name.
company	FK Company	CASCADE, related_name=audit_logs.
event_type	CharField(32)	login_success/failure, logout, password_changed, profile_updated, settings_updated, account_locked/unlocked, session_created/expired, etc.
ip_address / user_agent	GenericIP / Text	Request context.
device_info	JSONField	Blank dict.
details	TextField	Blank.
success	BooleanField	Default True.
created_at	DateTimeField	Indexed.

32.1.10 LoginHistory / LoginAttempt / DeviceFingerprint

Field	Type	Purpose
LoginHistory	login_time, logout_time, session_duration, ip_address, user_agent, browser, os, device_type, location, is_current	Per-user login timeline (user FK SET_NULL, company FK).
LoginAttempt	identifier, ip_address, success, created_at	Brute-force tracking, identifier indexed.
DeviceFingerprint	user, fingerprint, device_name, browser, os, last_seen, trusted	Trusted-device tokens (auto BigAutoField id).

32.1.11 Location (locations)

Field	Type	Purpose
company	FK Company	CASCADE, related_name=locations.

Field	Type	Purpose
office_name / building_name / floor / room_number	CharField	Physical address details.
address / city / state / country / postal_code	Char/Text	Country default USA.
contact_number / office_manager / timezone	CharField	Timezone default UTC.
status	CharField(20)	Active / Archived / Closed; default Active.
notes	TextField	Blank.
deleted	BooleanField	Soft delete.
created_at / updated_at	DateTimeField	Auto timestamps.
constraint	unique_together	(office_name, city).

32.1.12 Department (departments)

Field	Type	Purpose
company	FK Company	CASCADE, related_name=departments.
name / code	CharField	Name + short code.
description / department_head / email / phone_number	Char/Text	Optional contact info.
location	FK Location	SET_NULL, related_name=departments.
budget	DecimalField(14,2)	Null/blank.
status	CharField(20)	Active / Disabled / Archived; default Active.
deleted	BooleanField	Soft delete.
created_at / updated_at	DateTimeField	Auto timestamps.

32.1.13 Employee (employees)

Field	Type	Purpose
company	FK Company	CASCADE, related_name=employees.
employee_code	CharField(50)	Unique per company.
full_name / email	Char / Email	Email unique per company.
phone_number	CharField(20)	Blank.
department	FK Department	PROTECT, related_name=employees.
designation / manager_name	CharField	Job info.
reports_to	FK self	SET_NULL, related_name=direct_reports.
location	FK Location	PROTECT, related_name=employees.
joining_date	DateField	Null/blank.

Field	Type	Purpose
status	CharField(20)	Active/Inactive/Resigned/On Leave/Terminated/Retired; default Active.
profile_image	TextField	Base64, blank.
notes / deleted	Text / Bool	Notes and soft delete.
created_at / updated_at	DateTimeField	Auto timestamps.

32.1.14 EmployeeAssetAssignment (employee_asset_assignments)

Field	Type	Purpose
employee	FK Employee	PROTECT, related_name=asset_assignments.
client	FK Client	SET_NULL, related_name=employee_assignments.
assigned_at / returned_at	DateTimeField	Auto-set / null.
is_active	BooleanField	Default True.
assigned_by / notes	Char / Text	Actor and notes.

32.1.15 OrgAuditLog (org_audit_logs)

Field	Type	Purpose
company	FK Company	CASCADE, related_name=org_audit_logs.
entity_type	CharField(20)	employee / department / location.
entity_id / entity_name	CharField	Referenced entity.
action	CharField(20)	created/updated/deactivated/archived/disabled.
previous_value / new_value	JSONField	Change snapshots.
performed_by / ip_address / user_agent	Char / IP / Text	Audit context.
created_at	DateTimeField	Indexed.

32.1.16 Asset categories and vendors

Field	Type	Purpose
AssetCategory	name, code, description, parent (self FK), icon, is_active	Hierarchical asset taxonomy; parent SET_NULL related_name=children.
AssetVendor	name, contact_person, email, phone, address, website, notes	Vendor directory; company FK.

32.1.17 Asset (assets) - largest table

Field	Type	Purpose
company	FK Company	CASCADE, related_name=assets.
asset_id	CharField(32)	Unique; auto AST%06d in save().
asset_name / asset_tag	CharField	asset_tag unique+indexed.

Field	Type	Purpose
serial_number	CharField(255)	Unique, indexed.
qr_code / barcode	UUIDField	Both unique and indexed, default uuid4.
category	FK AssetCategory	SET_NULL, related_name=assets.
manufacturer / model_name / description / specifications	Char/Text/JSON	specifications is JSON dict.
image	TextField	Base64, blank.
purchase_date / purchase_cost / current_value / residual_value	Date/Decimal	Financial values.
depreciation_pc t	DecimalField(5,2)	Default 0.
invoice_number / purchase_orde r_number	CharField	Procurement refs.
vendor	FK AssetVendor	SET_NULL, related_name=assets.
department / location	FK	SET_NULL, related_name=assets.
warranty_start / warranty_end / warranty_prov ider / amc_details	Date/Char/Text	Warranty info.
asset_status	CharField(24)	13 states Draft..Disposed; indexed, default Draft.
parent	FK self	SET_NULL, related_name=children (asset hierarchy).
client	FK Client	SET_NULL, related_name=linked_assets.
is_insured / insurance_value / insurance_exp iry	Bool/Decimal/Date	Insurance.
created_by / last_audit_date / tags / notes	Char/Date/Text	Meta info.
is_active / deleted	BooleanField	State flags.
assigned_to	FK Employee	SET_NULL, related_name=assigned_assets.
created_at / updated_at	DateTimeField	Auto timestamps.

32.1.18 AssetAssignment / AssetTransfer / AssetHistory / AssetDocument

Field	Type	Purpose
AssetAssignment	asset (PROTECT), employee (PROTECT), department, location, assigned_at, expected_return_date, returned_at, is_active, assigned_by, notes	Table asset_assignments_v2.
AssetTransfer	asset (PROTECT), from/to employee/department/location, transfer_date, reason, transferred_by, notes	Asset movement trail.
AssetHistory	asset (PROTECT), action, timestamp, previous_value, new_value, performed_by, ip_address, user_agent, notes	Immutable: save() raises when pk set; update() raises.
AssetDocument	asset (PROTECT), name, file_data (base64), file_type, file_size, uploaded_by	Attached documents.

32.1.19 ApiKey (api_keys)

Field	Type	Purpose
key	CharField(64)	Unique; plaintext shown once at creation.
key_hash	CharField(128)	SHA-256 stored hash, indexed.
name / user	Char / FK User	Label and owner (CASCADE, related_name=api_keys).
is_active	BooleanField	Default True.
rate_limit	IntegerField	Requests/minute, default 60.
allowed_ips / expires_at / last_used	Text / DateTime	Optional restrictions.
created_at	DateTimeField	Auto-set.

32.2 monitoring: device intelligence

32.2.1 DeviceMonitoringInfo (monitoring_device_info)

Field	Type	Purpose
client	OneToOne Client	CASCADE, related_name=monitoring_info, indexed.
monitoring_status	CharField(20)	pending/approved/rejected/blocked/online/offline/inactive/maintenance/unknown; default pending.
health_level	CharField(20)	healthy/warning/critical/unknown; default unknown.
health_score	IntegerField	0-100.
ip_address / mac_address / public_ip	IP / Char	Network identity.
os_name / os_build / os_architecture	CharField	OS details.
agent_version / last_heartbeat / heartbeat_count	Char/DateTime/Int	Agent status.
device_type	CharField(32)	laptop/desktop/server/workstation/vm/cloud; default desktop.
department / location_name / current_user	CharField	Placement info.
tags / notes	Char / Text	Free-form.
created_at / updated_at	DateTimeField	Auto timestamps.

32.2.2 HardwareInventory / SoftwareInventory

Field	Type	Purpose
HardwareInventory	client FK, component_type (cpu/ram/storage/gpu/motherboard/network), component_data JSON, fingerprint, scan_id, created_at	Per-component hardware snapshots.
SoftwareInventory	client FK, name (512, indexed), version, publisher, category (application/driver/update/browser/office/antivirus/service/other), raw_data JSON, scan_id, is_present, created_at	Installed software entries with presence flag.

32.2.3 DeviceHeartbeat (monitoring_device_heartbeat)

Field	Type	Purpose
client	FK Client	CASCADE, related_name=monitoring_heartbeats.

Field	Type	Purpose
cpu_usage_pct / ram_usage_pct / disk_usage_pct / disk_free_gb / disk_total_gb / load_average	FloatField	Metric readings, default 0.
network_connected	BooleanField	Default True.
uptime_seconds / scan_running / pending_commands	Int/Bool/Int	Status extras.
agent_version / ip_address	Char / IP	Context.
response_time_ms / created_at	Int / DateTime	Latency and timestamp.

32.2.4 DeviceMetrics (monitoring_device_metrics)

Field	Type	Purpose
client	FK Client	CASCADE, related_name=device_metrics.
period	CharField(8)	hourly / daily; default hourly.
avg/max/min_cpu_pct, avg/max/min_ram_pct, avg/max/min_disk_pct	FloatField	9 aggregates.
health_score / health_level	Int / Char	Rolled-up health.
uptime_pct / total_heartbeats / missed_heartbeats	Float/Int	Availability.
period_start / period_end	DateTimeField	Bucket window.

32.2.5 DeviceHistory (monitoring_device_history) - immutable

Field	Type	Purpose
client	FK Client	PROTECT, related_name=device_history.
category	CharField(20)	registration/status_change/hardware_change/software_change/health_change/security_event/alert_generated/admin_action/agent_update/remote_command.
event_type / description	Char / Text	Event name and text.
previous_value / new_value	JSONField	Change diff.
severity	CharField(16)	info/warning/critical; default info.
source	CharField(64)	system/agent/etc.
timestamp	DateTimeField	Auto-set; save() raises if pk set.

32.2.6 DeviceAlert (monitoring_device_alerts)

Field	Type	Purpose
client	FK Client	CASCADE, related_name=monitoring_alerts.

Field	Type	Purpose
alert_type	CharField(64)	Indexed (e.g. device_offline).
severity	CharField(16)	info/warning/critical; default warning.
status	CharField(16)	active/acknowledged/resolved/dismissed; default active.
title / message / details	Char / Text / JSON	Alert content.
acknowledged_by / acknowledged_at / resolved_at	Char / DateTime	Workflow stamps.

32.2.7 AgentVersion / AgentSecret

Field	Type	Purpose
AgentVersion	version (unique), release_notes, download_url, is_mandatory, min _python_version (default 3.7), file_hash, is_active	Version registry used by agent/version-check.
AgentSecret	client FK, agent_id (unique), secret_key, device_fingerprint, is_active, last_used	HMAC secrets issued at agent registration.

32.2.8 ScheduledScan / PendingScan / ScanScheduleLog

Field	Type	Purpose
ScheduledScan	name, description, schedule_type (int erval/daily/weekly/ monthly/once), interval_seconds, cron_expression, time_of_day, day_of_week, day_of_month, target_all, target_clients (ManyToMany - the only M2M in the schema), target_platforms, scan_type, enabled, last_run, next_run, run_count	Scan schedules mirrored into APScheduler.

Field	Type	Purpose
PendingScan	client FK, scheduled_scan FK, scan_type, priority, status (pending/sent/executed/failed/expired), sent_at, executed_at, error_message	Offline-scan queue; immutable (save raises when pk set).
ScanScheduleLog	scheduled_scan FK, client FK, triggered_at, completed_at, status, changes_detected, alerts_generated, details	Execution log per schedule+client.

32.3 intelligence: alerts, insights, audit

32.3.1 Alert (intelligence_alerts)

Field	Type	Purpose
title / description	Char / Text	Alert headline and detail.
module	CharField(20)	asset / monitoring / maintenance / license.
source_object_id / source_object_type	CharField	Generic source reference.
severity	CharField(16)	information/warning/critical/emergency; default warning.
category	CharField(20)	asset/monitoring/maintenance/license/security/compliance/system.
status	CharField(16)	open/acknowledged/resolved/dismissed; default open.
assigned_user	CharField(128)	Blank.
generated_time / resolved_time	DateTimeField	Auto / null until resolved.
escalation_level	IntegerField	0 by default.
dedup_hash	CharField(64)	SHA-256 dedupe key, indexed.
notification_sent	BooleanField	Default False.
resolution_notes	TextField	Blank.

32.3.2 AlertHistory / AlertRule

Field	Type	Purpose
AlertHistory	alert FK, action, previous_status/new_status, previous_severity/new_severity, performed_by, notes, timestamp	Immutable state-transition log per alert.
AlertRule	name, description, module, category, severity, condition_type, condition_field, condition_value, suppress_duplicates, suppress_window_minutes (60), auto_resolve_minutes, is_active, created_by	Rule-engine definitions for automated alerting.

32.3.3 Notification / NotificationPreference

Field	Type	Purpose
Notification	user FK, title, message, severity, status (unread/read/archived), module, source_alert FK, source_url, created_time, read_time	Per-user in-app notifications.
NotificationPreference	user OneToOne, email_enabled, in_app_enabled, severity_* (4 flags), module_* (7 flags), frequency (instant/daily/weekly/never), quiet_hours_start/end	Delivery preferences.

32.3.4 Report / ScheduledReport

Field	Type	Purpose
Report	name, report_type (25 choices), format (pdf/excel/csv), filters JSON, generated_by, file_data (base64), file_size, row_count, status (pending/generating/completed/failed), error_message, generated_at, completed_at	Generated report artifacts.
ScheduledReport	name, report_type, format, frequency (daily/weekly/monthly/quarterly), filters, recipients, next_run, last_run, is_active, retention_policy (1_year/3_years/permanent)	Recurring report config (CRUD today; no background executor).

32.3.5 AuditLogEntry / ComplianceLog / DashboardAnalytics / RetentionPolicy

Field	Type	Purpose
AuditLogEntry	user_id (plain int, no FK), username, timestamp, ip_address, browser_info, device_info, module (9 choices), action (21 choices), object_type, object_id, object_repr, old_value, new_value, severity, description	Immutable security audit trail.

Field	Type	Purpose
ComplianceLog	framework (iso_27001/itil/soc2/internal/gdpr), control_id, control_name, status, severity, description, finding_details, asset FK, audited_by, audited_at, next_audit_date	Compliance findings.
DashboardAnalytics	9 counters (total_alerts, open_alerts, critical_alerts, notifications_today, reports_generated, security_violations, audit_events_today, compliance_violations, pending_notifications), snapshot_time	Periodic KPI snapshots.
RetentionPolicy	scope (unique: alerts/notifications/reports/audit_logs/compliance_logs), retention_period, is_active	Per-scope retention settings.

32.4 maintenance: asset care, licenses, compliance

32.4.1 MaintenanceRecord (maintenance_records)

Field	Type	Purpose
maintenance_id	CharField(32)	Unique; auto MNT%06d.
asset	FK Asset	PROTECT, related_name=maintenance_records.
asset_category_name	CharField(128)	Copied from asset.category in save().
maintenance_type	CharField(20)	Preventive/Corrective/Emergency/Inspection/Upgrade/Repair/Replacement/Calibration.
status	CharField(24)	Draft/Pending Approval/Approved/Scheduled/In Progress/Waiting Parts/Completed/Cancelled/Overdue.
approval_status	CharField(16)	Pending/Approved/Rejected.
vendor_name / vendor_contact / technician	CharField	Service provider.
description / notes	TextField	Details.

Field	Type	Purpose
scheduled_date / start_date / completion_date / due_date	DateField	Timeline.
estimated_cost / actual_cost / downtime_hours	DecimalField	Costs and downtime.
priority	CharField(16)	Low/Medium/High/Critical; default Medium.
recurring / recurrence_interval_days / next_occurrence	Bool/Int/Date	Recurring maintenance.
created_by / approved_by	CharField	Actors.
department	FK Department	SET_NULL, related_name=maintenance_records.
deleted	BooleanField	Soft delete.

32.4.2 MaintenanceHistory / MaintenanceDocument

Field	Type	Purpose
MaintenanceHistory	maintenance FK, action, description, previous_value, new_value, performed_by, ip_address, timestamp	Immutable workflow log.
MaintenanceDocument	maintenance FK, name, file_data (base64), file_type, file_size, uploaded_by	Attached documents.

32.4.3 WarrantyRecord / DowntimeRecord

Field	Type	Purpose
WarrantyRecord	warranty_id (auto WAR%06d), asset FK (PROTECT), warranty_start (required), warranty_end (required), warranty_provider (required), contract_number, amc_details, support_contacts, coverage_type (Full/Parts/Labor/Limited), status (Active/Expiring Soon/Expired/Claimed/Archived), cost	Warranty tracking.

Field	Type	Purpose
DowntimeRecord	asset FK (PROTECT), maintenance FK (SET_NULL), start_time, end_time, duration_hours (auto), reason (Ma intenance/Repair/ Failure/Upgrade/P ower/Network/Oth er), description	Downtime events.

32.4.4 SoftwareLicense / LicenseAssignment / LicenseHistory

Field	Type	Purpose
SoftwareLicense	license_id (auto LIC%06d), software_name, vendor, product_edition, version, license_k ey_encrypted, lice nse_key_masked, license_type (Per User/Per Device/S ubscription/OEM/ Enterprise/Volume /Trial/Open Source), purchased_seats, seats_used, purchase_date, expiration_date, renewal_date, cost, status (Draft/ Active/Expiring So on/Expired/Suspe nded/Archived), department FK, notes, deleted	Software entitlement tracking.
LicenseAssignme nt	license FK (PROTECT), assignable_type (Asset/Employee/D epartment), asset/ employee/departm ent FKs, assigned_date, removal_date, assigned_by, is_active	Seat consumption.

Field	Type	Purpose
LicenseHistory	license FK, action, description, previous_value, new_value, performed_by, timestamp	Immutable license log.

32.4.5 ComplianceRecord / MaintenanceAlert

Field	Type	Purpose
ComplianceRecord	category (license_expiration/seat_overuse/unauthorized_software/missing_license/compliance_violation), severity, title, description, license FK, asset FK, details JSON, status (active/acknowledged/resolved/dismissed), acknowledged_by, resolved_at	Compliance findings.
MaintenanceAlert	Generated by maintenance/alerts.py from due/overdue maintenance, warranty, and license expiry rules.	Alert queue for the maintenance module.

32.5 Relationship map

Component	Description
Client (hub)	Referenced by ScanResult, AddonDevice, ActivityLog, EmployeeAssetAssignment, Asset.client, DeviceMonitoringInfo, HardwareInventory, SoftwareInventory, DeviceHeartbeat, DeviceMetrics, DeviceHistory, DeviceAlert, AgentSecret, PendingScan, ScanScheduleLog, ScheduledScan.target_clients.
Asset	Referenced by AssetAssignment, AssetTransfer, AssetHistory, AssetDocument, MaintenanceRecord, WarrantyRecord, DowntimeRecord, LicenseAssignment, ComplianceRecord, ComplianceLog.
Employee	Referenced by EmployeeAssetAssignment, AssetAssignment, AssetTransfer, LicenseAssignment.
Company	Tenant root for most scanner_api, monitoring, intelligence, and maintenance tables.
auth.User	Owns AdministratorProfile, DeviceFingerprint, AuditLog, LoginHistory, LoginAttempt, ApiKey, Notification, NotificationPreference.
Department Location	/ Referenced by Employee, Asset, MaintenanceRecord, SoftwareLicense, LicenseAssignment, AssetTransfer.

33. Deep-Dive: Client Agent Source Walkthrough

The client agent is a lightweight Python package under `client/`. It runs on every managed machine, collects inventory and metrics, monitors system events, and talks to the admin server over REST and WebSocket. The paths below are relative to `client/`. Every class and function is referenced so any question about the agent can be traced to the exact location.

33.1 `main.py` - entry point (1247 lines)

Constants: `DISCOVERY_PORT=45000` (`main.py:134`), `VERSION="1.6.1"` (`main.py:135`), `OUTPUT_DIR` under the data directory (`main.py:136`). CLI flags are parsed manually from `sys.argv`: `--silent` (`main.py:550`) and `--rescue` (`main.py:553`).

Component	Location	Description
<code>_log_crash</code>	<code>main.py:12</code>	Bootstraps crash logging to a file before anything else runs.
<code>print_header</code>	<code>main.py:159</code>	Prints the ASCII banner and connection info.
<code>cloud_discovery_loop</code>	<code>main.py:209</code>	Background thread polling the cloud registry for the admin URL.
<code>listen_admin_broadcast</code>	<code>main.py:228</code>	UDP listener on <code>DISCOVERY_PORT 45000</code> to find the admin server on the LAN.
<code>_try_rediscover</code>	<code>main.py:254</code>	Fallback discovery after repeated heartbeat failures.
<code>handle_ws_command</code>	<code>main.py:279</code>	Handles WS commands: <code>scan_now</code> (runs <code>collect_all</code> + submits), <code>config_update</code> (merges scan config), <code>ping</code> (replies pong).
<code>heartbeat_loop</code>	<code>main.py:326</code>	Main loop: pings the server, re-registers if the row is missing, waits for approval, flushes offline queue, sends public heartbeats, and handles <code>trigger_scan</code> responses.
<code>HeartbeatWatchdog</code>	<code>main.py:446</code>	Monitors the heartbeat thread and restarts it if it dies (up to 5 restarts).
<code>start_websocket_client</code>	<code>main.py:481</code>	Starts the <code>WebSocketClient</code> thread.
<code>_start_event_monitors</code>	<code>main.py:495</code>	Starts USB, process, software, and file event monitors with their dispatchers.
<code>_spawn_background</code>	<code>main.py:589</code>	Windows: spawns a hidden background copy (noconsole), redirecting output to <code>client_agent.log</code> .
<code>_ensure_single_instance</code>	<code>main.py:668</code>	Named mutex so only one agent instance runs; closing the console window spawns a rescue copy.
<code>_register_autostart</code>	<code>main.py:739</code>	Registers the agent to start at login (registry Run key / Startup folder).
<code>main</code>	<code>main.py:776</code>	Top-level orchestration: print header, load config, resolve admin URL, ensure single instance, start background if needed, run heartbeat loop + monitors + WS client.

33.2 `communicator.py` - HTTP + WebSocket (400 lines)

Component	Location	Description
<code>class Communicator</code>	<code>communicator.py:14</code>	HTTP client: <code>register</code> , <code>ping</code> , <code>submit_scan</code> , <code>check_status</code> , <code>get_scan_config</code> , <code>monitor_register</code> , <code>monitor_heartbeat</code> , <code>monitor_heartbeat_public</code> , and the offline queue. Retries 3 times with exponential backoff + jitter.

Component	Location	Description
register	communicator.py:136	POST /api/register with key, hostname, platform, version, fingerprint.
ping	communicator.py:145	POST /api/ping; response drives approval/trigger_scan logic.
submit_scan	communicator.py:159	POST /api/scan with the full scan payload.
monitor_heartbeat	communicator.py:189	Signed POST /api/monitoring/agent/heartbeat using the monitoring secret.
monitor_heartbeat_public	communicator.py:209	Unsigned POST /api/monitoring/agent/heartbeat-public (the variant actually used).
supports_websocket	communicator.py:224	Blocks .vercel.app domains; used to decide WS vs HTTP polling.
class WebSocketClient	communicator.py:235	Async client wrapping websockets. connect with ping_interval=20, ping_timeout=10, close_timeout=5 (communicator.py:354-359).
_run_loop	communicator.py:308	Runs send_task + recv_task; on completion reconnects with exponential backoff 2s to 60s; on HTTP rejection sets ws_unsupported (permanent HTTP fallback, communicator.py:319-331).
_authenticate	communicator.py:381	Sends the auth frame: {type, agent_id, secret, timestamp}.
_send_loop	communicator.py:393	Flushes queued messages while connected.
message handling	communicator.py:423-456	Handles auth_success (runs pending_commands), auth_failed, command, ping (queues pong), heartbeat_ack (logs).

33.3 config, discovery, runtime

Component	Location	Description
config.py	config.py:9	CONFIG_PATH = /client_config.json; LOCALHOST_URL=http://localhost:80.
load_config	config.py:13	Reads client_config.json (admin_url, scan_interval, auto_start).
get_admin_url	config.py:47	Resolution order: env ADMIN_SERVER_URL -> cached config -> cloud registry -> UDP discovery -> prompt (config.py:104 prompt_admin_url).
discover_admin	config.py:134	UDP broadcast on DISCOVERY_PORT 45000 to find a LAN admin.
discovery.py	discovery.py:24	discover_admin_url + register_server against the Supabase server_registry table; detect_public_ip (discovery.py:99). Note: embeds a service-role JWT.
runtime.py	runtime.py:9	is_frozen() and get_client_data_dir() (%APPDATA% on Windows).

33.4 identity: fingerprint and keys

Component	Location	Description
fingerprint.py	fingerprint.py:108	generate_fingerprint: SHA-256 of motherboard serial + CPU id + disk serial + MACs, truncated to 16 hex chars.

Component	Location	Description
<code>_get_motherboard_serial / _get_cpu_id / _get_disk_serial / _get_mac_addresses</code>	<code>fingerprint.py:33/48/74/90</code>	Platform-specific hardware queries (WMI on Windows, <code>/sys</code> and <code>/proc</code> on Linux, <code>system_profiler</code> on macOS).
<code>key_manager.py</code>	<code>key_manager.py:14</code>	<code>KEY_FILE = client_key.json</code> ; <code>generate_key (8 chars)</code> , <code>load_or_create_key</code> , <code>load_or_create_fingerprint</code> .

33.5 scanner.py and metrics.py

Component	Location	Description
<code>collect_all</code>	<code>scanner.py:51</code>	Runs the full inventory and returns the scan payload.
<code>hardware collectors</code>	<code>scanner.py:77-612</code>	<code>_get_processor</code> , <code>_get_ram</code> , <code>_get_storage</code> , <code>_get_motherboard</code> , <code>_get_os_info</code> , <code>_get_network</code> , <code>_get_gpu</code> , <code>_get_accounts</code> , <code>_get_software</code> , <code>_get_updates</code> , <code>_get_peripherals</code> , <code>_get_antivirus</code> (WMI/PowerShell on Windows, <code>/proc/lspci/lusb</code> on Linux, <code>system_profiler</code> on macOS).
<code>metrics.py</code>	<code>metrics.py:34-181</code>	<code>get_cpu_usage</code> , <code>get_ram_usage</code> , <code>get_disk_usage</code> , <code>get_uptime</code> , <code>get_network_connected</code> (TCP probe 8.8.8.8:53 / 1.1.1.1:53), <code>collect_metrics</code> .

33.6 events/ - monitors and dispatcher

Component	Location	Description
<code>dispatcher.py</code>	<code>dispatcher.py:28</code>	class <code>EventDispatcher</code> : batches events (5s window / max 50), then sends via WS -> signed HTTP -> public HTTP -> disk queue. <code>OFFLINE_QUEUE_DIR=/offline_events</code> (<code>dispatcher.py:22</code>); failed batches persisted as <code>batch_*.json</code> and replayed on next start.
<code>usb_monitor.py</code>	<code>usb_monitor.py:124</code>	class <code>USBMonitor</code> polls every 5 s; <code>get_usb_devices</code> via <code>WMI/lusb/system_profiler</code> (<code>usb_monitor.py:113</code>); emits <code>usb_device_connected/disconnected</code> events.
<code>file_monitor.py</code>	<code>file_monitor.py:184</code>	class <code>FileMonitor</code> + <code>CriticalFileHandler</code> (watchdog real-time, 2s debounce). <code>MONITOR_PATHS_*</code> (<code>file_monitor.py:26-42</code>), <code>CRITICAL_EXTENSIONS</code> (<code>file_monitor.py:44</code>), <code>CRITICAL_FILENAMES</code> (<code>file_monitor.py:51</code>).
<code>process_monitor.py</code>	<code>process_monitor.py:157</code>	class <code>ProcessMonitor</code> polls every 10 s; <code>SUSPICIOUS_NAMES</code> (mimikatz, psexec, netcat, meterpreter... at <code>process_monitor.py:130</code>), <code>SUSPICIOUS_PATHS_KEYWORDS</code> (<code>process_monitor.py:136</code>).
<code>software_monitor.py</code>	<code>software_monitor.py:186</code>	class <code>SoftwareMonitor</code> polls every 60 s; <code>_check_antivirus_status</code> (<code>software_monitor.py:118</code>), <code>_check_firewall_status</code> (<code>SecurityCenter2</code> , <code>software_monitor.py:148</code>), <code>AV_KEYWORDS</code> (<code>software_monitor.py:174</code>).

33.7 Startup sequence

1. `_log_crash` bootstraps logging.
2. `main()` prints the header and loads `client_config.json`.
3. `get_admin_url` resolves the admin URL (env -> cached -> cloud -> UDP -> prompt).
4. `_ensure_single_instance` (named mutex); if console mode and allowed, spawn background copy.

5. `_register_autostart` adds the agent to startup.
6. `heartbeat_loop` thread starts (ping + registration approval + public heartbeats).
7. `_start_event_monitors` starts USB/process/software/file monitors + dispatchers.
8. `start_websocket_client` connects `ws/agent/<id>/` with auth + reconnect loop.
9. `HeartbeatWatchdog` watches the heartbeat thread and restarts it if it dies.

34. Deep-Dive: Admin Server Source Walkthrough

The admin server is a Django project under `admin/`. It hosts four apps (`scanner_api`, `monitoring`, `intelligence`, `maintenance`) plus a project-config package (`django_admin`). Paths below are relative to `admin/` unless stated. File:line references allow every behavior to be located in the source.

34.1 Project configuration (`django_admin/`)

Component	Location	Description
settings.py	<code>django_admin/settings.py</code>	301 lines. Env config (<code>DJANGO_SECRET_KEY</code> , <code>DJANGO_DEBUG</code> , <code>DJANGO_ALLOWED_HOSTS</code> , <code>VERCEL</code> , <code>SCANNER_DATA_DIR</code>), <code>INSTALLED_APPS</code> , <code>middleware</code> , <code>REST_FRAMEWORK</code> , <code>CHANNEL_LAYERS</code> (InMemory, capacity 1000), JWT settings, database isolation, logging. See section 35.
urls.py	<code>django_admin/urls.py</code>	Mounts <code>api/</code> -> <code>scanner_api</code> , <code>api/monitoring/</code> , <code>api/maintenance/</code> , <code>api/intelligence/</code> ; <code>health/diag</code> ; all template pages; catch-all <code>_catch_all</code> (<code>django_admin/urls.py:13</code>).
asgi.py	<code>django_admin/asgi.py</code>	<code>ProtocolTypeRouter</code> : HTTP + WebSocket via Channels <code>AuthMiddlewareStack</code> + <code>monitoring.routing</code> ; falls back to plain ASGI on <code>ImportError</code> .
wsgi.py / manage.py	<code>django_admin/wsgi.py</code> , <code>django_admin/manage.py</code>	WSGI entry; <code>manage.py</code> injects an <code>AdminClient.admin</code> namespace (<code>manage.py:55 _setup_admin_client</code>).
main.py	<code>main.py</code>	Local admin launcher (279 lines): <code>migrate</code> , <code>superuser</code> , <code>ensure_admin_client</code> , UDP discovery (<code>main.py:98-122</code>), cloud registration (<code>main.py:69-95</code>), <code>--asgi daphne</code> option.
runtime.py	<code>runtime.py</code>	<code>get_data_dir</code> with legacy DB migration support.
scanner.py	<code>scanner.py</code>	Local scan collector (625 lines), re-exported by <code>scanner_api/scanner.py</code> .
api/index.py	<code>../api/index.py</code>	Vercel serverless WSGI bootstrap: <code>migrate --run-syncdb</code> , default superuser <code>admin/admin123</code> , skips <code>ensure_admin_client</code> when <code>VERCEL=1</code> .

34.2 `scanner_api` app (core)

Component	Location	Description
models.py	<code>scanner_api/models.py</code>	774 lines: <code>Company:11</code> , <code>ClientGroup:30</code> , <code>Client:44</code> , <code>ScanResult:110</code> , <code>AddonDevice:125</code> , <code>ActivityLog:142</code> , <code>Setting:169</code> , <code>AdministratorProfile:196</code> , <code>AuditLog:222</code> , <code>LoginHistory:258</code> , <code>LoginAttempt:281</code> , <code>DeviceFingerprint:295</code> , <code>Location:312</code> , <code>Department:344</code> , <code>Employee:370</code> , <code>EmployeeAssetAssignment:408</code> , <code>OrgAuditLog:426</code> , <code>AssetCategory:466</code> , <code>AssetVendor:487</code> , <code>Asset:507</code> , <code>AssetAssignment:678</code> , <code>AssetTransfer:700</code> , <code>AssetHistory:723</code> , <code>AssetDocument:771</code> .

Component	Location	Description
views.py	scanner_api/views.py	3863 lines. RegisterClientView:104, PingClientView:256, ClientListView:348, ClientDetailView:366, ScanConfigView:498, SubmitScanView:314, AuthLoginView:1071, AuthLogoutView:1154, AuthMeView:1166, AuthProfileView:1201, admin user/stats views:785-872, ChangePasswordView:939, ensure_admin_client:984, admin_self_scan:1020, admin_client_heartbeat_loop:1045, full org module:1425-2315, full asset module:2317-3541, GlobalSearchView:3442, ExecutiveAnalyticsView:3543, health_check:3763, diag_check:3799, SupabaseRegisterView:3814.
serializers.py	scanner_api/serializers.py	543 lines, 45 serializer classes covering clients, scans, org, assets.
urls.py	scanner_api/urls.py	130 lines: auth/JWT routes 44-57, org 61-95, assets 99-126, executive/global-search 128-129.
diff_utils.py	scanner_api/diff_utils.py	compute_scan_diff:135, _compare_peripherals:98, _compare_storage:116.
auth/security files	scanner_api/	jwt_auth.py (JWT auth backend), jwt_views.py (TokenObtainView:23, TokenRefreshView:80, TokenVerifyView:127, ApiKeyListView:167, ApiKeyDeleteView:236), api_key_auth.py (ApiKey model, SHA-256 hash), auth_utils.py (check_account_lock:31, record_login_attempt:52), validators.py (strong password + UA parsing), permissions.py (ROLE_HIERARCHY + 9 RBAC classes), session_auth.py (CsrfExemptSessionAuthentication), auth_backend.py (ResilientModelBackend), middleware.py (CookieAuthMiddleware:48, CompanyPrefixMiddleware:66, SessionTimeoutMiddleware:175, SecurityHeadersMiddleware:213), supabase_client.py (cloud registry + legacy clients).
templates.py	scanner_api/templates.py	305 lines: all HTML pages (dashboard, client detail, settings, login, signup, scans, audit log, org, assets).

34.3 monitoring app (device intelligence)

Component	Location	Description
models.py	monitoring/models.py	344 lines: DeviceMonitoringInfo:6, HardwareInventory:79, SoftwareInventory:108, DeviceHeartbeat:143, DeviceMetrics:172, DeviceHistory:209 (immutable), DeviceAlert:255, AgentVersion:294, AgentSecret:313.
scheduler_models.py	monitoring/scheduler_models.py	ScheduledScan:7, PendingScan:56 (immutable), ScanScheduleLog:94.
views.py	monitoring/views.py	1113 lines: AgentRegisterView:74, AgentHeartbeatView:157, HeartbeatPublicView:279, AgentInventoryView:361, AgentVersionCheckView:491, MonitorDashboardView:522, MonitorDeviceListView:643, MonitorDeviceDetailView:678, MonitorDeviceApproveView:891, MonitorDeviceBlockView:928, MonitorAlertListView:961, MonitorAlertActionView:986, MonitorBulkActionView:1013, MonitorTrendsView:1049, MonitorAgentVersionsView:1070, MonitorUnauthorizedSwView:1085.
consumers.py	monitoring/consumers.py	577 lines: AgentConsumer:15 (auth-first handshake then heartbeat/scan_result/event/status_update/pong handlers; dispatch table 74-80), DashboardConsumer:502.

Component	Location	Description
routing.py	monitoring/routing.py	ws/agent// and ws/dashboard/ routes.
event_bus.py	monitoring/event_bus.py	EventType enum:28, Event dataclass:74, EventBus singleton:105.
subscribers.py	monitoring/subscribers.py	353 lines: _on_hardware_change_alert:46, _on_software_change_alert:81, _on_device_offline_alert:141, _on_device_deleted:161, _broadcast_change:225, _broadcast_alert:245, _record_change_history:257, register_default_subscribers:294.
scheduler.py	monitoring/scheduler.py	get_scheduler:19, _sync_schedules_to_jobs:55, _add_schedule_job:72, _execute_schedule:139, _get_target_clients:211, add_or_update_schedule:236.
scheduler_views.py	monitoring/scheduler_views.py	ScheduleListView:22, ScheduleDetailView:103, ScheduleToggleView:171, ScheduleHistoryView:189, SchedulerStatusView:221, PendingScansView:234, AgentPendingScansView:275 (GET lists, POST acknowledges).
AI layer	monitoring/	health.py (calculate_health_score:4, determine_health_level:74), alerts.py (check_and_create_alerts:11, check_offline_alerts:35), change_detection.py (detect_hardware_changes:17, detect_software_changes:96), anomaly_detection.py (AnomalyDetector:59), feature_store.py (FeatureStore:24), predictive.py (PredictiveEngine:59).
security.py	monitoring/security.py	generate_api_secret:13, compute_signature:18, verify_signature:27, verify_timestamp:42, RateLimiter:63, authenticate_agent:92, validate_fingerprint_match:131, get_client_ip:153.
signals_helpers.py	monitoring/signals_helpers.py	notify_agent:45, broadcast_to_dashboard:54, broadcast_alert:81, send_device_update:108.
reports.py / report_views.py	monitoring/reports.py	Fleet/device/alerts PDF+CSV report builders; report_views.py exposes 6 csrf_exempt views (ReportFleetPDFView:22, ReportDevicePDFView:34, ReportAlertsPDFView:48).
urls.py	monitoring/urls.py	55 lines: agent endpoints, dashboard/devices, alerts, reports, schedules, agent-versions, unauthorized-software.

34.4 intelligence app (insights)

Component	Location	Description
models.py	intelligence/models.py	Alert:6, AlertHistory:77, AlertRule:100, Notification:151, NotificationPreference:189, Report:227, ScheduledReport:293, AuditLogEntry:337 (immutable), ComplianceLog:417, DashboardAnalytics:465, RetentionPolicy:486.
views.py	intelligence/views.py	830 lines, 33 views: IntelligenceDashboardView:55, AlertListView:107, AlertActionView:182, AlertsRunChecksView:324, NotificationListView:341, NotificationPreferenceView:442, ReportListView:467, ReportGenerateView:494, AuditLogListView:613, AuditLogExportView:671, ComplianceDashboardView:765, IntelligenceSettingsView:818.
alerts.py	intelligence/alerts.py	create_alert:17 (SHA-256 dedup), acknowledge_alert:49, resolve_alert:68, escalate_alerts:130, run_alert_checks:155.

Component	Location	Description
notifications.py	intelligence/notifications.py	create_alert_notifications:55.
audit.py	intelligence/audit.py	log_audit_entry:7 + 10 helpers.
reports.py	intelligence/reports.py	generate_report:15; implements 9 of the 25 report types (see section 37).
urls.py	intelligence/urls.py	66 lines incl. audit detail:53, compliance:57-59, retention:62, settings:65.

34.5 maintenance app (asset care)

Component	Location	Description
models.py	maintenance/models.py	MaintenanceRecord:12, MaintenanceHistory:134, MaintenanceDocument:162, WarrantyRecord:185, DowntimeRecord:273, SoftwareLicense:323, LicenseAssignment:446, LicenseHistory:488, ComplianceRecord:520, MaintenanceAlert:566.
views.py	maintenance/views.py	1400 lines, 29 views: MaintenanceListView:95, MaintenanceDetailView:197, MaintenanceStatusView:252, MaintenanceApprovalView:305, MaintenanceDocumentUploadView:335, WarrantyListView:368, DowntimeListView:456, DowntimeEndView:501, LicenseListView:520, LicenseDetailView:613, LicenseAssignView:688, ComplianceListView:777, MaintenanceDashboardView:870, analytics 966-1260, exports 1261-1345, AlertCheckView:1346.
alerts.py	maintenance/alerts.py	check_and_generate_alerts:15 (overdue, due within 7 days, warranty expiring 30/60/90 days, warranty expired, license expiring 7/30/60 days, license expired); ack/resolve/dismiss helpers.
serializers.py	maintenance/serializers.py	298 lines, 22 classes incl. license-key masking (_mask_license_key).
urls.py	maintenance/urls.py	45 lines: all maintenance, warranty, downtime, license, compliance, alerts, dashboard, analytics, export routes.

34.6 Management commands and operations

Component	Location	Description
offline_detector.py	monitoring/management/commands/offline_detector.py	30s loop; flips devices to offline after stale_threshold_seconds (default 120s) and creates a device_offline DeviceAlert.
health_checker.py	monitoring/management/commands/health_checker.py	300s; recalcs health via calculate_health_score + top-200 software.
alert_checker.py	monitoring/management/commands/alert_checker.py	60s; runs check_offline_alerts at 300/900/1800s thresholds.
stale_checker.py	monitoring/management/commands/stale_checker.py	Marks stale clients offline.

Component	Location	Description
scan_local.py	monitoring/management/commands/scan_local.py	Runs a local scan and stores a ScanResult.
clear_data.py	monitoring/management/commands/clear_data.py	Wipes non-Django tables and recreates the admin client.
run_alert_checks.py	intelligence/management/commands/run_alert_checks.py	Runs intelligence alert checks. Known issue: passes escalate_only to run_alert_checks(), which takes no arguments.

35. Deep-Dive: Configuration and Environment Reference

35.1 Environment variables (admin/django_admin/settings.py)

Field	Type	Purpose
DJANGO_SECRET_KEY	str	Django secret key; signs sessions and JWTs.
DJANGO_DEBUG	bool	Debug mode; default True.
DJANGO_ALLOWED_HOSTS	str	Default "".
VERCEL	bool	Presence switches to serverless behavior (IS_VERCEL).
SCANNER_DATA_DIR	path	Overrides the data directory (default: APPDATA / ~/.scanner).
DATABASE_URL	url	Used only on Vercel; Supabase pooler rewritten to port 6543 with sslmode=require.
SUPABASE_URL / SUPABASE_KEY	url / key	Cloud registry (server_registry) access.
ADMIN_SERVER_URL	url	Client side: forces the admin URL, bypassing discovery.

35.2 Django settings groups

INSTALLED_APPS	Django defaults + rest_framework, corsheaders, daphne@0/channels@5 (only when not on Vercel), scanner_api, monitoring, maintenance, intelligence.
MIDDLEWARE	Security -> WhiteNoise -> CORS -> Session -> Common -> CSRF -> Auth -> CompanyPrefixMiddleware -> SessionTimeoutMiddleware -> SecurityHeadersMiddleware.
REST_FRAMEWORK	Default authentication CsrfExemptSessionAuthentication (settings.py:63-69); no global permissions; pagination handled manually in views.
CORS_ALLOW_ALL_ORIGINS	True (settings.py:61).
SESSION_COOKIE_AGE	30 days; cookie name scanner_auth signed with TimestampSigner (salt scanner-auth-cookie).
AUTHENTICATION_BACKENDS	ResilientModelBackend first (settings.py:250-253).
SESSION_ENGINE	DB-backed locally; signed cookies on Vercel; secure cookies on Vercel.

CHANNEL_LAYERS	InMemoryChannelLayer capacity 1000 in both local and Vercel branches (settings.py:94-111); WS_DASHBOARD_GROUP="dashboard", WS_HEARTBEAT_INTERVAL=30.
MONITORING_THRESHOLD S	offline alert thresholds 300/900/1800 s; stale_threshold_seconds default 120.
SCHEDULER_CONFIG	coalesce, max_instances=1, misfire_grace=300 (settings.py:122-128).
JWT	secret=SECRET_KEY, HS256, access 60 min, refresh 7 days, issuer "system-scanner-pro".
STATIC	WhiteNoise with WHITENOISE_USE_FINDERS=True (no collectstatic needed on Vercel).

35.3 Database and storage modes

Component	Description
Local / VPS (non-Vercel)	Always SQLite at /scanner.db (settings.py:138-227); DB_CONN_MAX_AGE 10.
Vercel with DATABASE_URL	Supabase/PostgreSQL via dj-database-url; pooler port rewritten to 6543, sslmode=require, connect_timeout=10, keepalives, DB_CONN_MAX_AGE=0.
Vercel without DATABASE_URL	Warning logged; ephemeral SQLite at /tmp/vercel.db.
Data directory	SCANNER_DATA_DIR or APPDATA (runtime.py get_data_dir); holds scanner.db, admin_config.json, client keys.

35.4 Client configuration

Component	Description
client/client_config.json	admin_url, scan_interval (3600), auto_start (true).
client/client_key.json	registration_key + fingerprint (created by key_manager).
CLI flags	--silent (suppress console UI), --rescue (rescue/background copy).
Admin URL resolution	env ADMIN_SERVER_URL -> cached config -> cloud registry -> UDP discovery (port 45000) -> prompt.
Events	Monitors poll at fixed intervals (USB 5s, process 10s, software 60s); events batch 5s/50 max; offline queue at /offline_events.
WebSocket client	connect ping_interval=20 s, ping_timeout=10 s, close_timeout=5 s; reconnect backoff 2s -> 60s; Vercel domains force HTTP polling.

36. Deep-Dive: Event Bus, Change Detection and Intelligence Pipeline

This section traces how a metric, a scan diff, or a monitor event becomes a history row, a DeviceAlert, an intelligence Alert, a notification, and a dashboard message. Everything is event-driven through the monitoring EventBus (admin/monitoring/event_bus.py) and its subscribers (admin/monitoring/subscribers.py).

36.1 Event catalog (EventType)

Field	Type	Purpose
hw_component_added / removed / modified	Hardware	Hardware inventory change detection.
sw_installed / sw_removed / sw_version_changed	Software	Software inventory change detection.
sw_unauthorized / sw_antivirus_removed	Software	Unauthorized software and AV removal.
health_level_changed / health_score_updated	Health	Health level or score transitions.
alert_created / acknowledged / resolved / dismissed	Alerts	Intelligence alert lifecycle.
device_registered / status_changed / approved / blocked / deleted / offline / online	Lifecycle	Device lifecycle transitions.
agent_version_changed	Agent	Agent version changed.
heartbeat_received	Heartbeat	A heartbeat was stored.
scan_completed / scan_scheduled	Scan	Scan execution events.

36.2 Subscribers (what each event does)

Component	Location	Description
_on_hardware_change_alert	subscribers.py: 46	Creates a DeviceAlert for hardware changes.
_on_software_change_alert	subscribers.py: 81	Creates a DeviceAlert for software changes.
_on_device_offline_alert	subscribers.py: 141	Creates a DeviceAlert when a device goes offline.
_on_device_deleted	subscribers.py: 161	Handles device deletion (cascade cleanup + dashboard notify).
_broadcast_change	subscribers.py: 225	Forwards every event to the dashboard group and to device subscribers.
_record_change_history	subscribers.py: 257	Writes DeviceHistory rows for changes.
register_default_subscribers	subscribers.py: 294	Wires all subscribers to the bus (called at app startup).

36.3 Health scoring (admin/monitoring/health.py)

calculate_health_score (health.py:4) blends five sub-scores with fixed weights: CPU 25%, RAM 25%, disk 20%, connectivity 15%, software 15%. Each sub-score maps usage to points (e.g. cpu/ram 100 points at 70% or below, linear decline to 50 points at 85%, minimum 10 above that; disk 100 at 80% or below, 30 at 95%, 0 above; connectivity 100/0; software 100 with an antivirus present, 60 without). determine_health_level (health.py:74) returns healthy >= 80, warning >= 50, critical below.

36.4 Anomaly detection (anomaly_detection.py)

Field	Type	Purpose
Threshold	Static rules	cpu/ram >= 95% critical, >= 85% warning; disk >= 98% critical, >= 90% warning; disk_free_gb <= 2 critical, <= 5 warning (confidence 0.95).
Z-score	Statistical	z > 2.5 (default z_threshold) flags; severity info/z>3 warning/z>4 critical; needs >= 10 samples.
IQR	Outlier	Outside q1-1.5*IQR / q3+1.5*IQR; needs >= 20 samples; confidence 0.7.
Trend	Moving average	Short (5) vs long (20) window; > 50% change flags rising/falling trend.
History	Lookback	Per-device metric history trimmed to lookback_hours=168 (7 days).

36.5 Predictive engine (predictive.py)

PredictiveEngine (predictive.py:59) uses linear regression and exponential smoothing on feature-store time series (no external ML). It predicts disk-full time (when usage reaches 100%), device failure risk, and maintenance windows; each prediction carries metric, value, unit, confidence, timeframe_hours, description, and recommended_action.

36.6 Change detection (change_detection.py)

detect_hardware_changes (change_detection.py:17) and detect_software_changes (change_detection.py:96) diff new inventory snapshots against stored ones. Storage-related changes are rated critical; other component changes are warning/info. Results publish the hw_*/sw_* events above and write DeviceHistory rows.

36.7 Maintenance and license alert rules (maintenance/alerts.py)

Field	Type	Purpose
maintenance_overdue	critical	MaintenanceRecord status in Scheduled/In Progress/Waiting Parts and due_date before today.
maintenance_due	warning	Status Approved/Scheduled and scheduled_date within the next 7 days.
warranty_expiring	warning/info	Warranty end within 30 (warning), 60, or 90 (info) days.
warranty_expired	warning	Warranty end before today; warranty status flips to Expired.
license_expiration	critical/warning/info	License expiry within 7 (critical), 30 (warning), or 60 (info) days.
license_expired	critical	License expiration before today.

36.8 Intelligence alert lifecycle (intelligence/alerts.py)

create_alert (alerts.py:17) computes a SHA-256 dedup_hash from module, category, title, and source_object_id; if an open/acknowledged alert with the same hash exists it is refreshed instead of duplicated. escalate_alerts (alerts.py:130) raises escalation_level over time. run_alert_checks (alerts.py:155) drives the rule engine. Alerts write AlertHistory on every transition, notify users via create_alert_notifications (notifications.py:55), and can auto-resolve per AlertRule.auto_resolve_minutes.

37. Deep-Dive: Reports, Analytics and Dashboards

37.1 Intelligence report types

The Report model defines 25 report types (intelligence/models.py:228-254). The generator (intelligence/reports.py generate_report, reports.py:15) currently implements 9 of them:

Field	Type	Purpose
asset_inventory	reports.py:27	Full asset inventory export.
asset_assignment	reports.py:44	Asset-to-employee assignments.
expiring_licenses	reports.py:60	Licenses expiring soon.
upcoming_maintenance	reports.py:74	Scheduled maintenance ahead.
device_health	reports.py:87	Per-device health scores and levels.
compliance_report	reports.py:98	Compliance findings.
monthly_summary	reports.py:110	Monthly aggregate summary.
software_inventory	reports.py:134	Installed software across the fleet.
audit_report	reports.py:144	Audit log extract.

The remaining 16 report types (asset_lifecycle, asset_utilization, online_devices, hardware_change, maintenance_cost, downtime_analysis, vendor_performance, seat_utilization, cost_analysis, unauthorized_software, security_violations, device_risk, department_performance, top_problematic_assets, cost_overview, compliance_summary) are selectable and stored but do not yet have a generator branch - requesting one returns an unsupported-type result.

37.2 Monitoring reports (PDF and CSV)

Field	Type	Purpose
GET /api/monitoring/reports/fleet/pdf	report_views.py:22	Fleet summary as a PDF attachment.
GET /api/monitoring/reports/fleet/csv	-	Fleet summary CSV.
GET /api/monitoring/reports/device/pdf	report_views.py:34	Single device report PDF (404 JSON if missing).
GET /api/monitoring/reports/device/csv	-	Single device report CSV.
GET /api/monitoring/reports/alerts/pdf	report_views.py:48	Alerts report PDF.
GET /api/monitoring/reports/alerts/csv	-	Alerts report CSV.

37.3 Maintenance exports and analytics

Field	Type	Purpose
GET /api/maintenance/maintenance/export	views.py:1261	Maintenance records as CSV / XLSX / PDF.

Field	Type	Purpose
GET /api/maintenance/licenses/export	views.py:1301	License list export.
GET /api/maintenance/dashboard	views.py:870	Aggregated counts + cost.
GET /api/maintenance/analytics/cost-trend	views.py:966	Maintenance cost over N months.
GET /api/maintenance/analytics/vendor-performance	views.py:1001	Vendor reliability metrics.
GET /api/maintenance/analytics/failure-rate	views.py:1026	Asset failure rates.
GET /api/maintenance/analytics/downtime	views.py:1052	Downtime analytics.
GET /api/maintenance/analytics/license-dashboard	views.py:1122	License counts, seats, utilization, cost, compliance issues.
GET /api/maintenance/analytics/license-utilization	views.py:1186	Per-license seat utilization percentages.
GET /api/maintenance/analytics/license-cost	views.py:1209	License cost breakdown.

37.4 Dashboards and analytics endpoints

Component	Location	Description
Monitoring dashboard	/api/monitoring/dashboard (views.py:522)	Totals for online/offline/blocked/pending devices, alerts, and heartbeats in the last 24h.
Fleet trends	/api/monitoring/trends (views.py:1049)	Trend data over N days.
Maintenance dashboard	/api/maintenance/dashboard (views.py:870)	Maintenance, warranty, license, compliance, and alert aggregates.
Asset dashboard	/api/assets/dashboard	Asset counts, categories, and states.
Executive analytics	/api/executive-analytics (views.py:3543)	Cross-module KPIs (assets, licenses, alerts, maintenance).
Global search	/api/global-search (views.py:3442)	Search across clients, assets, employees, locations, departments.
Intelligence dashboard	/api/intelligence/dashboard (views.py:55)	Alert/notification/compliance/report overview.

Component	Location	Description
DashboardAnalytics snapshots	intelligence/mo dels.py:465	Periodic 9-counter KPI snapshots (total/open/critical alerts, notifications_today, reports_generated, security_violations, audit_events_today, compliance_violations, pending_notifications).

38. Deep-Dive: Troubleshooting Matrix

Common symptoms, their verified root causes (with code locations), and the fix. If a symptom is not listed, use the file map (sections 33-34) to locate the responsible component and check the relevant endpoint (section 30) and WebSocket message (section 31).

Component	Location	Description
Agent never appears online	Heartbeats fail or are throttled	Verify the agent reaches POST /api/monitoring/agent/heartbeat-public (client/main.py:407-412, monitoring/views.py:279). Check firewall/port 80 and the offline_detector stale threshold (default 120s).
Agent shows pending forever	Client not approved	POST /api/approve with the client key (scanner_api/views.py:202) or approve from the panel. Approval is never auto-granted for deleted clients.
WS connection drops in a loop	Upgrade rejected (serverless) or auth timestamp expired	Check ws_unsupported HTTP fallback (client/communicator.py:319-331); on Vercel the agent intentionally polls HTTP. Timestamp check is 300s max age (monitoring/security.py:42).
Agent offline queue grows	HTTP POST fails repeatedly	Inspect /offline_events/batch_*.json; flushed when consecutive failures return to 0 (client/main.py:383-387).
Scan requested but nothing happens	Agent offline / schedule targets	Online agents get WS scan_now; offline agents get PendingScan rows fetched over /agent/pending-scans (scheduler_views.py:275). Confirm scan_enabled on the client (scanner_api/views.py:498).
Dashboard shows stale device data	WebSocket reconnect or InMemory layer reset	Channel layer is InMemory (settings.py:94-111) - data is not retained across server restarts. Refresh triggers the offline_detector/heartbeat paths.
new_alert toasts never fire on monitoring page	alert_created vs new_alert mismatch	The frontend listens for alert_created (monitoring.js:33) but the server only emits new_alert (consumers.py:496). Expected behaviour: WS events still deliver device_event/new_alert.
Report returns unsupported type	Not implemented in generator	Only 9 of 25 report types have branches in intelligence/reports.py (see section 37.1).
run_alert_checks crashes from CLI	Signature mismatch	intelligence/management/commands/run_alert_checks.py passes escalate_only, but run_alert_checks() takes no args (intelligence/alerts.py:155).
Version check reports old client	Version skew	client/main.py VERSION="1.6.1" vs the exe resource version 3.0.0.0; AgentVersion table is authoritative for update decisions.
Scheduled reports never generate	No background executor	ScheduledReport is CRUD-only; nothing computes next_run or runs reports automatically.
Admin self-scan behaves unexpectedly	ensure_admin_client / admin_client_heartbeat_loop	Located at scanner_api/views.py:984 and :1045; skipped when VERCEL=1 (api/index.py).

Verified repository-level issues: README claims Mangum serverless but api/index.py is plain WSGI; README claims Redis for prod but settings.py uses InMemoryChannelLayer everywhere; service-role JWT is embedded in client/discovery.py and real Supabase keys exist in .env* files (rotate before production); admin/admin_config.json hosts 0.0.0.0 while

39. FAQ

How does an agent get approved?

The agent registers (POST /api/register), the server creates a Client with status pending, and the admin approves via POST /api/approve or the panel. The agent's heartbeat loop watches the approval flag and only scans once approved.

What is the difference between the two registration flows?

scanner_api /api/register manages the core Client row (hostname, fingerprint, scans). monitoring /api/monitoring/agent/register creates the DeviceMonitoringInfo + AgentSecret (HMAC secret) pair used for live metrics and health. The agent runs both.

Why does the agent use HTTP heartbeats instead of WebSocket?

The WS consumer fully implements heartbeat, but the current agent sends metrics over POST /api/monitoring/agent/heartbeat-public every ~5s (client/main.py:407-412). The WS channel carries events and scan_now commands.

How are offline clients handled during scheduled scans?

The scheduler sends scan_now over WS to online agents and creates PendingScan rows for offline ones (monitoring/scheduler.py:139-208). Agents fetch pending scans over HTTP (/agent/pending-scans) and acknowledge them as executed or failed.

Where is the scan diff computed?

scanner_api/diff_utils.py compute_scan_diff (diff_utils.py:135) compares the newest ScanResult against the previous one; the client detail page highlights the changes.

Can the admin server run without a database?

No. Non-Vercel uses SQLite at /scanner.db; on Vercel you must provide DATABASE_URL (Supabase), otherwise it falls back to an ephemeral /tmp/vercel.db.

What ports does the agent use?

HTTP on port 80 (admin), UDP discovery on port 45000, and WebSocket ws://HOST/ws/agent/.

How do I reset a locked admin account?

Login lockout is enforced by auth_utils.py check_account_lock (auth_utils.py:31). Reset via Django shell (python manage.py shell) by clearing the failed-attempt counters or setting the user active, then change the password through /api/auth/change-password.

Why does the PDF say some WebSocket features are dormant?

Section 31 documents the protocol as implemented. heartbeat over WS, ping/pong, status_update, and the pending_commands channel are implemented on the server but not actively driven by the current agent; documenting them lets you find the exact lines if they are enabled later.

40. Glossary

Agent	The installed client process (client/) running on a managed machine.
Admin server	The Django server (admin/) that aggregates agents, devices, alerts, and reports.
Registration key	The 8-character Client.registration_key that identifies a client.
Fingerprint	16-hex-char SHA-256 hash of motherboard/CPU/disk/MAC identity (client/fingerprint.py).
AgentSecret	The per-agent HMAC secret issued by /api/monitoring/agent/register.
Health score	0-100 composite of CPU/RAM/disk/connectivity/software (admin/monitoring/health.py).
Health level	healthy (>=80) / warning (>=50) / critical (<50).
Event bus	The pub/sub singleton (admin/monitoring/event_bus.py) that fans events out to subscribers and dashboards.
DeviceAlert	Monitoring-scope alert (monitoring/models.py:255) with active/acknowledged/resolved/dismissed states.
Intelligence Alert	Cross-module alert (intelligence/models.py:6) with dedup, escalation, and history.
PendingScan	An offline queue entry created when a scheduled scan cannot reach an agent.
Scan diff	The added/removed/changed comparison between two ScanResult payloads.
Online/offline	Driven by last heartbeat vs stale_threshold_seconds (default 120s).
HMAC	SHA-256 HMAC over the raw request body used to sign agent HTTP calls (monitoring/security.py).
JWT	HS256 tokens (access 60 min, refresh 7 days) for admin API access.
API key	Hashed admin keys (token_hex(32)) rate limited at 60/min.
Session cookie	scanner_auth signed cookie (30 days with remember_me).
Company	Tenant root for most tables; scoped by CompanyPrefixMiddleware.
Vercel mode	Serverless runtime: HTTP polling only, ephemeral DB without DATABASE_URL.
Manual scan	An on-demand scan triggered by the admin (WS scan_now or HTTP trigger_scan).
Scheduled scan	A scan fired by the APScheduler according to a ScheduledScan.
Compliance	Records/findings for ISO 27001, ITIL, SOC 2, internal, and GDPR frameworks.
Maintenance record	Work-order style record with approval and status workflow (MNT%06d ids).
Software license	Seat-tracked entitlement (LIC%06d ids) with expiration and utilization.
Retention policy	Per-scope (alerts/notifications/reports/audit_logs/compliance_logs) retention periods.
Dashboard group	The Channels group "dashboard" receiving real-time broadcast messages.

41. Appendix: Quick Reference

41.1 Ports and protocols

Field	Type	Purpose
80 / 443	HTTP(S)	Admin server REST API and pages (agent uses port 80 by default).

Field	Type	Purpose
45000	UDP	LAN discovery broadcast (client DISCOVERY_PORT, main.py:134; admin main.py:98-122).
ws/agent/<id> / ws/dashboard	WebSocket	Agent and dashboard sockets (monitoring/routing.py).
6543	PostgreSQL	Supabase pooler port used on Vercel (settings.py database isolation).

41.2 Key constants

Agent VERSION	1.6.1 (client/main.py:135); exe resource says 3.0.0.0.
Scan interval default	3600 s (Client.scan_interval).
Session lifetime	30 days (scanner_auth cookie; remember_me).
JWT access / refresh	60 minutes / 7 days, HS256, issuer system-scanner-pro.
HMAC timestamp max age	300 s (monitoring/security.py:42).
API key rate limit	60 requests/minute.
Stale threshold	120 s default (Setting stale_threshold_seconds).
Offline alert thresholds	300 / 900 / 1800 s (monitoring/alerts.py check_offline_alerts).
Health weights	cpu .25, ram .25, disk .20, connectivity .15, software .15.
Health levels	healthy >= 80, warning >= 50, critical < 50.
Anomaly defaults	z=2.5, IQR x1.5, lookback 168 h.
Maintenance id prefix	MNT%06d; warranty WAR%06d; license LIC%06d; asset AST%06d.
Warranty/license alert tiers	warranty 30/60/90 days; license 7/30/60 days.

41.3 HTTP status conventions

Field	Type	Purpose
200 / 201	Success	200 for reads/actions; 201 for created resources (register, scan, schedules, maintenance, licenses).
204	No content	DELETE /api/auth/api-keys/<id> and similar.
400	Bad request	Invalid or missing JSON fields.
401	Unauthorized	Bad credentials; includes attempts_remaining on login/token failure.
403	Forbidden	Disabled account, locked account, or RBAC denial.
404	Not found	Missing client/device/alert/etc.; JSON detail body.
409	Conflict	Duplicate registration key or fingerprint collision.

41.4 Where-is-it index

Client registration		POST /api/register (scanner_api/views.py:104); agent client/communicator.py:136.
Monitoring registration	agent	POST /api/monitoring/agent/register (monitoring/views.py:74); HMAC security.py:92.
Heartbeats		HTTP public (monitoring/views.py:279; client/main.py:407-412); WS heartbeat consumers.py:147.

WebSocket auth	AgentConsumer._handle_auth (consumers.py:91-145); client _authenticate communicator.py:381.
Scheduled scans	Scheduler jobs (monitoring/scheduler.py:139); pending scans scheduler_views.py:275.
Scan diffs	scanner_api/diff_utils.py:135.
Health scoring	monitoring/health.py:4.
Anomaly detection	monitoring/anomaly_detection.py:59.
Predictive analytics	monitoring/predictive.py:59.
Event bus	monitoring/event_bus.py:105 (singleton); subscribers.py:294.
Intelligence alerts	intelligence/alerts.py:17; dedup + escalation; AlertHistory on transitions.
Maintenance alerts	maintenance/alerts.py:15.
Dashboard broadcasts	monitoring/consumers.py:472-499 and signals_helpers.py:54.
Reports	intelligence/reports.py:15; monitoring/reports.py; exports maintenance/views.py:1261.
Admin auth/session	scanner_api/views.py:1071 (login), jwt_views.py:23 (JWT), api_key_auth.py (API keys), auth_utils.py:31 (logout).
Deployment	admin/main.py (local), api/index.py (Vercel), vercel.json, build_client.py (agent exe).

Every claim in this document was verified against the source tree at the referenced file:line. If something misbehaves, start from section 38 (troubleshooting) or the where-is-it index above, open the referenced file, and trace the flow.